FEC/ARQ Unified Correction Symbol Model

Pierre Kisters

July 2026

Contents

Abstract

Data traversing lossy multipath channels (WiFi, LTE, satellite) has specific requirements: some needs low latency (VoIP), some needs high throughput (file transfer), some tolerates partial loss (sensor telemetry). Each available path has different characteristics — bandwidth, latency, and loss rate. The goal of this model is to **optimally use each path or combination of paths to satisfy the constraints of the data sent over them**, without ad-hoc tuning knobs.

Three properties — bandwidth, tail latency, and reliability — form a triangle: fix any two, the third is determined by the channel. The protocol hint (Realtime, Bulk, etc.) selects which two

to fix. The model computes the optimal correction rate, taper schedule, and path allocation from measured channel parameters — no magic offsets or manual thresholds.

The core mechanism is the **correction symbol** — a unified concept that subsumes both FEC repair symbols (proactive, flexible) and ARQ source retransmits (reactive, immediately decodable). A single taper function, shaped by the Gilbert-Elliott channel model's burst survival function, controls correction density over time. A probabilistic per-slot decision based on loss confidence P_lost(t) — driven by time since send — determines whether each correction is a retransmit or a repair.

The optimal correction rate r* = e/(1-e) + z_delta x sqrt(e x s2_burst / (W(1-e))) combines the information-theoretic minimum with a burst-variance- corrected tail margin. Copa congestion control is recommended over BBR for its taper-compatible rate oscillation (no FEC protection gaps from ProbeRTT).

For multipath, each path runs its own Copa, taper, and GE estimator with a unified symbol stream preserving interleaved burst protection. A global correction deficit tracks outstanding recovery needs across paths. The scheduler adjusts per-path source/correction ratios using an interpolated latency/bandwidth objective, with QoS priority cascading for mixed traffic.

ACK-based feedback with SACK replaces the NACK mechanism — the same proven approach TCP has used for 40 years, with sender-side loss inference from ACK absence.

---

1. System Model

1.1 Notation

Symbol	Meaning	Unit	Example
ε	Average channel loss rate	probability (0-1)	0.025 (2.5% WiFi)
p	P(Good → Bad) in GE model	probability (0-1)	0.013
q	P(Bad → Good) in GE model	probability (0-1)	0.5
B	Mean burst length = 1/q	symbols (count)	2.0
W	Encoder window size	symbols (count)	50
RTT	Round-trip time	seconds	0.050 (50ms)
P_lost(t)	P(symbol lost given no ACK after time t)	probability (0-1)	≈2ε at t=SRTT
t_fec	FEC recovery time = m × (1+r) / (r × (1-ε)) × t_sym	seconds	0.0013 (1.3ms)
t_sym	Symbol transmission time = symbol_size / throughput	seconds	0.000096 (96μs at 100Mbps)
SRTT	Smoothed RTT estimate	seconds	0.050 (50ms)
RTTVAR	RTT variance estimate (standard deviation)	seconds	0.005 (5ms)
r	Total correction rate	ratio (corrections/source)	0.08 (8%)

Symbol	Meaning	Unit	Example
$\tau(t)$	Taper function: correction density at offset t	ratio (corrections/symbol)	0.04
A	Taper amplitude (scaling factor)	ratio (corrections/symbol)	0.04
P_fec	Probability a lost symbol is FEC-recovered	probability (0-1)	0.95
δ	Tail latency target: P(late delivery) ≤ δ	probability (0-1)	1e-4
ρ	Reliability target: P(symbol delivered) ≥ ρ	probability (0-1)	1.0 (100%)
T_cut	Taper cutoff time (stop corrections after this)	seconds	∞ (100% reliability)
B_max	99.99th percentile burst length = ceil(ln(0.0001)/ln(1-q))	symbols (count)	14 (WiFi, q=0.5)
buffer_max	Max retransmit buffer size (derived)	symbols (count)	700 (WiFi 100Mbps)
L_prop	Propagation delay (base latency)	seconds	0.025 (25ms)
L_arq	ARQ recovery latency (time from loss to retransmit arrival)	seconds	0.075-0.100
σ^2_burst	Burst variance inflation factor	dimensionless	2.9
z_δ	Standard normal quantile for δ	dimensionless	3.72 (for δ=1e-4)
ε_burst	Current channel loss rate (fast EWMA or GE state, optional — see C.6)	probability (0-1)	0.5 (during burst)
ε_codec	Codec decode overhead	ratio (0-1)	0.01 (RaptorQ)

1.2 Glossary

Correction symbol — any symbol sent to recover lost data. Either a source retransmit (exact copy, immediate decode) or a repair symbol (FEC linear combination, needs decoder). The taper function controls how many; P_lost(t) controls which type. See Section 5.2.

Repair symbol — a correction symbol generated by the FEC encoder. It is a random linear combination of source symbols in the encoder window over GF(256). The receiver feeds it to the decoder. See Section 3.2.

Source retransmit — a correction symbol that is an exact copy of a previously-sent source symbol from the retransmit buffer. The receiver can use it immediately without decoding. See Section 3.3.

Encoder window — the W most recent source symbols tracked by the FEC encoder. Repair symbols are linear combinations of symbols in this window. When a symbol is evicted (window slides forward), it can no longer be covered by new repair symbols. See Section 3.2.

Taper function τ(t) — specifies the correction symbol density at time offset t from a source symbol. Shaped to match the channel's burst-length distribution: τ(t) = A × (1-q)^t for Gilbert-Elliott channels. See Section 4.

Systematic code — an FEC scheme where source symbols are sent as-is (not encoded). The decoder is only invoked when at least one source symbol is lost. If nothing is lost, the data passes through without any decoding cost.

Codec overhead — the extra symbols the decoder needs beyond the number of losses, because not all random linear combinations are linearly independent. E.g., RaptorQ needs ~1% extra, METTLE needs ~15%. See Section 9.

SACK (Selective Acknowledgement) — an extension to cumulative ACK that reports out-of-order received blocks beyond the cumulative point. Lets the sender know exactly which symbols arrived despite gaps. See Section 6.1.

GE states (Good/Bad) — the two states of the Gilbert-Elliott channel model. In the simplified model: Good = no loss, Bad = total loss. Transition probabilities p (Good->Bad) and q (Bad->Good) determine burst behavior. See Section 2.

P_lost(t) — probability that a specific symbol was lost, given no ACK received after time t. The per-slot mixing probability for repair vs retransmit. See Section 3.4.

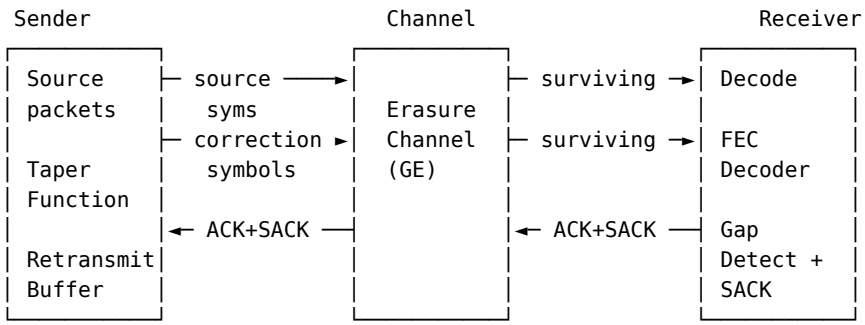
P_fec — probability that a lost symbol is recovered by FEC (proactive correction) before ARQ retransmit kicks in. See Section 8.2.

P_arq — probability that ARQ retransmit succeeds, given FEC failed. Derived from reliability target: P_arq = 1-(1-ρ)/(ε×(1-P_fec)). See Section 6.3.

z_delta — standard normal quantile at probability (1-δ). Controls the tail margin in the repair rate formula. See Section 8.4.

σ²_burst (s2_burst) — burst variance inflation factor for the GE channel: 1+2(1-p-q)/(p+q). Corrects the iid normal approximation for burst-correlated losses. See Section 8.3.

1.3 Components



The system provides **reliable** delivery. Three properties form a triangle — bandwidth, tail latency, and reliability. Fix any two, the third is determined by the channel. The protocol hint selects which two to fix.

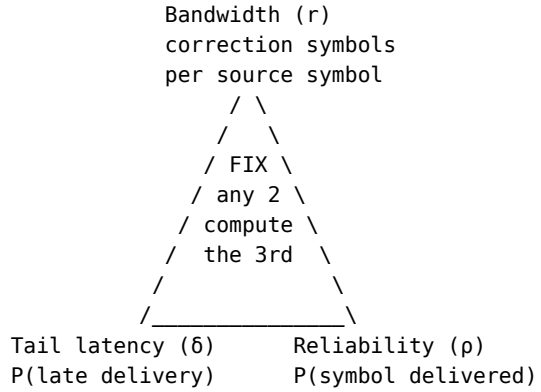
One unified mechanism — **correction symbols** — handles both proactive and reactive recovery. The taper function controls correction symbol density, and each correction slot either

retransmits an exact source symbol from the retransmit buffer (preferred, if old enough) or generates a random repair symbol (FEC fallback):

Aspect	Correction Symbol (retransmit)	Correction Symbol (FEC repair)
When chosen	With probability P_lost(t)	With probability 1-P_lost(t)
Content	Exact copy of source symbol	Random linear combination
Decode cost	Zero (immediate use)	Needs FEC decoder
Bandwidth cost	Same (one symbol slot)	Same (one symbol slot)
Latency cost	Depends on when P_lost triggers	Zero additional (arrives with source)

1.4 The Bandwidth / Latency / Reliability Triangle

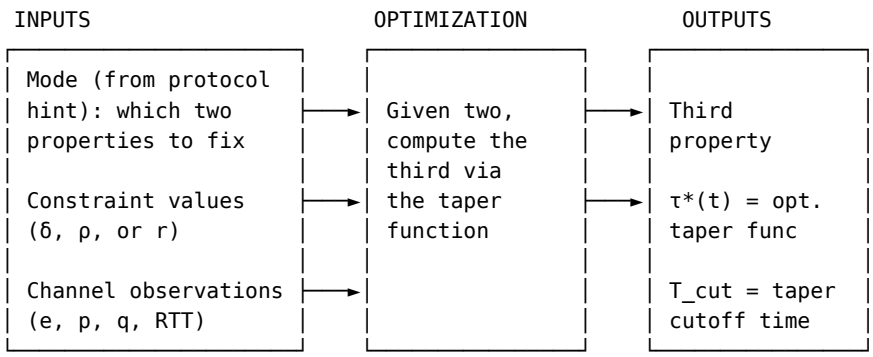
Three properties are linked by the channel. Fix any two, the third is determined:



Mode	Fix	Compute	Use case
Bulk transfer	ρ=100%, minimize r	δ (tail latency)	File transfer, backup
VoIP	δ (max latency), r (codec rate)	ρ (reliability)	Interactive voice
Live video	δ (frame deadline), ρ (≥99.9%)	r (bandwidth)	Streaming, conferencing
Gaming	δ (tight), ρ (≥99%)	r (bandwidth)	Real-time game state
Sensor/IoT	r (minimal), ρ (≥95%)	δ (tail latency)	Periodic telemetry

For Bulk transfer, “minimize r” is taken literally: the tail target is “late is fine” ($\delta\_bulk = \hat{\varepsilon} + (0.05 - \hat{\varepsilon})\cdot\chi$, Sections 5.3 and 14.26), so r = 0 identically in the steady state ($\chi = 0$) and the residual is pure ARQ — except over the final ~1.5 SRTT of a known-length transfer, where the completion-exposure χ ramps r up to the tail budget and buys completion time (Sections 14.25, 14.26).

The protocol hint selects the mode and constraints:



When ρ < 100%, the taper has a finite cutoff T_cut. Symbols not recovered by T_cut are permanently lost. When ρ = 100%, T_cut = ∞ (correction symbols continue until ACK, the infinite-tailed taper from Section 4).

---

2. Channel Model

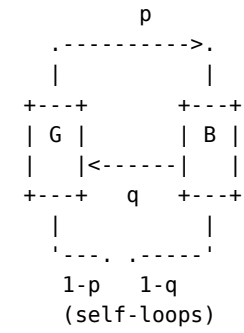
2.1 Gilbert-Elliott Two-State HMM

Why a two-state model? Real wireless channels don't lose packets independently. WiFi interference causes consecutive packet drops during fading events. LTE handovers create burst gaps as the device switches cells. Satellite links suffer weather-induced outages lasting many packets.

An independent (i.i.d.) loss model — where each packet is lost with probability ε regardless of its neighbors — badly underestimates the probability of burst loss. If ε = 5%, the i.i.d. model predicts P(3 consecutive losses) = 0.05^3 = 0.0125%. In reality, once the channel enters a bad state, consecutive losses are highly correlated, and P(3 consecutive) might be 2-5% — orders of magnitude higher.

The Gilbert-Elliott model captures this **memory** in the loss process with just two parameters (p, q). The channel switches between a Good state (no/low loss) and a Bad state (high/total loss). Once in the Bad state, it tends to STAY there for multiple symbols (a burst). This simple structure is rich enough to match real channel behavior [Gilbert1960] yet tractable enough for closed-form analysis — the burst survival function (1-q)^t directly gives us the optimal taper function shape (Section 4).

The channel alternates between Good (low loss) and Bad (high loss) states [Gilbert1960], [Elliott1963]:




```
p   = P(Good -> Bad)  = probability of entering a burst
q   = P(Bad -> Good)  = probability of exiting a burst
1-p = P(Good -> Good) = probability of staying in Good
1-q = P(Bad -> Bad)   = probability of burst continuing
```

Simplified model (used throughout): in Good state, no loss (h_G = 0). In Bad state, total loss (h_B = 1). For packet-level erasure channels, this is not an approximation — it is the correct model. UDP datagrams either arrive intact (transport-layer checksums verify integrity) or are fully dropped. There is no partial packet delivery. A symbol is either received completely or lost completely.

This would be an approximation for bit-level or symbol-level channels where partial corruption exists (e.g., analog radio). But raptorpath operates at the packet level over UDP/QUIC, where checksum verification makes h_G = 0 and h_B = 1 exact.

2.2 Stationary Properties

Stationary state probabilities:

```
π_B = p / (p + q)       probability of being in Bad state
π_G = q / (p + q)       probability of being in Good state
```

Average loss rate:

```
e = π_B x h_B + π_G x h_G = π_B = p / (p + q)
```

2.3 Burst Length Distribution

Given we just entered the Bad state, the burst length T follows a geometric distribution:

```
P(T = t) = q x (1-q)^(t-1)        for t = 1, 2, 3, ...

P(T ≥ t) = (1-q)^(t-1)            survival function

E[T] = 1/q = B                     mean burst length
```

This survival function is the key quantity — it tells us, given that a burst started, how likely it is to still be ongoing after t symbols.

2.4 Concrete Examples

Scenario	ε (loss)	p (G→B)	q (B→G)	B = 1/q	Character
DC	0.1%	0.0005	0.5	2.0	Rare, short bursts
WiFi	2.5%	0.013	0.5	2.0	Moderate, short

Scenario	ε (loss)	p (G→B)	q (B→G)	B = 1/q	Character
LTE	5%	0.02	0.4	2.5	Moderate, medium
Satellite	9%	0.03	0.3	3.3	Frequent, long
Bad WiFi	15%	0.053	0.3	3.3	Frequent, long

(Consistency check: ε = p/(p+q). E.g. DC: 0.0005/0.5005 ≈ 0.1%; WiFi: 0.013/0.513 ≈ 2.5%; Satellite: 0.03/0.33 ≈ 9.1%.)

---

3. Recovery Fundamentals

3.1 The Problem: Recovering Lost Symbols

When a source symbol is erased by the channel, the receiver has a gap in the data stream. There are two fundamentally different ways to fill that gap:

1. **Send extra data proactively** (before knowing what will be lost): the sender generates redundant symbols alongside source data. If a source symbol is lost, the redundant symbols can reconstruct it. This costs bandwidth (the redundant symbols are sent whether needed or not) but adds no latency (they arrive at roughly the same time as the source).
2. **Detect the loss and resend** (after knowing what was lost): the sender discovers that a symbol wasn't received (via ACK timeout) and retransmits it. This costs latency (at least one round-trip to detect + retransmit) but wastes no bandwidth (only lost symbols are resent).

These two approaches are called **FEC** (Forward Error Correction) and **ARQ** (Automatic Repeat reQuest). The challenge is finding the right balance between them — more FEC means lower latency but higher bandwidth; more ARQ means lower bandwidth but higher latency.

3.2 FEC: Proactive Recovery

The encoder window. The sender maintains a sliding window of the W most recent source symbols. This window is the encoder's "memory" — it tracks which source symbols are available for generating repair:

```
Encoder window (W = 5 symbols):

sent:   [S1] [S2] [S3] [S4] [S5] [S6] [S7] [S8] ...
                  |----- window ------|
                  S3  S4  S5  S6  S7

As new symbols arrive, old ones are evicted from the left.
```

Repair symbols are random linear combinations of source symbols in the window. Each repair symbol is computed as:

```
R = c1*S3 + c2*S4 + c3*S5 + c4*S6 + c5*S7
```

where c1..c5 are random coefficients in GF(256) (a finite field — arithmetic wraps around so values stay within 0-255). Each repair symbol is a different random combination, giving the decoder independent equations.

How decoding works. If k source symbols in the window are lost, the receiver needs at least k repair symbols that survived the channel. Each repair symbol provides one linear equation relating the lost symbols. With k equations and k unknowns, the decoder solves the system (Gaussian elimination over GF(256)) to recover the lost data.

```
Example: source S3 and S5 lost, S4/S6/S7 received

Time ------------------------------------------------------------>

Source:  [S1] [S2] [S3] [S4] [S5] [S6] [S7]
                     X         X              <- lost
Repair:    [R1]  [R2]    [R3]       [R4]
            |     |        |          |
            v     v        v          v
       covers   covers   covers    covers
       S1-S3    S2-S4    S3-S6     S5-S7

Decoder receives R1, R2, R3, R4 and knows S4, S6, S7.
Substituting known values into R3 and R4:
  R3 = c1*S3 + c2*(known S4) + c3*S5 + c4*(known S6)
  R4 = c5*S5 + c6*(known S6) + c7*(known S7)
-> 2 equations, 2 unknowns (S3, S5) -> solve.
```

Systematic codes. In our system, source symbols are sent as-is — they are not encoded. This means if nothing is lost, the receiver can use the data immediately without any decoding. The decoder is only invoked when at least one source symbol is missing. This is important for efficiency: at low loss rates, the decoder rarely runs.

Codec overhead. Not all random linear combinations are independent. Some may be (near-)linearly dependent, providing redundant equations. This means the decoder sometimes needs slightly more repair symbols than the number of losses. This extra requirement is the codec overhead — e.g., 1% for RaptorQ means the decoder needs 1% more symbols than the theoretical minimum of k.

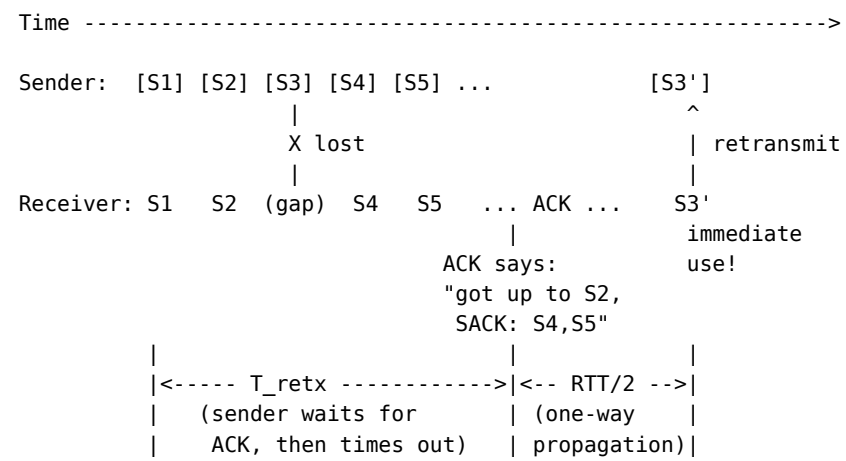
Properties of FEC: - Bandwidth cost: r repair symbols per source symbol (always-on, paid whether loss occurs or not) - Latency cost: zero additional (repair arrives at roughly the same time as source) - Bandwidth fraction used: r/(1+r) of total link capacity

3.3 ARQ: Reactive Recovery

ARQ recovers lost symbols by retransmitting them after detecting the loss. In our system, loss detection is **sender-side**: the sender tracks which symbols have been ACKed and retransmits any that remain un-ACKed after a timeout.

This is how TCP has worked for 40 years [RFC2018] and is proven robust for unicast connections. The alternative (receiver-side detection via NACK messages) adds detection delay and is fragile when the reverse path is lossy.

Timeline of ARQ recovery:



The sender sent S3 at time 0. After T_retx (roughly one RTT, enough time for an ACK to return), no ACK has confirmed S3. The sender retransmits S3. The retransmitted symbol travels one-way (RTT/2) to the receiver.

Key advantage over FEC: A retransmitted source symbol is immediately usable by the receiver — no decoder needed, no dependency on other symbols. The receiver gets the exact data it was missing and can process it right away.

Key disadvantage: The retransmission adds L_arq = T_retx + RTT/2 of latency. For a link with 50ms RTT and T_retx = 75ms: L_arq = 100ms. This is acceptable for bulk transfer but too slow for real-time applications.

Properties of ARQ: - Bandwidth cost: approximately ε per source symbol (only retransmit what's actually lost — no waste) - Latency cost: L_arq = T_retx + RTT/2 per recovery event - Self-healing: if the ACK itself is lost on the reverse path, the sender just waits longer and retransmits anyway. No special mechanism needed.

3.4 Recovery Latency and the P_lost(t) Model

A lost symbol is recovered by whichever mechanism finishes first — FEC or ARQ. Rather than a hard timeout threshold, the choice between repair and retransmit is **probabilistic**, based on the sender's confidence that a symbol was lost.

FEC recovery time (t_fec): After a loss, repair symbols keep arriving. Every repair is a linear combination of the entire encoder window (Section 3.2), so every surviving repair gives the decoder one more equation for the lost symbol — not just the repairs "attributed" to that symbol by its own taper. In steady state, corrections occupy a fraction r/(1+r) of wire slots (Section 5.3), so useful equations arrive at rate r/(1+r) × (1-e) per slot. For m lost symbols in the window, the decoder needs m surviving repairs:

```
t_fec = m x (1+r) / (r x (1-e)) x t_sym

where:
  m     = number of lost symbols in the window (usually 1)
  r     = total correction rate (corrections per source symbol)
  1-e   = probability each repair survives the channel
  t_sym = symbol_size / throughput (time to transmit one symbol)
```

Concrete examples for a single loss (m=1), r=0.08, e=0.025:

```
At 100 Mbps, 1200-byte symbols:  t_sym = 0.096ms, t_fec = 1.3ms
At  10 Mbps, 1200-byte symbols:  t_sym = 0.96ms,  t_fec = 13.3ms
At   1 Mbps, 1200-byte symbols:  t_sym = 9.6ms,   t_fec = 133ms
```

For burst loss (m=5 on WiFi at 100 Mbps): t_fec = 5 x 1.3ms = 6.6ms.

P_lost(t): the probability a symbol was lost. At time t after sending a symbol, given no ACK has arrived, what is the probability it was actually lost?

If the symbol was received, the ACK should arrive after roughly one RTT. If the symbol was lost, no ACK will ever come (until a correction recovers it). Using Bayes' theorem:

```
P_lost(t) = e / [e + (1-e) x P(RTT > t)]

where:
  e          = channel loss rate (prior probability of loss)
  P(RTT > t) = probability the ACK is delayed beyond time t
             = tail of the RTT distribution (from SRTT and RTTVAR)
```

Assuming RTT is normally distributed with mean SRTT and standard deviation RTTVAR: P(RTT > t) = 1 - Phi((t - SRTT) / RTTVAR), where Phi is the standard normal CDF. This is the same normal approximation used in TCP's RTO computation [RFC6298].

This gives a smooth transition from "probably fine" to "certainly lost":

```
t = 0:            P_lost = e              (just the base loss rate)
t = SRTT:         P_lost ≈ 2e             (ACK expected by now)
t = SRTT + 2s:    P_lost ≈ 0.5            (loss is now the likelier explanation)
t = SRTT + 4s:    P_lost > 0.99           (very confident it's lost)
t >> SRTT:        P_lost -> 1.0            (certainly lost)
```

Concrete example (WiFi, e = 0.025, SRTT = 50ms, RTTVAR = 5ms):

```
t = 0ms:    P_lost = 0.025    -> 97.5% repair, 2.5% retransmit
t = 40ms:   P_lost = 0.026    -> 97% repair, 3% retransmit
t = 50ms:   P_lost = 0.049    -> 95% repair, 5% retransmit
t = 55ms:   P_lost = 0.14     -> 86% repair, 14% retransmit
t = 60ms:   P_lost = 0.53     -> 47% repair, 53% retransmit
t = 70ms:   P_lost = 0.999    -> ~0% repair, ~100% retransmit
```

Note how sharp the transition is: P_lost stays near ε until roughly SRTT + 1×RTTVAR, then rises to near-certainty within another 2-3×RTTVAR. The smoothness of the mix in practice comes from the spread of symbol ages in the retransmit buffer, not from the curve itself being gradual.

The per-slot decision uses P_lost(t) directly:

```
P(retransmit) = P_lost(t_k)
```

No hard threshold. The transition from repair to retransmit is driven by TIME since send. Early (t small): P_lost is low, so corrections are mostly repair — flexible coverage for uncertain losses. Late (t >> SRTT): P_lost approaches 1, so corrections are mostly retransmit — targeted recovery of confirmed losses. No tuning knobs.

See Appendix C.6 for an optional channel-state discount (ε_burst) that provides modest improvement for long-burst scenarios.

Bandwidth efficiency. A retransmit is wasted if the symbol wasn't actually lost (duplicate). A repair is never wasted (always provides information to the decoder). The expected waste per correction slot is:

```
E[waste] = P(retransmit chosen) x P(symbol not actually lost)
         = P_lost(t) x (1 - P_lost(t))
```

This is maximized at P_lost = 0.5 (maximum uncertainty) and zero at the extremes. The probabilistic model automatically minimizes waste.

Proactive retransmit emerges naturally. At high loss rates (e.g., e = 0.5), P_lost(0) = 0.5 — half the correction slots are retransmits even at t = 0. This is proactive redundant source sending, no special mode needed. At low loss rates (e = 0.001), P_lost(0) = 0.001 — virtually all repair. The model adapts to the channel automatically.

Which mechanism wins on latency? FEC recovery (t_fec) is typically much faster than ARQ recovery (waiting for P_lost to rise + RTT/2):

```
Time ------------------------------------------------------->
t=0          t_fec                                    retransmit
|             |                                       arrives
S3 lost       FEC decodes      P_lost rises,           |
              (if enough       retransmit chosen       |
               repairs         in correction slots     |
               arrived)                                |
|<-- FEC -->|
|<---- taper generating corrections the whole time --->|
|     (gradually shifting from repair to retransmit)   |
```

At 100 Mbps on WiFi: t_fec = 1.3ms. Most losses are FEC-recovered long before P_lost rises high enough for retransmission. ARQ retransmit is only relevant for burst losses that overwhelm the FEC budget.

---

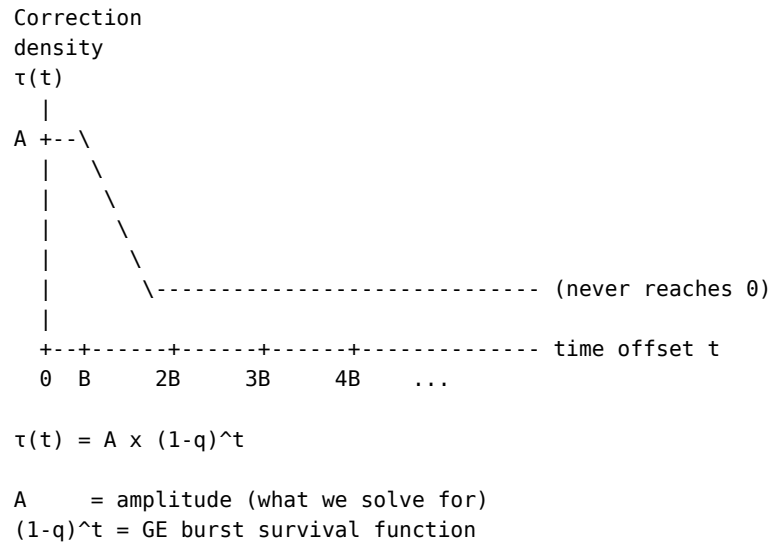
4. The Taper Function

Why not send correction symbols at a constant rate? If loss were uniformly distributed over time (i.i.d.), a constant rate would be optimal — every position is equally likely to need correction. But on real wireless channels, loss comes in **bursts** (Section 2). A burst wipes out consecutive symbols, then the channel recovers. Right after a burst starts, the probability of continued loss is high; as time passes, it decays exponentially.

A constant correction rate misallocates: it sends too many corrections during good periods (wasted) and too few right after a burst (when they're needed most). The taper function solves this by matching correction density to the burst survival probability — more corrections where loss is likely, fewer where it isn't. This minimizes the total correction budget for a given recovery target.

4.1 Definition

The taper function $\tau(t)$ specifies the correction density at time offset t from a source symbol. At offset t after symbol s enters the window, we generate $\tau(t)$ correction symbols covering s.




```
τ(t) = A x (1-q)^t

A     = amplitude (what we solve for)
(1-q)^t = GE burst survival function
```

4.2 Why Match the Loss Distribution?

The taper should allocate more correction where it does the most good. Given that a symbol was lost (we're in a burst), the conditional probability that the burst is still active at offset t is:

```
P(burst active at offset t | burst at offset 0) = (1-q)^t
```

A correction at offset t is useful only if BOTH (a) the lost symbol still needs help at t, and (b) the correction itself survives the channel. These pull in opposite directions: (a) decays like the burst survival function (1-q)^t, while (b) — conditioned on the loss at offset 0 — is the probability the burst has ENDED by t, roughly 1 - (1-q)^t. Corrections sent into an ongoing burst are themselves lost (Section 14.15). The product

```
benefit(t) ∝ (1-q)^t × (1 - (1-q)^t)
```

is hump-shaped: zero at t = 0, peaking around one mean burst length (t ≈ B), then decaying like (1-q)^t. The taper's exponential decay matches the correct TAIL behavior; its peak at t = 0 is an approximation that over-weights the first few offsets, where corrections have the lowest conditional survival probability. Two effects limit the cost of this approximation:

1. **Repairs are window-fungible.** Every repair covers the whole encoder window (Section 3.2), so a correction "attributed" to offset 0 of a fresh symbol simultaneously sits at larger offsets for every older symbol in the window. Per-symbol attribution is bookkeeping, not physics — recovery depends on the aggregate correction rate (Section 8.2), which the shape does not change in steady state (see below).
2. **The exponential tail is the operationally important part.** The (1-q)^t decay controls how long un-ACKed (increasingly likely lost) symbols keep drawing correction coverage after everything else has been ACKed.

Steady-state shape invariance. With a continuous source stream, the aggregate correction rate per wire slot is $\Sigma_t \tau(t) = r$ regardless of the taper's shape — every in-window symbol contributes its taper at a different age, and the sum over ages telescopes to the same total. The shape only affects behavior in transients: when the source pauses, when the window is not yet full, and — most importantly — through ACK truncation (Section 4.4): once a symbol is ACKed, its taper contribution stops, so beyond one RTT only un-ACKed (likely lost) symbols

continue to generate corrections. The decay rate q determines how aggressively that residual coverage tapers off. The taper shape is therefore a policy for allocating corrections across UNCERTAIN symbols, not a mechanism that changes the steady-state budget.

Truncation at the stream end (forward integral). The shape-invariance argument assumes the sum telescopes over a FULL window of ages — every in-window symbol present at every offset. That holds mid-stream but breaks at the END of a finite transfer. A symbol at source position i draws correction coverage from repairs generated while it is in the encoder window, i.e. over the FUTURE source positions [i, i+W). When i+W exceeds the last source position N, those future positions never arrive, so the symbol's forward taper integral is TRUNCATED: a symbol at distance j = N − i from the end (j < W) receives coverage from only j of its W window-lifetimes, i.e. it is missing ≈ r·(W − j) of its steady-state repairs. The last W symbols are therefore progressively under-covered (full at j = W, ~none at j = 0), and because end-of-stream losses have nothing to overlap their recovery with, each falls to a serial ARQ round (~1.5 RTT). This is the end-of-stream reliability cliff; Section 14.29 derives the loss and the completion term that refills it for every hint (Section 14.26 is the Bulk special case).

For an i.i.d. channel (q = 1, no burst memory): τ(t) = constant (flat taper). This is correct — every position is equally likely to need correction.

4.3 Total Correction Rate

The total correction rate (correction symbols per source symbol) is:

```
r = Σ_{t=0}^{∞} τ(t) = A x Σ_{t=0}^{∞} (1-q)^t = A / q
```

Since 0 < q ≤ 1, this geometric series converges. Therefore:

```
A = r x q
```

The amplitude is uniquely determined by the correction rate and the GE parameter.

4.4 The Taper Never Reaches Zero

The exponential (1-q)^t is always positive for 0 < q < 1. This is correct behavior: there is always a nonzero probability of a burst still continuing. As long as a symbol has not been ACK'd, there is a nonzero probability it was lost, so we should continue generating (increasingly rare) correction coverage.

```
t = 0:    τ(0) = A                          peak correction density
t = B:    τ(B) = A x e^{-1} ≈ 0.37 x A   one mean burst length
t = 2B:   τ(2B) = A x e^{-2} ≈ 0.14 x A  two mean burst lengths
t = 5B:   τ(5B) = A x e^{-5} ≈ 0.007 x A five mean burst lengths
t → ∞:    τ(t) → 0                          but never zero
```

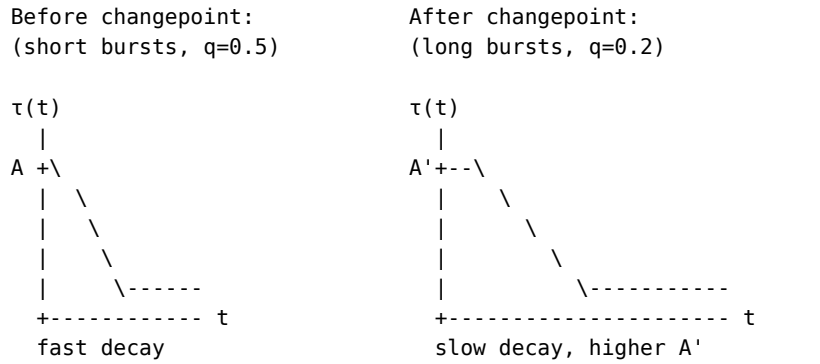
In practice, once a symbol is ACK'd, we stop generating correction for it (the encoder window advances past it). The theoretical infinite tail is truncated by the ACK mechanism.

4.5 Real-Time Adaptation

The taper function adapts in real time through two mechanisms:

1. **GE parameter updates**: The estimator continuously tracks q (and p) from observed loss patterns. As q changes, the taper shape changes — slower decay for longer bursts, faster decay for shorter bursts.

2. **BOCD changepoint detection**: If the loss regime changes abruptly (e.g., path switches from WiFi to LTE), BOCD detects the changepoint within 5-15 batches (a batch = one ACK feedback cycle, typically at intervals of 10-100ms depending on sending rate and path RTT) and widens the posterior, increasing the correction budget until the new regime is characterized.



---

5. Unified Correction Symbol Model

5.1 Why Unify FEC and ARQ?

Both FEC and ARQ produce one symbol on the wire. Both cost the same bandwidth per symbol. The difference is timing and content:

```
FEC repair:        generated immediately, random combination, needs decoder
ARQ retransmit:    generated after timeout, exact source copy, immediately usable

Both occupy one symbol slot on the wire.
Both are subject to the same channel loss rate e.
Both serve the same purpose: recover a lost source symbol.
```

The key insight: **the taper function controls WHEN to generate correction symbols** (the density schedule). **The per-slot decision controls WHAT to generate** (repair or retransmit). These are orthogonal choices:

- **Early in the taper** (right after sending source): P_lost(t) is low, so almost all correction symbols are FEC repair. This is pure proactive protection — we don't yet know what's lost.
- **Late in the taper** (t >> SRTT): P_lost(t) approaches 1, so correction symbols are almost all retransmits. We're confident about which symbols are lost and send the exact data the receiver needs.
- **In between**: a smooth probabilistic mix. No hard phase switch.

This means the taper function **naturally transitions from FEC to ARQ** as time passes, driven by the P_lost(t) posterior. The unified mechanism is called a **correction symbol** — it is either a repair symbol or a source retransmit, decided probabilistically at generation time.

5.2 Correction Symbols — The Unified Concept

A **correction symbol** is any symbol sent to recover lost data. It occupies one symbol slot on the wire and serves one of two purposes:

Aspect	Source retransmit (ARQ)	Repair symbol (FEC)
When chosen	With probability P_lost(t)	With probability 1-P_lost(t)
Content	Exact copy of source symbol	Random linear combination
Receiver action	Immediate use, no decoder	Feed to FEC decoder
Bandwidth cost	Same (one symbol slot)	Same (one symbol slot)
Waste risk	Duplicate if symbol wasn't lost	Never wasted (always useful)
Best for	Confirmed loss + good channel	Uncertain loss or burst (flexible)

The taper function (Section 4) determines the **density** of correction symbols over time. This section explains what happens in each slot.

5.3 Three-Stream View: Source, Repair, Retransmit

Conceptually, the sender manages **three streams** that compete for wire capacity. Each stream has a different effect on the bandwidth/latency/ reliability triangle:

```
Stream         | Latency       | Bandwidth     | Reliability
---------------+---------------+---------------+-------------
Source         | ++ immediate  | neutral       | neutral
Repair (FEC)   | - decoder     | + (no waste   | ++ (covers
               |   wait        |   if taper    |    any loss)
               |               |   matched)    |
Retransmit     | + immediate   | - (duplicate  | + (targeted
(ARQ)          |   decode      |   risk)       |    recovery)
```

The hierarchical model (taper + P_lost) determines the three-stream mix:

```
Total capacity: C (from Copa)

source_rate     = C / (1 + r)                          [from taper ratio r]
retransmit_rate = C x r / (1+r) x P_lost(t)            [confirmed losses]
repair_rate     = C x r / (1+r) x (1 - P_lost(t))      [uncertain losses]

where r        = taper correction ratio (Section 4)
      P_lost(t) = loss confidence (Section 3.4)
```

The three rates sum to C. The taper ratio r controls the source/correction split. P_lost(t) controls the repair/retransmit split within corrections.

How the protocol hint shifts the mix:

```
Realtime (low latency):
  -> tight delta -> high r (more corrections for fast FEC recovery)
  -> BUT also high P_retx when confident (retransmit = immediate decode)
  -> Mix: moderate source, moderate repair, moderate retransmit

Bulk (high bandwidth):
  -> loose delta -> low r (fewer corrections, more source)
  -> low P_retx (prefer repair over retransmit, less duplicate risk)
  -> Mix: lots of source, some repair, little retransmit

  Concretely, Bulk's tail target is "late is fine", weighted by the
  completion exposure chi (Section 14.26):

    delta_bulk = epsilon_hat + (delta_tail - epsilon_hat) x chi(T_rem)
```

```
  Mid-stream chi = 0, so delta_bulk = epsilon_hat and the continuous
  r* formula (Section 8.4, z_{delta/epsilon} = Phi^-1(1 -
  delta/epsilon) = Phi^-1(0) = -inf) yields r = 0 IDENTICALLY in the
  steady state — independent of estimator uncertainty: pure ARQ, wire
  volume at parity with retransmission transports (~1 +
  epsilon/(1-epsilon) per source symbol). A mid-transfer loss
  recovered one RTT late costs a bulk transfer nothing — recovery
  overlaps ongoing sends. The one place FEC still buys completion
  time is the stream tail, where chi rises smoothly to 1 and
  delta_bulk glides to the delta_tail = 0.05 completion budget
  (Sections 14.25, 14.26).

VoIP (fixed bandwidth + latency, variable reliability):
  -> r constrained by bandwidth budget
  -> high P_retx when confident (immediate decode matters)
  -> Mix: fixed source rate, split corrections by P_retx
```

Per-path three-stream fractions: Each path has its own r_i and P_retx_i, so the three-stream fractions naturally differ per path. A low-latency path with low loss has: high source, low repair, low retransmit. A high-loss path has: lower source, high repair, moderate retransmit. The hierarchical model produces these automatically — no separate three-way optimization needed.

5.4 Per-Slot Decision

When the taper function (Section 4) decides to generate a correction symbol, the sender makes a probabilistic choice based on P_lost(t) for the oldest un-ACKed symbol in the retransmit buffer:

```
Taper decides: "generate a correction symbol now"
                    |
                    v
+-------------------------------------------+
| Compute mixing probability:               |
|                                           |
|   P_retx = P_lost(t_k)                    |
|                                           |
| With probability P_retx:                  |
|   -> Retransmit exact source              |
|      (immediate decode at receiver)       |
|                                           |
| With probability 1 - P_retx:              |
|   -> Generate random repair symbol        |
|      (FEC, covers any loss in window)     |
+-------------------------------------------+

P_lost(t_k) = e / [e + (1-e) x P(RTT > t_k)]    per-symbol loss confidence
```

The FEC/ARQ mix is regulated by TIME since send. Early correction slots (t small) have low P_lost — the ACK hasn't had time to return, so loss is uncertain. Almost all corrections are FEC repair, which flexibly covers any lost symbol. Late correction slots (t >> SRTT) have high P_lost — the ACK should have arrived by now, so loss is confirmed. Corrections shift to retransmit, which is immediately decodable.

This naturally produces Mehrotra & Li's optimal policy [Mehrotra2010]: during bursts (before RTT elapses), P_lost stays low → mostly repair. After bursts (ACKs reveal gaps), P_lost rises → targeted retransmit.

The expected WASTE, E[waste] = P_lost × (1 - P_lost) (Section 3.4), is zero at the extremes (P_lost = 0: all repair, no waste; P_lost = 1: all retransmit, definitely lost so no waste) and maximized at P_lost = 0.5 (maximum uncertainty). The model automatically allocates the right mix.

See Appendix C.6 for an optional channel-state discount (ε_burst) that provides modest improvement for long-burst scenarios.

---

6. Protocol Mechanics

6.1 The Retransmit Buffer

The sender maintains a **retransmit buffer**: an ordered list of source symbols that have been sent but not yet ACKed.

```
Retransmit buffer (ordered by send time, oldest first):

+-----+-----+-----+-----+-----+-----+-----+
| S12 | S13 | S17 | S18 | S19 | S24 | S25 |  <- un-ACKed symbols
+-----+-----+-----+-----+-----+-----+-----+
  ^                                 ^
  oldest                            newest
  (first retransmit                 (too recent,
   candidate if                     wait for ACK)
   age > T_retx)
```

Operations: - **Enqueue**: every sent source symbol is added to the tail - **Dequeue (ACK)**: when an ACK or SACK confirms receipt, remove the symbol - **Peek (retransmit)**: when generating a correction symbol, check the head — if its age exceeds T_retx, retransmit it (but keep it in the buffer until ACKed)

The retransmit buffer is NOT bounded by the encoder window. The encoder window (W symbols) bounds what FEC repair can cover. The retransmit buffer bounds what ARQ retransmit can cover. They serve different purposes:

- Symbol in encoder window AND buffer: protected by FEC + ARQ
- Symbol evicted from window, still in buffer: protected by ARQ only
- Symbol ACKed: removed from buffer (delivered)

Two independent mechanisms manage the buffer:

Age eviction (T_cut): Symbols older than T_cut (derived from the reliability target ρ in Section 8.6) are evicted from the buffer — accepted as permanently lost. T_cut is derived from the triangle optimization (Section 8.6): for reliability target ρ, T_cut is the time where P(not recovered) = 1-ρ. When ρ = 100%, the equation has no finite solution, so T_cut = ∞ (never evict). This emerges from the math — not a special case.

Size backpressure (buffer_max): When the buffer reaches buffer_max entries, the sender pauses accepting new source symbols from the application (backpressure). Correction symbols continue being generated for existing buffer contents — corrections recover lost symbols, ACKs arrive, and the buffer drains. Once space is available, source resumes.

buffer_max is derived, not configured:

```
For ρ < 100%:  buffer_max = source_rate x T_cut
               (eviction keeps buffer within this bound)

For ρ = 100%:  buffer_max = source_rate x (RTT + B_max / (r* x (1-e)) x t_sym)
               where B_max = ceil(ln(0.0001) / ln(1-q))
               (99.99th percentile burst length from GE model;
                ≈ 9.2/q for small q, = 14 exactly at q = 0.5)
```

Both mechanisms always run. T_cut determines which triggers first: - Finite T_cut ⟶ eviction keeps buffer small ⟶ backpressure rarely triggers - T_cut = ∞ ⟶ no eviction ⟶ backpressure is the only limit

What bends when: - ρ < 100%: reliability bends (eviction at T_cut) - ρ = 100%: latency bends (backpressure at buffer_max, rare)

6.2 SACK-Extended ACK

The receiver sends an **ACK per received symbol** — every incoming symbol triggers an immediate ACK in the reverse direction. This provides the fastest possible loss detection and the most RTT samples for estimation.

Overhead: Each ACK is ~80 bytes (fields + QUIC/UDP headers). With 1200-byte symbols, per-symbol ACKs generate reverse-path traffic of about 80/1200 ≈ 6.7% of the forward data rate. On symmetric links this is an acceptable price for per-symbol loss detection and RTT sampling. On strongly asymmetric uplinks (LTE, satellite, DOCSIS) it can be significant, and the per-batch or piggyback alternatives below become attractive; the model itself is agnostic to ACK batching (it only slows the P_lost transition by the batch interval).

Alternatives considered: - Per-batch ACK: lower overhead, but slower loss detection (batch granularity) - Delayed ACK (TCP style, every 200ms): 50% less traffic, but adds 200ms detection delay — too slow for our P_lost model - Piggyback ACK on reverse-direction data: zero overhead for bidirectional traffic — a future optimization for the bidirectional case

ACK contents (5 fields):

```
+-----------------------------------------------------------+
| cumulative_ack:       u64    all seqs <= this received    |
| sack_ranges:          [(u64,u64)]  additional ranges      |
| echo_timestamp:       u64    sender's timestamp           |
| jitter_us:            u32    interarrival jitter (us)     |
| cumulative_received:  u64    total symbols received       |
+-----------------------------------------------------------+
```

- **cumulative_ack**: highest sequence number such that ALL symbols up to and including it have been received. An optimization — equivalent to a SACK range starting at 0, but compressed to one number.
- **sack_ranges** (Selective ACK [RFC2018]): out-of-order ranges received beyond the cumulative point. Tells the sender exactly which symbols arrived despite gaps. Cumulative within the T_cut window — all received fragments are reported, not just the most recent (unlike TCP which limits to 3-4 blocks).
- **echo_timestamp**: sender's own send timestamp echoed back for RTT measurement. RTT = now - echo. No clock synchronization needed.

- **jitter_us**: interarrival jitter in microseconds [RFC3550 A.8]. u32 holds up to 4300 seconds (matches RFC 3550's 32-bit field). Used for reorder buffer sizing, congestion detection, and real-time jitter budgets.
- **cumulative_received**: running total of symbols received. Self-healing counter that survives ACK loss and SACK pruning. Gives the sender a reliable aggregate reliability metric: ρ_actual = received / sent.

Gap pruning at T_cut: For ρ < 100%, gaps older than T_cut are pruned by the receiver — cumulative_ack advances past abandoned symbols. Both sender (buffer eviction) and receiver (gap pruning) use the same T_cut, communicated at connection start and updatable mid-connection via control message.

For ρ = 100% (T_cut = ∞): no pruning. cumulative_ack advances only when gaps are filled by corrections. SACK ranges are temporary.

ACK loss handling: Cumulative ACKs are self-healing — each new ACK supersedes all previous ones. If ACK_5 is lost but ACK_6 arrives, the sender gets all information from ACK_6. No ACK retransmission needed.

6.3 Per-Symbol Delivery Outcomes

A lost symbol has three possible outcomes, depending on whether the taper function's correction symbols recover it before the cutoff T_cut:

```
Outcome 1: FEC-recovered (proactive repair arrives before T_retx)
  Latency: L_prop + small delay (same as source)
  Probability: e x P_fec

Outcome 2: ARQ-recovered (retransmit arrives after T_retx, before T_cut)
  Latency: L_prop + L_arq  where L_arq = T_retx + RTT/2
  Probability: e x (1 - P_fec) x P_arq

Outcome 3: Lost (not recovered by T_cut)
  Latency: infinity (never delivered)
  Probability: e x (1 - P_fec) x (1 - P_arq)
```

P_arq is the probability that ARQ retransmission succeeds, given that FEC failed to recover the symbol. It is a conditional probability in this chain:

```
Symbol sent
  |
  +-- (1-e): arrives directly          -> delivered (on-time)
  |
  +-- e: lost on the channel
       |
       +-- P_fec: FEC recovers it      -> delivered (on-time)
       |
       +-- (1-P_fec): FEC failed
            |
            +-- P_arq: ARQ succeeds    -> delivered (late, L_arq)
            |
            +-- (1-P_arq): ARQ fails   -> permanently lost
```

P_arq is derived from the reliability target ρ, not computed independently:

```
P(lost) = e x (1 - P_fec) x (1 - P_arq) = 1 - ρ

Solving: P_arq = 1 - (1-ρ) / (e x (1-P_fec))
```

For ρ = 100%: P_arq = 1. We keep retransmitting until ACK (T_cut = infinity), so ARQ eventually succeeds. FEC is still active — it handles most recoveries fast. ARQ is the backstop for what FEC misses.

For ρ < 100%: P_arq < 1. We stop retransmitting at T_cut. Symbols not recovered by then are permanently lost.

The full delivery distribution:

```
P(on-time delivery) = (1 - e) + e x P_fec           not lost, or FEC
P(late delivery)    = e x (1 - P_fec) x P_arq       ARQ retransmit
P(permanent loss)   = e x (1 - P_fec) x (1 - P_arq) not recovered

Reliability: ρ = 1 - P(permanent loss)
Tail latency: δ = P(late delivery) / ρ               among delivered symbols
```

6.4 The Triangle in Action

Under **100% reliability** (ρ = 1, T_cut = infinity): outcome 3 never occurs. "Tail loss from FEC" equals "tail latency events" — they are the same thing. This is the special case from Section 8.

Under **variable reliability** (ρ < 1, T_cut < infinity): the taper is cut off. Symbols beyond T_cut are permanently lost. This saves bandwidth (fewer correction symbols) and bounds latency (no recovery beyond T_cut), at the cost of reliability.

```
p = 100%:  ---------------------------------- (taper runs until ACK)
           all symbols eventually delivered

p = 98%:   ----------------+
           98% delivered   | T_cut
           2% lost         +-- (taper stops, accept loss)

p = 95%:   ----------+
           95%       | T_cut (shorter)
           5% lost   +-- accept loss (sensor/VoIP)
```

6.5 Four-Mechanism Composition

The complete system is four independent mechanisms that compose without interfering:

```
Mechanism       Controls              Derives from
-------------------------------------------------------
Copa            Total wire rate        RTT, bandwidth (delay-based)
Taper           Source/correction      e, q, delta (GE + triangle)
                ratio
T_cut + buffer  Reliability guarantee  rho (triangle), GE B_max
P_lost          Repair/retransmit      ACK timing, e
                mix
```

Each mechanism has one job and one set of inputs. None needs to know about the others:

- Copa doesn't know about FEC — it just limits total rate
- The taper doesn't know about Copa — it just sets the ratio
- T_cut doesn't know about the taper — it just evicts old symbols
- P_lost doesn't know about T_cut — it just picks repair vs retransmit

The protocol hint flows through the triangle to set delta and rho, which determine the taper amplitude and T_cut. Copa independently discovers the pipe capacity. P_lost independently tracks per-symbol loss confidence. The four mechanisms compose to produce the correct behavior for each protocol class without branching or mode switches.

---

7. Estimation — From Observations to Channel Parameters

7.1 What We Observe

At the sender, we receive periodic feedback:

Observation	Source	Frequency
(sent, received) per batch	ACK messages	Every batch (~10-100ms)
RTT	Echoed timestamps in ACK	Every batch
SACK ranges (out-of-order blocks)	ACK+SACK messages	Every ACK
Throughput	Delivery rate tracking	Continuous

The sender infers losses from gaps: symbols not covered by the cumulative ACK or any SACK range are presumed lost after T_retx has elapsed.

7.2 EWMA — Fast Point Estimate

Exponentially Weighted Moving Average of the loss rate:

```
e_hat_ewma(n) = α x (lost/sent) + (1-α) x e_hat_ewma(n-1)

α = 0.1 → time constant of 10 samples (half-life ≈ 7 samples)
```

Strengths: Simple, fast, responsive to changes.

Limitations: Single number — cannot express "confident at 2%" vs "uncertain, somewhere between 0% and 10%". This inability to express uncertainty is why the old system needed three stacked mechanisms (EWMA + Beta margin + PI controller) to avoid under-provisioning.

7.3 Beta-Binomial Posterior — Uncertainty Quantification

The Beta distribution is the conjugate prior for Binomial observations:

```
Prior:     Beta(a, b)        (a = received, b = lost)
Update:    a' = a x decay + received
           b' = b x decay + lost
Posterior: Beta(a', b')

Mean loss rate:  b' / (a' + b')
Variance:        a'b' / ((a'+b')² (a'+b'+1))
Upper quantile:  beta_quantile(b', a', confidence)
```

The decay factor (0.995) causes old observations to fade, allowing adaptation. This gives a half-life of approximately 138 observations (ln(0.5)/ln(0.995) ≈ 138). The value is not sensitive — decay factors between 0.99 and 0.999 produce similar steady-state behavior. The choice trades off adaptation speed (lower decay = faster forgetting) against estimation stability (higher decay = smoother estimates).

Strengths: Principled uncertainty — the spread of the posterior tells us how confident we are. Tight posterior → low uncertainty → small safety margin needed.

Limitations: Cannot detect regime changes. If loss jumps from 1% to 10%, the posterior slowly drifts — it doesn't know the old data is from a different regime.

7.4 BOCD — Regime-Aware Prediction

Bayesian Online Changepoint Detection [Adams2007] maintains a distribution over "how long since the last regime change?" (the run length).

```
Observation stream:   ...oooooooooxxxxxooooooooooooooxxxooo...
                      <- regime 1  ->   <- regime 2 -> <r3>
                                   ^                ^
                              changepoint      changepoint

Run-length distribution P(r_t | data):

Regime 1 (steady):   Changepoint:          Regime 2 (steady):
Mass at r=50         Mass splits:          Mass at r=20
(confident)          r=0  (new regime)     (confident again)
                     r=51 (old continues)
 |                    |      |               |
_|_                  _|_    _|_             _|_
| |                  | |    | |             | |
------- r            ----------- r          ------- r
  50                 0      51               20
```

For each run length, BOCD maintains Beta-Binomial sufficient statistics. The predictive quantile integrates over all possible run lengths:

```
P̂_upper(confidence) = Σ_r P(r_t = r | data) x beta_quantile(stats_r, confidence)
```

Key properties: - Steady state: mass concentrates at one run length → tight posterior → small margin - Changepoint: mass spreads → wide posterior → large margin (conservative) - The predictive quantile IS the safety margin — no separate PI or margin needed

7.5 GE Parameter Estimation

The GE estimator tracks transition counts with exponential decay:

```
On each symbol observation (Good or Bad):
  decay all counters by factor 0.999
  increment the appropriate transition counter:
    Good→Good: g_to_g     Good→Bad: g_to_b
    Bad→Good:  b_to_g     Bad→Bad:  b_to_b

Estimated parameters:
  p̂ = g_to_b / (g_to_g + g_to_b)         P(Good → Bad)
  q̂ = b_to_g / (b_to_g + b_to_b)         P(Bad → Good)
  B̂ = 1/q̂                               mean burst length
```

Initialization: All transition counters start at zero, initial state assumed Good. The GE estimates are gated by a minimum sample count (the is_valid() check) — until enough transitions are observed (~50 symbols), the estimator returns default values (B=2.0, q=0.5). During this bootstrap period, the Beta prior (Section 7.3) and BOCD (Section 7.4) provide loss estimation without relying on GE burst parameters.

These estimates feed directly into the taper function shape: $\tau(t) = A \times (1-\hat{q})^t$.

7.6 ACK Loss and Self-Healing

The feedback channel (receiver ⟶ sender) may also be lossy. If ACKs get lost, the sender lacks up-to-date knowledge of what the receiver has.

Self-healing property: Unlike a NACK-based system where a lost NACK means the sender never learns about the loss, an ACK-based system is inherently self-healing:

- If an ACK is lost, the sender simply does not advance its knowledge of receiver state. Symbols remain in the retransmit buffer.
- After T_retx elapses without acknowledgment, the sender retransmits — this is correct behavior whether the original was lost or just the ACK was lost.
- If the original arrived but the ACK was lost, the receiver gets a duplicate (harmless — deduplicated by sequence number) and sends another ACK.
- Eventually an ACK gets through, and the sender's state catches up.

```
ACK lost scenario:

Sender:    [S1] [S2] ... wait T_retx ... [S1'] ... receives ACK ... done
                                            ↑
                                    retransmit (safe: either original
                                    or ACK was lost, both handled)

No separate mechanism needed — the retransmit timeout handles both cases.
```

This eliminates the need for RX path loss estimation, NackAck echo, and NACK effectiveness tracking. The system is simpler and more robust.

7.7 Estimation Error and Overhead

Estimation error directly maps to overhead:

```
If e_hat > e_true:  over-provisioning → wasted bandwidth
If e_hat < e_true:  under-provisioning → more ARQ latency events

Overhead gap = (e_hat - e_true) / e_true
```

BOCD minimizes this gap by adapting the estimation confidence to the regime: - Steady state: $\hat{\varepsilon} \approx \varepsilon\_true$ (tight posterior) - Transition: $\hat{\varepsilon} > \varepsilon\_true$ (conservative, correct behavior)

---

8. The Optimization Problem

8.1 Formal Statement

```
minimize:    r = A/q                      (correction rate = bandwidth cost)

subject to:  e x (1 - P_fec(A, q)) ≤ δ    (tail latency constraint)

where:       τ(t) = A x (1-q)^t           (taper function)
             P_fec depends on A, q, e, W  (FEC recovery probability)
```

Input: δ (tail latency target, from protocol hint)

Output: A^* (optimal taper amplitude), $r^* = A^*/q$ (optimal correction rate)

Note on the constraint: Section 8.2 actually derives r^* from the stricter per-window surrogate P(repairs < K) ≤ δ rather than e × (1 - P_fec) ≤ δ — see the "Which δ is this?" note there.

8.2 Corrected Model (canonical)

Let's reconsider. The taper generates correction symbols at known positions. For a window of size W, the taper generates exactly r × W correction symbols total. The question is: given that a source symbol is lost, how many correction symbols covering it will arrive at the receiver?

In a sliding window code, repair symbols are linear combinations of all source symbols in the window. A repair at offset t from source s covers s as long as s is still in the window (t < W).

Number of correction symbols generated while s is in the window:

```
N_correction = Σ_{t=0}^{W-1} τ(t)
```

Of these, each survives independently with probability (1-ε). Expected arrivals:

```
R = N_correction x (1-e) = (A/q) x (1-(1-q)^W) x (1-e)
```

For the decoder to recover s, we need at least 1 linearly independent repair (with systematic code, if all other source symbols arrived, one repair suffices).

But the actual requirement is more nuanced: we need as many repair symbols as there are lost source symbols in the window. If k symbols are lost in the window, we need k repair arrivals.

Average case: In a window of size W with loss rate ε, expected losses = εW. Expected repair arrivals = r × W × (1-ε). For recovery: r × W × (1-ε) ≥ εW, giving r ≥ ε/(1-ε) = r_IT. This confirms the IT minimum.

Tail case: We need recovery even when more symbols are lost than average. The number of losses in a window follows a distribution determined by the GE model. The tail latency constraint requires recovery at the δ-th percentile.

Let K be the number of lost symbols in a window. For the GE model:

```
P(K = k) depends on the burst structure
For large W: K ≈ e*W with variance determined by burstiness
```

For recovery: number of arriving repairs ≥ K.

Repairs arrive = Binomial(r × W, 1-ε), approximately Normal(rW(1-ε), rW(1-ε)ε).

Constraint: P(repairs < K) ≤ δ.

Which δ is this? Section 6.3 defines δ as P(late delivery) = e × (1 - P_fec). Since a symbol can only be late if it was lost first, the constraint on the window is P(repairs < K) ≤ δ/e — the canonical form used in Section 8.4, giving z_{δ/e} in the margin. Two consequences:

1. **Continuity.** As the channel improves relative to the target (e ⟶ δ), z_{δ/e} falls through zero and the required rate decreases continuously to r* = 0 — pure ARQ already meets the tail target. No cutoff rule is needed; the δ = e boundary behavior of Section 11.3 emerges from the closed form.
2. **A conservative variant exists.** Imposing P(repairs < K) ≤ δ directly (z_δ instead of z_{δ/e}) is stricter — it partially compensates for the size-bias of conditioning on "this symbol was lost" (a window known to contain a loss has more losses than average). At WiFi (e = 0.025, δ = 1e-4): z_{δ/e} = 2.65 vs z_δ = 3.72. Deployments wanting extra safety can use z_δ; for exactness, use the transfer-matrix computation (Appendix D, item 4) instead of either approximation.

Using normal approximation:

```
P(repairs < K) ≈ P(Normal(rW(1-e), rW(1-e)e) < K)
```

This requires the δ-quantile of repairs to exceed the (1-δ)-quantile of losses. The algebra gives:

```
r ≥ e/(1-e) + z_δ x √(e/(W(1-e)))
```

where z_δ is the standard normal quantile [Abramowitz1964] for probability δ.

The explicit P_fec formula. Given correction rate r, loss rate e, burst variance s2_burst, and window size W:

```
Losses in window:      K ~ Normal(We, We(1-e)s2_burst)
Surviving corrections: C ~ Normal(rW(1-e), rWe(1-e))

P_fec = P(C >= K) = Phi(z)

where z = sqrt(W) x (r(1-e) - e) / sqrt(e(1-e)(r + s2_burst))
```

At the IT minimum (r = e/(1-e)): z = 0, P_fec = 0.5 (coin flip — half the time corrections suffice, half the time they don't). The margin term in Section 8.4's r* pushes z positive, giving P_fec > 0.5.

Note: The r* formula from Section 8.4 is a first-order approximation of inverting this P_fec formula. It drops r from the variance denominator (assumes r << s2_burst). For practical r values (0.05-0.25) this is accurate. For exact computation, invert the P_fec formula numerically.

This normal approximation is valid when W >> B (window much larger than mean burst) — which covers all practical scenarios (W=50, B=2-3). For edge cases where W ≈ B, the exact GE distribution is computable via transfer matrix dynamic programming in O(W^2). See Appendix D item 4.

8.3 Burst Variance Correction

The normal approximation in 8.2 assumes iid losses (Binomial variance). On a GE channel, losses are correlated — bursts inflate the variance.

The GE autocorrelation decays with eigenvalue (1-p-q). The variance of losses in a window of size W is:

```
Var_iid(K) = W x e x (1-e)                    (independent losses)

Var_GE(K)  = W x e x (1-e) x σ²_burst         (burst-correlated losses)

σ²_burst = 1 + 2(1-p-q)/(p+q)                  (variance inflation factor)
```

Scenario	p+q	σ²_burst	Meaning
DC	0.5005	3.0	3× wider variance than iid assumption
WiFi	0.513	2.9	similar
LTE	0.42	3.8	significant inflation
Satellite	0.33	5.1	iid approximation seriously wrong

We compute σ^2_burst directly from the GE estimator's $\hat{p}$ and $\hat{q}$. Degenerate estimates need care: when the estimator has observed no Bad-state transitions (very clean channels — the decayed counters empty out), $\hat{q} = 0$ is a NO-DATA sentinel, not a measurement of infinite bursts. Treating it as a measurement makes σ^2_burst $= 1 + 2(1-\hat{p})/\hat{p} \approx 2/\hat{p}$ explode (≈4000 at $\varepsilon = 0.1\%$) and massively over-provisions exactly the cleanest links. No-data must map to the iid default $\sigma^2 = 1$.

Known limitation — repair survival is treated as i.i.d. The model inflates the variance of the loss count K by σ^2_burst but keeps the repair count C ~ Binomial(rW, 1-ε) with independent survival. Repairs cross the same GE channel: a burst that inflates K simultaneously kills interleaved repairs, so Var(C) is also burst-inflated and Cov(K, C) < 0. Both effects widen Var(K - C) beyond what the formula assumes, making P_fec optimistic in exactly the bursty regime σ^2_burst is meant to protect. Monte Carlo validation shows the normal approximation diverging by up to ~12% on high-loss/long-burst channels (LTE-like). For implementation-grade precision, use the exact $O(W^2)$ transfer-matrix computation (Section 8.7), which captures loss/repair correlation exactly.

8.4 The Corrected Optimal Correction Rate

```
r* = max(0,  e/(1-e) + z_{delta/e} x sqrt(e x s2_burst / (W x (1-e))) )
           '--v--'   '--------------------v----------------------'
         IT minimum                 tail margin
                                (accounts for burst correlation)

s2_burst = 1 + 2(1-p-q)/(p+q)

z_{delta/e} = standard normal quantile at (1 - delta/e)
```

Continuity — no cutoffs, the boundary emerges from the closed form. The quantile is taken at 1 - δ/e: the margin depends on how tight the target is RELATIVE to the channel. When δ << e (target much tighter than the loss rate), z is large and the margin dominates. As the channel improves toward the target (e ⟶ δ), z falls through zero, r* drops below the IT minimum, and reaches 0 continuously — at which point pure ARQ meets the tail target with no FEC at all. The max(0, ·) is the physical floor (repair counts cannot be negative), not a policy cutoff; there is no mode switch anywhere on the path from "heavy FEC" to "pure ARQ".

Properties: - $r^*(\delta, e)$ is continuous — it decreases to 0 as δ ⟶ e (Section 11.3 boundary) with no branch or threshold; all regime behavior comes from one closed-form expression - IT minimum ε/(1-ε) is the dominant term when the margin is active - Tail margin scales as $1/\sqrt{W}$ — larger windows need proportionally less margin - z_{δ/ε} controls the margin: tighter δ RELATIVE to ε ⟶ larger z ⟶ more margin - σ^2_burst amplifies the margin for bursty channels (large for small p+q) - On strongly bursty channels this closed form UNDER-provisions the tail (Gaussian tail + ignored loss/repair correlation) — see Section 8.7 for the exact computation and the size of the gap

Note: This formula uses the raw loss rate e. For the canonical production formula including codec overhead, replace e with e_hat = e + e_codec x (1-(1-e)^W) from Section 9.2. The codec-adjusted version accounts for decoder invocation probability on systematic codes.

8.5 Worked Examples

Using z_{δ/ε} = $\Phi^{-1}(1 - \delta/\varepsilon)$ — the margin depends on how tight the target is relative to the channel loss rate.

The margin term is: `z_{δ/ε} × √(ε × σ²_burst / (W × (1-ε)))`

DC (ε=0.001, W=50, σ²_burst=3.0):

```
Bulk (δ=1e-2):     δ ≥ ε → r* = 0   (pure ARQ already meets the target)
Auto (δ=1e-4):     r* = 0.1% + 1.28x0.78% = 0.1% + 1.0% = 1.1%
Realtime (δ=1e-6): r* = 0.1% + 3.09x0.78% = 0.1% + 2.4% = 2.5%
```

WiFi (ε=0.025, W=50, σ²_burst=2.9):

```
Bulk (δ=1e-2):     r* = 2.6% + 0.25x3.86% = 2.6% +  1.0% =  3.5%
Auto (δ=1e-4):     r* = 2.6% + 2.65x3.86% = 2.6% + 10.2% = 12.8%
Realtime (δ=1e-6): r* = 2.6% + 3.94x3.86% = 2.6% + 15.2% = 17.8%
```

Satellite (ε=0.09, W=50, σ²_burst=5.1):

```
Bulk (δ=1e-2):     r* = 9.9% + 1.22x10.0% =  9.9% + 12.3% = 22.2%
Auto (δ=1e-4):     r* = 9.9% + 3.06x10.0% =  9.9% + 30.7% = 40.6%
Realtime (δ=1e-6): r* = 9.9% + 4.24x10.0% =  9.9% + 42.6% = 52.5%
```

Note the emergent behavior across rows. At DC, a Bulk target is looser than the channel loss itself, so FEC vanishes entirely — ARQ suffices. On satellite the SAME δ = 1e-2 sits 9× below ε, so even Bulk needs a 12% margin. The margin responds to the ratio δ/ε, not to δ alone — one protocol hint adapts continuously across channels. The σ²_burst factor still amplifies the margin on bursty channels (satellite's unit margin is 10% vs WiFi's 3.9% per unit z).

8.6 Three-Variable Optimization

Section 8.4 solves for r* given δ (with ρ=100%). Here we generalize to all three modes of the bandwidth/latency/reliability triangle.

Taper with cutoff

When ρ < 100%, the taper is truncated at T_cut:

```
τ(t) = A x (1-q)^t    for t ≤ T_cut
τ(t) = 0              for t > T_cut
```

Total correction rate with cutoff:

```
r = A x Σ_{t=0}^{T_cut} (1-q)^t = A x (1 - (1-q)^{T_cut+1}) / q
```

For T_cut = ∞ (ρ = 100%): reduces to r = A/q (Section 4.3).

Mode 1: Given (δ, ρ) → compute r

Fix tail latency target δ and reliability ρ. Compute minimum bandwidth r*.

Step 1: From ρ, find T_cut. The reliability ρ = P(recovered within T_cut). Using the corrected model (Section 8.4):

```
T_cut such that: e x (1 - P_fec(T_cut)) x (1 - P_arq(T_cut)) = 1 - ρ
```

For ρ = 100%: T_cut = ∞ (no finite solution — see Section 6.4). For ρ < 100%: solve via binary search. P(recovered by T_cut) is monotone increasing in T_cut (more time = more corrections = higher recovery):

```
  Algorithm: find T_cut from ρ

  P_fec = Phi(sqrt(W) x (r(1-e)-e) / sqrt(e(1-e)(r+s2_burst)))   (Section 8.2, time-
independent)

  lo = 0
  hi = W x 10                          (upper bound: many window lengths)
  while hi - lo > tolerance:
      mid = (lo + hi) / 2
      corrections_by_mid = r x (1 - (1-q)^(mid+1))       (taper integral up to mid)
      P_arq(mid) = min(corrections_by_mid / (e/(1-e)), 1) (fraction of needed
corrections available)
      P_recovered(mid) = 1 - e x (1-P_fec) x (1-P_arq(mid))
      if P_recovered(mid) < ρ:
          lo = mid                 (need more time)
      else:
          hi = mid                 (enough time)
  T_cut = hi
```

Convergence is guaranteed because P(recovered) is monotone in T_cut. Typically converges in ~20 iterations (log2 of search range).

Step 2: From δ, find A using the tail latency constraint (Section 8.4):

```
   A* such that: e x (1 - P_fec(A*)) ≤ δ    (among delivered symbols)
```

Step 3: Compute $r^* = A^* \times (1 - (1-q)^{T\_cut+1}) / q$.

Special case ρ = 100%: T_cut = ∞, $r^* = A^*/q = \varepsilon/(1-\varepsilon) + z\_\delta\sqrt{(...)}$. This is the formula from Section 8.4.

Mode 2: Given (r, ρ) → compute δ

Fix bandwidth r and reliability ρ. Compute resulting tail latency δ.

```
   From ρ: find T_cut (same as Mode 1, Step 1)
   From r and T_cut: A = r x q / (1 - (1-q)^{T_cut+1})
   From r and W: P_fec = Phi(sqrt(W) x (r(1-e)-e) / sqrt(e(1-e)(r+s2_burst)))
(Section 8.2)
   Result: δ = e x (1 - P_fec) x P_arq / ρ
```

Mode 3: Given (r, δ) → compute ρ

Fix bandwidth r and tail latency δ. Compute resulting reliability ρ.

```
   From r: A = r x q / (1 - (1-q)^{T_cut+1})     (depends on T_cut)
   From δ: determine how much of the taper is "on-time" vs "late"
   From A and the taper integral: ρ = total recovery probability within T_cut
```

With r fixed, the implied tail latency is monotone in ρ: writing F = e(1-P_fec), Mode 2 gives $\delta(\rho) = F \times P\_arq(\rho) / \rho = 1 - (1-F)/\rho$, which increases with ρ (demanding higher reliability forces more of the FEC misses to be recovered late by ARQ instead of dropped). So binary search on ρ finds the largest reliability whose implied lateness stays within the δ budget:

```
Algorithm: find ρ given (r, δ)

P_fec = Phi(sqrt(W) x (r(1-e)-e) / sqrt(e(1-e)(r+s2_burst)))   (Section 8.2)

lo = 0.5
hi = 1 - 1e-12
while hi - lo > tolerance:
    mid = (lo + hi) / 2                          (candidate ρ)
    P_arq(mid) = clamp(1 - (1-mid)/(e x (1-P_fec)), 0, 1)   (Section 6.3)
    δ(mid) = e x (1-P_fec) x P_arq(mid) / mid   (Mode 2 with ρ = mid)
    if δ(mid) > δ:
        hi = mid        (candidate ρ implies more lateness than allowed)
    else:
        lo = mid        (budget allows higher ρ)
ρ = lo
T_cut = find T_cut from ρ                        (Mode 1, Step 1)
```

Convergence guaranteed by monotonicity. The resulting ρ is the maximum reliability achievable within the bandwidth budget r at tail latency δ.

Worked examples

Example 1: Bulk file transfer (WiFi, ε=0.025)

```
Fix: ρ = 100%, minimize r
Compute: δ (tail latency)

r* = 2.6% + z_{δ/ε} x 3.9%     (from Section 8.5, WiFi row)

At minimum r = r_IT = 2.6%:  P_fec = 0.5 (Section 8.2: z = 0), so
δ = e x (1-P_fec) = 1.25% — half the lost symbols go to ARQ
Tail latency ≈ T_retx + RTT/2 for those symbols
For RTT = 50ms: L_arq ≈ 100ms for 1.25% of symbols

To get δ = 1e-2: r* = 3.5% (Section 8.5, WiFi Bulk row)
```

Example 2: VoIP (WiFi, ε=0.025)

```
Fix: δ = 150ms budget → T_cut = 150ms / symbol_time
     r = 5% (codec + small overhead)
Compute: ρ (reliability)

With r = 5%, the correction budget gives (Section 8.2, W=50, σ²=2.9):
- P_fec ≈ 0.73 (73% of lost symbols FEC-recovered, no added latency)
- Remaining 0.68% of all symbols go to ARQ; one retransmit round
  (L_arq ≈ 100ms) fits the 150ms budget, succeeding w.p. ≈ 1-e
- P(miss deadline) ≈ e x (1-P_fec) x e ≈ 0.02%  ->  ρ(150ms) ≈ 99.98%

The 150ms budget is generous at RTT = 50ms: reliability is limited by
double losses (symbol AND its retransmit both lost), not by the
correction budget. The VoIP codec conceals the residual frame loss.
```

Example 3: Live video (WiFi, ε=0.025)

```
Fix: δ = 33ms (one frame at 30fps), ρ = 99.9%
Compute: r (bandwidth)

Need 99.9% of symbols delivered within 33ms.
T_cut determined by ρ = 99.9%: T_cut ≈ 3 x RTT
A determined by δ: need P(recovery within 33ms) ≥ 0.999
33ms < L_arq, so recovery must be FEC — r* ≈ 13-18%
(between the Auto and Realtime rows of Section 8.5)
```

Example 4: Gaming (LTE, ε=0.05)

```
Fix: δ = 20ms (tight), ρ = 99% (1% loss acceptable)
Compute: r (bandwidth)

Very tight latency + moderate reliability → aggressive FEC
T_cut ≈ 2 x RTT (short: accept 1% loss)
r* ≈ 11% (δ/ε = 0.2 → z = 0.84 → margin ≈ 5.3% over the 5.3% IT
minimum; the ρ = 99% cutoff trims the taper tail further)
```

Example 5: Sensor telemetry (Satellite, ε=0.09)

```
Fix: r = 5% (minimal bandwidth), ρ = 95%
Compute: δ (tail latency)

Low bandwidth budget + high loss → many symbols go to ARQ
T_cut determined by ρ = 95%
δ ≈ 15% (15% of delivered symbols arrive late)
Acceptable for periodic sensor readings.
```

8.7 Exact P_fec via Transfer-Matrix DP

Both gaps in the normal model — burst-inflated repair variance and the negative K–C correlation (Section 8.3) — vanish if we compute the joint distribution of losses and surviving repairs EXACTLY, by walking the GE chain across the interleaved wire sequence.

Setup. A window of W source symbols with R = round(r × W) repairs interleaved evenly: N = W + R wire slots with fixed types T_i ∈ {source, repair} (slot i is a repair iff ⌊(i+1)R/N⌋ > ⌊iR/N⌋). The channel is the two-state GE chain started from its stationary distribution; a symbol is lost iff the chain is Bad at its slot. Because ONE chain walks across both slot types, burst-correlated source losses, burst-correlated repair erasures, and the negative correlation between them are all captured.

Recursion. Track the running deficit D = (#source losses) – (#surviving repairs). FEC succeeds for the window iff D ≤ 0 at the end — the same criterion as Section 8.2, applied to the exact joint distribution. With f_i(x, d) = P(chain in state x, deficit d after slot i):

```
f_0(G, 0) = π_G = q/(p+q),   f_0(B, 0) = π_B = p/(p+q)

step to slot i+1 of type T:
  mass arriving in state x':  Σ_x f_i(x, d) × P(x → x')
  d' = d + 1   if T = source and x' = B     (source lost)
  d' = d − 1   if T = repair and x' = G     (repair survives)
  d' = d       otherwise

P_fec = 1 − Σ_{d > 0} Σ_x f_N(x, d)
```

State space 2 × (W+R+1) over N slots ⟶ O(W²) work (≈ 6,000 operations at W = 50, r = 0.1) — trivially cheap. Validation: on a memoryless channel (p + q = 1) the DP reproduces the

independent-Binomial reference to machine precision, and against Monte Carlo it agrees to sampling error (< 0.002 at 20k trials), where the normal approximation errs by ≈ 1–2%:

```
Scenario (W=50)          r      exact    normal(8.2)  |error|
WiFi  (p=.013, q=.5)    0.10   0.9522    0.9695       0.017
LTE   (p=.02,  q=.4)    0.12   0.8868    0.8694       0.017
Sat   (p=.03,  q=.3)    0.25   0.9180    0.9272       0.009
```

(R = round(r × W), half rounding away from zero — e.g. Sat: R = 13.)

Exact r*. P_fail(r) = 1 − P_fec(r) decreases in r (up to the 1/W rounding of R), so binary search yields the exact minimum rate for P_fail ≤ δ. The tail is where the normal approximation is weakest, and the effect is material:

```
δ = 1e-2, W = 50       r*_exact    r*_normal (8.4)
WiFi                     0.170       0.116
LTE                      0.270       0.193
Satellite                0.450       0.334
```

The closed-form r* UNDER-provisions by ~30–50% of itself on bursty channels: the K–C correlation widens Var(K − C) beyond the model, and the true tail of K − C is heavier than Gaussian. The closed form remains valuable for insight and cheap incremental updates; rate selection on strongly bursty channels should use the exact computation (implemented as `p_fec_exact` / `compute_r_star_exact` in raptorpath-math).

Caveats. Codec overhead is not modeled (apply ε_codec_eff from Section 9.2 on top). The success criterion inherits Section 8.2′s per-window block view of the sliding-window code. R is rounded to an integer, so r* is resolved in steps of 1/W.

8.8 Choosing the Window

Every rate formula so far takes W as a given. But W is itself a knob with large, opposing effects, and picking it by hand ("W = 64") leaves free overhead or free latency on the table. This section derives W* from the same channel state the rate uses, closing the last free parameter.

W pulls three ways. All three appear elsewhere in this document; here they are read as bounds on a single choice.

```
1. Overhead  (Section 8.4).  The r* margin is
      margin(W) = z x sqrt(eps x sigma2 / (W (1-eps))),   z = z_{delta/eps}
   It decays as W^{-1/2}, with slope d(r*)/dW ~ -W^{-3/2}: strong at
   small W, flattening fast. Bigger W => less overhead. (favours large W)

2. Latency   (Sections 14.5, 14.21, 14.25).  A window loss waits for a
   covering repair within the window horizon, which is traversed at the
   source rate: recovery latency ~ W x t_sym = W / send_rate. It is also
   the tail_svc dilution term of Section 14.21 and the coverage span of
   the completion glide (Section 14.26). Bigger W => slower recovery,
   and once in-flight span >> W the last window cannot cover the exposed
   tail (Section 14.25). (favours small W)

3. Burst absorbency (Section 14.5).  Ambient FEC at r = eps/(1-eps) must
   accumulate B surviving repairs to cover a mean burst B = (sigma2+1)/2.
   (needs W at least a floor)
```

There is no scale-free "knee" in a bare W^{-1/2} law — diminishing returns appear only RELATIVE to a fixed scale. The natural scale is the IT floor eps/(1-eps): keep sizing the window

up while the margin is a meaningful fraction of the floor it sits on; stop once the margin has been pushed to a fraction alpha of it. That, the latency ceiling, and the burst floor give three closed-form bounds:

```
W_over = z^2 x sigma2 x (1-eps) / (eps x alpha^2)      (margin <= alpha x floor)
W_lat  = budget x send_rate                            (recovery <= budget)
W_bur  = B / (eps x (1-eps)),   B = (sigma2+1)/2        (absorb a mean burst)

W* = clamp( W_over,  min(W_bur, W_lat),  W_lat ),  then clamp to [16, 512]
```

with z = z_{delta/eps} = Phi^{-1}(1 - delta/eps) (the SAME quantile as r*, Section 8.4), alpha = 0.25, and `budget` the Realtime hint's latency budget or ~1 RTT otherwise — the Section 14.5 setting W x t_sym ~ RTT that aligns the FEC and ARQ recovery horizons. The burst floor never overrides the latency ceiling (`min(W_bur, W_lat)`): if a mean burst cannot be absorbed within the budget, no window size fixes it and latency wins — the Realtime reliability/latency trade made explicit.

Continuity and the three regimes. `clamp`/`min`/`max` are continuous (piecewise-linear), so W* is continuous in every input and finite by the [16, 512] clamp. Which term binds is itself the story:

- **Loose delta (Bulk, delta >= eps).** z <= 0, so W_over = 0: no margin to amortise. The window collapses to the burst floor min(W_bur, W_lat) — the SMALLEST window that still catches a mean burst, minimising recovery latency and decode cost. This is exactly the "no steady-state FEC" stance of Sections 14.25/14.26 read through W instead of r.
- **Tight delta (Auto/Realtime).** The margin dominates the small IT floor, W_over is large (often unreachable), and W* rides the latency ceiling W_lat: as large as the budget allows, to shrink the overhead as far as latency permits.
- **Moderate delta/eps.** W_over lands between the floor and the ceiling and the overhead knee itself binds (WiFi-Bulk below).

Monotonicities (verified as unit tests in raptorpath-math): tighter delta raises z and W_over, so W* is non-decreasing as delta tightens (until capped by W_lat); higher sigma2 raises both W_over and W_bur, so W *increases with burst variance; a tighter latency budget lowers W_lat and shrinks W*. Degenerate inputs are safe: no throughput/RTT sample disables the latency ceiling (W_lat = 512, overhead knee governs), and eps -> 0 leaves the burst/overhead terms inert so latency alone sets W.

Worked examples. Per-hint delta and latency budget: Bulk (delta = 1e-2, budget = 2 RTT), Auto (1e-4, 1 RTT), Realtime (1e-6, 0.5 RTT). `r*(W*)` is the resulting overhead at the derived window; `r*(64)` is the overhead a fixed W = 64 would pay at the same target; `recov` = W* / send_rate.

```
DC          eps=0.001  sigma2=3.0  RTT=1 ms    send_rate=100k/s
  hint       delta     W*    r*(W*)   r*(64)   recov    binding
  Bulk       1e-2     200     0.0%     0.0%    2.0 ms   latency (pure ARQ)
  Auto       1e-4     100     0.8%     1.0%    1.0 ms   latency
  Realtime   1e-6      50     2.5%     2.2%    0.5 ms   latency

WiFi        eps=0.025  sigma2=3.0  RTT=13 ms   send_rate=10k/s
  hint       delta     W*    r*(W*)   r*(64)   recov    binding
  Bulk       1e-2     120     3.2%     3.4%   12.0 ms   overhead knee
  Auto       1e-4     130     9.0%    11.8%   13.0 ms   latency
  Realtime   1e-6      65    16.1%    16.2%    6.5 ms   latency

Satellite   eps=0.09   sigma2=5.0  RTT=210 ms  send_rate=2.04k/s
  hint       delta     W*    r*(W*)   r*(64)   recov    binding
  Bulk       1e-2     512    13.7%    20.6%  250.9 ms   overhead (W_max)
  Auto       1e-4     429    20.3%    36.8%  210.0 ms   latency
  Realtime   1e-6     214    30.3%    47.2%  105.0 ms   latency
```

The overhead saving is largest exactly where W matters most: on Satellite the derived window nearly halves the fixed-64 overhead (Auto 20.3% vs 36.8%) because the 1/sqrt(W) margin is fattest at high eps, while recovery latency stays within the hint's budget by construction. On DC every hint is latency-bound — the channel is clean enough that overhead is negligible at any window, so W* simply maximises the window the budget allows. `derive_window` in raptorpath-math is the shared implementation; the production window-mode sender and the visualizer both read W* from it.

---

9. Codec Overhead Integration

9.1 Decoder Invocation Probability

For systematic codes (METTLE, RLC, RaptorQ), the decoder is only invoked when at least one source symbol in the window is lost:

```
P(decoder invoked) = 1 - (1-e)^W
```

Scenario	ε	W=50	W=200
DC	0.001	4.9%	18.1%
WiFi	0.025	72.1%	99.4%
Satellite	0.09	99.1%	≈100%

9.2 Effective Codec Overhead

The codec overhead (ε_codec) represents extra symbols the decoder needs beyond the information-theoretic minimum:

Backend	ε_codec	Reason
Reed-Solomon	0.0%	MDS: any k of n suffices
RLC	0.4%	Near-MDS: rare rank deficiency [RFC8681]
RaptorQ	1.0%	Fountain code convergence overhead [RFC6330]

Backend	ε_codec	Reason
METTLE	15.0%	Sparse random matrix: needs more eqns [Yu2025]
Streaming	0.0%	Rate-optimal by construction [Badr2017]

Weighted by decoder invocation probability:

```
e_codec_eff = e_codec x P(decoder invoked) = e_codec x (1 - (1-e)^W)
```

The corrected correction rate becomes:

```
r* = max(0, (e + e_codec_eff)/(1-e)
           + z_{δ/e_hat} x √((e + e_codec_eff) x σ²_burst / (W x (1-e))))
```

(the same margin structure as Section 8.4, with e replaced by e_hat = e + e_codec_eff — including inside the quantile ratio δ/e_hat).

9.3 Impact on METTLE at DC

At DC (ε=0.1%, W=50, σ^2=3.0, Auto δ=1e-4 — under Bulk the DC rate is 0 with or without weighting, so Auto is the informative case):

Without weighting: ε_hat = 0.1% + 15% = 15.1%, z_{δ/ε_hat} = 3.21 ⟶ r^* = 15.1% + 30.6% = 45.7%. With weighting: ε_codec_eff = 0.15 × 0.049 = 0.74%, ε_hat = 0.84%, z_{δ/ε_hat} = 2.26 ⟶ r^* = 0.8% + 5.1% = 5.9%.

The weighting reduces METTLE's DC overhead by ~8×.

---

10. Multi-Protocol Extension

10.1 Per-Symbol Latency Classes

Different traffic types have different δ targets:

```
Symbol s has latency class c(s) with target δ_{c(s)}
```

10.2 Interleave Before vs After FEC

After FEC (separate streams):

```
Realtime packets ⟶ [FEC encoder (δ=1e-6)] ⟶ channel
Bulk packets     ⟶ [FEC encoder (δ=1e-2)] ⟶ channel
```

Each stream has its own taper. No correction sharing. Simple.

Before FEC (shared stream):

```
All packets ⟶ [mixed FEC encoder] ⟶ channel
```

One taper covers everything. Repair symbols are linear combinations of ALL source symbols [RFC8681] — a repair can recover ANY lost symbol regardless of class.

Advantage of shared: repair symbols are fungible. A repair generated "for" a Realtime symbol can recover a Bulk symbol if needed. Total correction budget can be lower than the sum of separate budgets (statistical multiplexing).

Disadvantage of shared: the taper must be sized for the tightest class. If 1% of traffic is Realtime (δ=1e-6) and 99% is Bulk (δ=1e-2), you pay Realtime-level overhead on everything.

10.3 When Shared Wins

Shared FEC is cheaper when the traffic mix is balanced or dominated by the tight class. Separate streams are cheaper when the tight class is a small fraction. The crossover depends on the specific δ values and loss rate.

Decision rule: Compare total correction bandwidth:

```
shared_cost = r*(e, min(δ_c))         (one encoder, tightest δ)
separate_cost = Σ_c f_c x r*(e, δ_c)  (per-class, weighted by fraction f_c)
```

Choose whichever is lower.

10.4 Extending the Formula

For shared FEC with per-symbol δ, the constraint becomes:

```
For each class c: e x (1 - P_fec) ≤ δ_c
```

Since P_fec is the same for all symbols (shared repair), the binding constraint is the tightest: δ_min = min(δ_c). The formula reduces to the single-class case with δ = δ_min.

---

11. Verification

11.1 Simulation Approach

Generate synthetic loss traces from the GE model, apply the taper function, and measure whether the actual tail latency matches the theoretical prediction.

```
For each trial:
  1. Generate GE loss trace: {lost_1, lost_2, ..., lost_N}
  2. Generate correction schedule from taper: {correction_1, correction_2, ...}
  3. Apply channel loss to both source and correction symbols
  4. For each lost source symbol:
     a. Check if enough repair symbols arrived (FEC recovery)
     b. If not, mark as ARQ-recovered (retransmit needed)
  5. Count ARQ events (symbols not FEC-recovered)
  6. Measure: actual_arq_fraction = arq_events / N

Over many trials:
  Verify: P(actual_arq_fraction > δ) is small
  Verify: mean correction rate ≈ r*
```

11.2 Analytical Predictions to Verify

Prediction	Formula	Test
IT minimum dominates at high loss	$r^* \approx \varepsilon/(1-\varepsilon)$ when W large	Satellite scenario
Tail margin scales as $1/\sqrt{W}$	$r(W=200) < r(W=50)$	Compare window sizes
Taper shape matches GE	Decay rate = $\hat{q}$ from estimator	Compare simulated vs theoretical
Codec overhead weighting	METTLE DC overhead ~ 0.74% not 15%	DC scenario with METTLE
Protocol hint only affects δ	r(Realtime, δ=1e-6) vs r(Auto, δ=1e-4) differ only via z_δ	Same estimator, different hints

11.3 Boundary Cases

Case	Expected behavior	Why
$\varepsilon = 0$ (no loss)	$r^* = 0$	No correction needed
$\varepsilon \to 1$ (total loss)	$r^* \to \infty$	Can't recover anything with FEC alone
$\delta = \varepsilon$ (every loss to ARQ)	$r^* = 0$	No FEC needed, all ARQ
$\delta \to 0$ (zero ARQ tolerance)	$r^* \to \infty$	Must FEC-recover everything
$W = 1$ (no window)	Margin term large	Single-symbol recovery needs more redundancy
$q = 1$ (no burst memory)	$\tau(t)$ = flat	Reduces to iid case

11.4 Connection to Existing Benchmarks

The bench_suite already measures `overhead_pct` and `recovery_pct` per scenario. To verify the model:

1. Compute r^* from the formula for each scenario's (ε, δ, W)
2. Compare with the bench_suite's measured overhead
3. The gap between r^* and measured overhead = estimation tax + implementation overhead
4. Track this gap across benchmark runs — it should decrease as we improve the implementation

Canonical evaluation suite. The scenario matrix, baseline fidelity ladder (in-process model → real TCP/QUIC/MPTCP stacks over netem → real links), and the quantitative win conditions that make "surpasses TCP/MPTCP" a falsifiable claim are defined in ADR-0051 (canonical evaluation scenarios). The channel rows there are exactly the Section 2.4 parameterizations, so the model, the simulator, and the benchmarks share one vocabulary. The suite deliberately includes cells where naive FEC loses (clean links, congestion-dominant loss with a competing TCP flow, mid-transfer path outage) — ties are the win condition there.

---

12. Congestion Control Integration

12.1 Why Delay-Based CC is Required

On lossy links (WiFi, LTE, satellite), loss-based CC (NewReno, CUBIC) is catastrophic. Every random channel loss triggers cwnd halving, collapsing throughput on exactly the links raptorpath is designed for:

```
NewReno on 2.5% WiFi loss:
  -> cwnd halves on every loss event
  -> throughput oscillates wildly, average << capacity
  -> our FEC tries to compensate -> more traffic -> more congestion

Delay-based CC on 2.5% WiFi loss:
  -> loss + stable RTT = channel loss -> ignore, let FEC handle it
  -> loss + rising RTT = congestion   -> reduce rate
  -> throughput stays near capacity
```

Delay-based CC distinguishes congestion (queue buildup = rising RTT) from channel loss (random drops = stable RTT). This is essential for raptorpath.

12.2 QUIC Datagrams Bypass Quinn's CC

Raptorpath sends symbol data as QUIC unreliable datagrams, which bypass Quinn's built-in congestion control (NewReno). Our own CC is the sole rate limiter for data traffic. Quinn's CC only applies to QUIC streams (handshake, reliable control messages — small and infrequent).

This means our CC is solely responsible for not flooding the network.

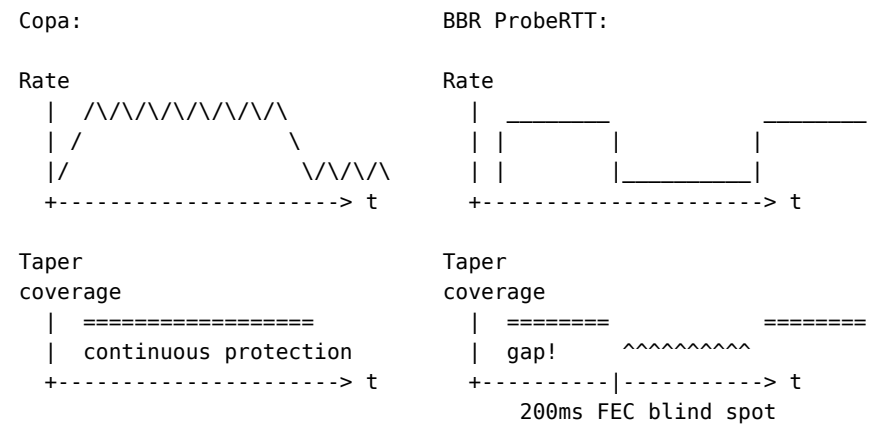
12.3 Copa vs BBR

The current implementation uses BBR (ADR-0019). We recommend migrating to Copa [Copa2018] because of better interaction with the taper function:

Aspect	BBR	Copa
Core idea	measure max_bw x min_rtt = BDP	rate = 1/(d x dq)
Queue draining	ProbeRTT: 200ms forced drain every 10s	Natural oscillation
Phases	Startup -> ProbeBw -> ProbeRtt (state machine)	No phases, one formula
Lossy links	RTT-trend check to ignore random loss	Delay-based, same effect
Complexity	~300 lines, multiple edge cases	One rate formula
Taper interaction	ProbeRTT creates FEC protection gap	Smooth, no gaps

Implementation status: Copa is recommended as the target congestion control algorithm. The current codebase uses BBR (ADR-0019). Migration to Copa is a future implementation task. The model and formulas in this paper are CC-agnostic — they work with any delay-based CC that provides a total sending rate.

The critical difference — taper compatibility:



Copa's rate oscillation is smooth and fast (order of RTT). The taper function uses RATIO (corrections per source symbol), which stays constant. Absolute rates oscillate gently, but the EWMA smooths throughput estimates. The taper doesn't notice Copa's oscillation.

BBR's ProbeRTT drops cwnd to 4 symbols for 200ms. During this period: - Source rate drops to nearly zero - Correction rate drops proportionally (ratio constant, absolute count tanks) - Source symbols sent during ProbeRTT get almost no FEC protection - If a burst loss hits

during ProbeRTT, recovery is severely degraded - After ProbeRTT exits: data burst -> potential queue buildup

12.4 Copa's Rate Formula

Copa targets a queue occupancy that balances throughput and delay:

```
rate = 1 / (d_copa x dq)

where:
  d_copa = Copa parameter (controls target queue depth; default 0.5)
  dq     = queuing_delay = RTT_current - RTT_min
```

When dq is small (queue empty): rate is high -> fill the pipe. When dq is large (queue building): rate drops -> drain the queue.

This naturally oscillates: send fast -> queue builds -> send slow -> queue drains -> send fast again. The oscillation frequency is ~1/RTT and amplitude depends on d_copa. No periodic forced drain phase needed.

Copa's d_copa is NOT the same as our tail latency target δ. Copa's d_copa controls the target queue depth at the bottleneck (1/d_copa packets). Our δ controls the tail probability of late delivery. They are independent parameters that happen to use similar notation in their respective papers.

d_copa = 0.5 (from the Copa paper [Copa2018]) nominally targets a queue of 2 packets. That does NOT mean the realized cost is ~1 packet of delay: at high utilization the sender oscillates around the controller's queue target, so the median delivery latency carries a standing queue on the order of that target — not of a single packet. Measured in the gate simulation (100 Mbps WiFi under sustained load), the tolerated standing queue adds ~9 ms to the median latency versus an idle baseline (p50 ~17.6 ms loaded vs ~8.1 ms idle) — a dominant latency term, not a negligible one. The protocol hint must therefore set the delay target too, not only the FEC rate: Realtime ⟶ a near-empty queue target (accepting some utilization loss), Bulk ⟶ a deeper queue target for better link utilization, and Auto keeps the d_copa = 0.5-style default. A continuous mapping from the tail latency budget to d_copa is the natural refinement; a coarse per-hint target already recovers most of the standing-queue latency.

min_rtt estimation: Copa uses the minimum observed RTT in a 10-second sliding window (same duration as BBR's min_rtt window). Copa's natural rate oscillation periodically reduces the queue to near-empty, refreshing the minimum within this window. If min_rtt has not been refreshed for an extended period (>20s), the system can force a brief rate reduction as a fallback — though this is rarely needed because Copa's oscillation provides natural refreshing.

Production implementation notes (P7 port). The production scheduler implements the Copa-lite scheme exactly as proven in the gate driver (windowed-min queue signal, hint-coupled queue target x{1.08, 1.125, 1.25}, two-speed ramp x1.5+1 then +2/x0.92, per-SRTT update cadence, token-bucket pacing at cwnd/SRTT with burst max(10, cwnd/8)), with these honest deviations in constants and mechanics:

- **Propagation floor** = min RTT sample over the 10 s sliding window (the driver used the lifetime min because its trials were shorter than any sane window; the 10 s window is the faithful production analogue and self-heals after a route change).

- **dq clamp at 0.1 ms**, applied to BOTH the queuing-delay estimate and the backoff threshold (queue_mult - 1) x floor. This keeps the closed-form rate 1/(d_copa x dq) finite on LAN-class floors and makes the backoff comparison continuous there (dq at the clamp cannot exceed a threshold at the same clamp) — no branch cliffs. The rate formula itself is retained as a diagnostic only; the cwnd dynamics are the ramp/backoff scheme.
- **Jitter-robust queue signal** (three coupled changes, one cause). The P1 mapping implicitly assumed path jitter << the queue target; the L1 C2 cell (10 ms floor, netem +-3 ms per direction) violates it, and the sender took a x0.92 backoff on ~60% of updates — cwnd pinned near the 8-symbol floor vs BDP ~160 (the dominant term in the 16x rp-vs-quinn gap at C2). Measured root cause: the queue signal compares a min-of-N statistic (N ~ 4-30 ACK samples per SRTT window) against the 10 s propagation floor, a min-of-thousands. Under jitter these are different statistics: at C2 the 10 s floor found 7.0 ms while a typical window min sits at 12-13 ms — a permanent apparent dq of ~5 ms with an EMPTY queue, which no windowed min can see through (and the link's jitter FIFO correlates consecutive samples, so consecutive-difference jitter measures only ~0.85 ms of the ~6 ms spread). The production remedy:

  (1) **Quantile queue floor**: the backoff comparison uses queue_floor = P10 of the per-update window-min history (10 s window) instead of the raw 10 s min — the floor becomes the same min-of-N statistic as the signal, so it is self-calibrating under any jitter correlation structure. A genuine standing queue shifts every window min within one SRTT while the 10 s quantile lags, so congestion is still detected within a few updates; a queue SUSTAINED longer than the window becomes the new baseline (the same staleness bound the 10 s min floor already had).

  (2) **Jitter headroom**: threshold = (queue_mult - 1) x queue_floor

  ▸ k x max(jitter_est, win_jitter_est), k = 2. jitter_est is an EWMA (gain 1/8) of |consecutive RTT sample differences| (RFC 3550-style); win_jitter_est is the SAME consecutive-difference EWMA one level up (gain 1/4, over per-update window minima) — under correlated jitter the raw differences collapse (~0.85 ms at C2) while window minima wander 3-5 ms per update, and only the window-level estimator sees that amplitude. Both are shift-robust: a standing queue contributes one transition sample, not persistent inflation (a quantile-spread term was rejected for exactly this reason — a level shift inflates it for a full window and kills congestion detection; caught by the cwnd-floor unit test).

  (3) **Ramp fast-exit needs >= 3 samples**: a partial window's min can be a single draw from the jitter tail; one sample must not end the exponential ramp. Continuity: on a clean link every window min equals the floor, the quantile equals the floor, jitter_est -> 0, and P1 semantics are recovered exactly; no cliff in any variable. Honest cost: the tolerated standing queue grows to ~(queue_mult - 1) x queue_floor + 2 x jitter above the TYPICAL window min rather than the extreme min; for Realtime this is fundamental, not incidental — a queue smaller than the jitter spread is statistically indistinguishable from jitter at windowed-min sample counts.
- **cwnd floor = 8 symbols** (the driver backed off to >= 4). An L1 run on a real emulated link (100 Mbit, 10 ms RTT) showed the pre-P7 rate-formula collapse crawling at cwnd = 2; the raised floor guarantees a trickle that keeps RTT samples, and therefore recovery, flowing.

- **Ramp fast-exit**: during the ramp the backoff check also runs per ACK (not only per SRTT), so the exponential phase ends within one feedback message of the first standing-queue evidence.
- **Pacing is symbol-level via a carry queue**: the interleaver's drain is all-or-nothing, so drained symbols land in a per-path carry queue and each pace tick sends only floor(tokens) symbols; the remainder waits for the next tick (wakeup = next-token refill time, clamped 0.5-50 ms). The first cut gated at batch granularity and let a whole block overdraft; measured at L1 (C2, Bulk) every 56-symbol block burst serialized ~5.4 ms of self-queue — above Bulk's 2.5 ms backoff threshold — so every block bought a x0.92 backoff and cwnd pinned just under one block (~34 symbols). Symbol-level pacing removes the self-queue entirely. Carried symbols count toward the TUN-read gate (in_flight + carried >= cwnd), which remains the outer backpressure.
- **Bulk flush timeout 5 ms** (was 50 ms): while the CC gate pauses TUN reads, 64KB block assembly stalls mid-block; a 50 ms flush serialized with the congestion window and clumped the pipeline into ~300 ms ACK bursts at L1. 5 ms bounds the assembly wait well under one C2 RTT.
- **in_flight is a schedule-time budget with time-based release.** Each symbol is charged exactly once when scheduled (covering interleaver + pacing carry + wire) and released by ACK feedback. Because ACKs are best-effort datagrams, a lost ACK strands its release; stranded budget expires after max(4 x SRTT, 250 ms) (RFC 9002-style time threshold at budget granularity) instead of jamming the TUN gate until a 2 s decay backstop. The second L1 round found the gate cycling at exactly that 2 s cadence (~30 KB/s): the send path was charging the budget a SECOND time at wire time, leaking +1 per symbol. Pacing tokens, not the gate, remain the wire-rate limiter, so an early expiry can only let the encoder run ahead, never the wire.
- **Loss reaction** (the driver has none, by design): a decode FAILURE with the standing queue above target takes the same x0.92 backoff as the delay signal; a decode failure with an empty queue ends the ramp and steps cwnd down by 1 (FEC under-provision, not congestion). Loss that FEC recovered never touches cwnd.

12.5 CC + Taper: The Complete Architecture

```
Copa determines: total_rate (symbols/sec on the wire)
Taper determines: r* (correction symbols per source symbol)

source_rate     = total_rate / (1 + r*)
correction_rate = total_rate x r* / (1 + r*)

When channel worsens: e rises -> r* rises -> source_rate falls
When congestion:      dq rises -> total_rate falls -> both fall
When channel clears:  e drops -> r* drops -> source_rate rises
```

The two controllers are orthogonal: - Copa controls HOW FAST (total symbols per second) - Taper controls HOW MUCH redundancy (correction per source ratio)

Neither needs to know about the other. Copa sees total traffic; taper sees loss rate. They compose naturally.

Bulk operating point. Under the Bulk hint the taper side degenerates by design: the tail target is "late is fine" weighted by completion exposure ($\delta\_bulk = \hat{\varepsilon} + (0.05 - \hat{\varepsilon})\cdot\chi$, see Sections 5.3, 14.26), so the continuous r* is 0 identically mid-stream ($\chi = 0$) and the steady state is pure

ARQ — source_rate ≈ total_rate, wire volume at parity with a retransmission transport. FEC reappears only near a known end of stream, where χ ramps to 1 and recovery can no longer overlap sending (Sections 14.25, 14.26).

12.6 BtlBw-Anchored Recovery

Copa-lite's steady-state recovery is additive: after a ×0.92 delay backoff, cwnd climbs at +2/SRTT. From a trough at half the pipe that is dozens of SRTTs of under-utilization. Measured at L1 C2 (100 Mbit, 5 ms one-way, ±3 ms jitter, 1.3% loss): cwnd p50 sat at ~80-110 symbols against a BDP of ~160 — the additive climb never caught up before the next jitter-driven backoff (the residual ~30% backoff rate the §12.4 jitter-robust signal could only halve).

BBR does not crawl: it holds an explicit operating point BtlBw × RTprop and returns *to* it [BBR-Queue]. We already track both quantities — a delivery-rate max-filter `max_bw` and the 10 s min-RTT floor `min_rtt` (§12.4) — but they fed only the diagnostic rate formula. This section promotes their product to an active recovery anchor, keeping the continuous, phase-free character of Copa-lite (no Startup/Drain/ProbeBW state machine, no ProbeRTT drain — §12.3′s taper-compatibility argument is preserved because nothing here drains the pipe).

The estimator.

```
BtlBw  = max_bw  = windowed MAX of per-update delivery-rate samples
                   (symbols/s), 10 s window
RTprop = min_rtt = windowed MIN of RTT samples (s), 10 s window
BDP    = BtlBw × RTprop                                (symbols)
```

Units check: (symbols/s) × s = symbols — the in-flight the bottleneck rate keeps outstanding over one propagation RTT, i.e. a congestion window.

The continuous anchor pull. In the steady-state (post-first-backoff) per-SRTT update, the additive step is replaced by a proportional pull toward the BDP target that decays into the additive probe as cwnd approaches it:

```
target = cwnd_gain · BDP                              (cwnd_gain = 1.0)
cwnd  += max( ADDITIVE_STEP, α · (target − cwnd) )   when cwnd < target
cwnd  +=      ADDITIVE_STEP                            when cwnd ≥ target
```

with $\alpha = 0.25$. This is a first-order relaxation toward the operating point: from a trough at 0.5·BDP it closes ~90% of the gap in ~8 SRTTs versus ~40 for +2, and the pull term vanishes smoothly into the gentle +2 probe as cwnd → target (no discontinuity, no phase counter). cwnd_gain is 1.0, not BBR's ProbeBW cwnd_gain = 2: BBR sizes cwnd to 2·BDP so it keeps sending through one RTT of delayed ACKs and deliberately tolerates a 1·BDP standing queue [CACM]; here the standing queue ABOVE BDP is still governed by the §12.4 hint-coupled delay backoff, so the anchor only needs to restore the pipe, not to buffer a second one.

Why it is a FLOOR, not a cap — the app-limited caveat. BBR's BtlBw is trustworthy only because of two mechanisms we do not have: **per-packet** delivery-rate sampling, and **app-limited detection** that *discards* samples taken while the sender was application- (or window-) limited, because such samples measure the application's send rate, not the bottleneck's [BBR-Draft, delivery-rate-estimation]. Our `record_delivery` divides coarse ACK-batch counts by wall-clock elapsed and has no app-limited flag. For a warm-up-limited transfer — the dominant regime for a 1.8 MB object, which spends much of its life in inner-flow slow-start — `max_bw` reads LOW exactly when we would want it high. A BtlBw used as a *cap* would then throttle

a flow that could have gone faster; a BtlBw used only as a *floor* can, at worst, fail to lift cwnd — it can never suppress it. So the anchor is applied strictly to RAISE cwnd:

- the recovery pull only ever increases the additive step (never below +2);
- an explicit floor `cwnd ≥ ANCHOR_FLOOR_GAIN · BDP` ratchets cwnd up after any backoff, bounded above by the hard cwnd ceiling.

Both are gated: the anchor is `None` until the delivery-rate window holds ≥ 8 samples AND a min-RTT sample exists (a handful of coarse samples is too noisy to steer cwnd; before an RTT floor there is no RTprop). A stale or over-read `max_bw` (ACK aggregation can momentarily inflate a batch rate, the mirror risk to under-read) is bounded by `ANCHOR_FLOOR_GAIN < 1` and by the delay backoff retaining authority above the floor.

Honest constants (and how ANCHOR_FLOOR_GAIN was set).

Symbol	Value	Meaning
`ANCHOR_MIN_SAMPLES`	8	delivery samples before the anchor is trusted
`ANCHOR_RECOVERY_GAIN` (cwnd_gain)	1.0	pull target = 1·BDP
`ANCHOR_PULL_ALPHA` (α)	0.25	relaxation rate toward the target per SRTT
`ANCHOR_FLOOR_GAIN`	0.85	floor = 0.85·BDP

The floor gain started at 1.0 (floor = full BDP) and was corrected to 0.85 by L1 measurement. At 1.0 the floor pinned cwnd exactly at the BDP estimate even when the delay signal reported queue-above-target — the C2 cwnd trace showed `above=true` on ~100% of updates with cwnd held at bdp_anchor, i.e. the floor was *maintaining* a ~16 ms standing queue the backoff could no longer drain. 0.85 leaves the §12.4 delay backoff ~15% of authority around BDP: the above-target update fraction fell to ~52% (the queue drains on roughly half the updates), while the recovery pull still re-fills toward full BDP each clean update, so cwnd oscillates just under the pipe instead of sitting in standing bufferbloat.

Interaction with the taper. None — this changes only the cwnd trajectory, not the correction ratio r*, and introduces no drain phase, so the §12.5 taper coverage stays continuous (the reason §12.3 rejected BBR's ProbeRTT does not apply). The two controllers remain orthogonal (§12.5): Copa-lite-with-anchor sets HOW FAST, the taper sets HOW MUCH redundancy.

Measured result — mechanism confirmed, completion refuted. L1 rp-native (`perf`, no inner TCP, 1.8 MB, seed 42, 10 runs/arm) at C2: cwnd p50 rose from the ~80-110 baseline to **139** (peaks 165, bdp_anchor p50 137) — cwnd now reaches BDP as intended. But the C2 median completion was flat: 0.883 s baseline ⟶ 0.911 s anchored, inside one run-to-run standard deviation (~0.2 s), and each aggressiveness step moved the median <10% (converged). C3 was flat within its larger variance. This matches the bounded-leverage prediction that motivated the change (`docs/research/bbr-lessons.md` #1): the 1.8 MB completion is dominated by the tunnel-pipeline / inner-flow warm-up term (L2 ws3, fair-geometry), and the delay backoff binds only 8-24% of ACKs, so re-filling the window to BDP does not move that headline. The change fixes a genuine, measured cwnd deficiency and is retained for that reason — its expected payoff is the sustained-throughput and multipath-aggregation cells, where

`B_eff` reads cwnd/SRTT directly (§13.5) — but the completion improvement is a **refutation**, recorded honestly alongside P10a (`docs/goal-gate.md`, P-CC).

12.7 ECN as Opportunistic Enhancement

If the network path supports ECN [RFC3168], congestion is signaled by router marking (CE bit) instead of dropping. This provides: - Congestion detection without loss -> even better for delay-based CC - Positive identification: marked = congestion, dropped = channel loss - No need to distinguish via RTT trends (direct signal)

QUIC validates ECN support at connection startup. If supported, use it. If not (common on wireless), fall back to Copa's delay-based detection.

12.8 Application Back-Pressure

When r* is so high that source_rate = total_rate/(1+r*) drops below the application's minimum (e.g., VoIP codec needs 64kbps), the system signals back-pressure: "the channel cannot support your required quality at this loss rate."

The application must respond: - Reduce quality (lower bitrate codec, lower video resolution) - Accept lower reliability (increase d, decrease p) - Wait for better conditions

This is a resource allocation problem, not a CC problem. The CC works correctly; it's the application that needs to adapt.

Back-pressure mechanism: The sender stops reading from the source (TUN interface, application socket) when the retransmit buffer reaches buffer_max. This causes the kernel's write buffer to fill, which causes the application's write() to block — identical to TCP behavior when the send buffer is full.

No special API is needed for basic operation: applications using the TUN interface experience standard blocking write() semantics. Data is accepted when the channel can handle it and blocked when it can't.

Optional stats API (for advanced applications): Applications that want finer control can subscribe to channel statistics: - Per-path: ε, RTT, throughput, correction rate r, Copa rate - Aggregate: effective reliability ρ, tail latency δ, correction deficit - Events: backpressure start/end, path added/removed, regime change

The API also allows overriding protocol characteristics mid-connection: - Adjust δ (tail latency target) or ρ (reliability target) - Change T_cut (affects eviction and gap pruning) - Force path preferences

This API is implementation-specific and not part of the core model.

---

13. Multi-Path Scheduling

13.1 Why FEC Beats MPTCP

MPTCP (Multi-Path TCP) schedulers are fundamentally limited by **head-of-line (HOL) blocking**: TCP requires in-order delivery, so a packet on the slow path blocks all fast-path packets at the receiver. MPTCP schedulers (round-robin, weighted, BLEST [Ferlin2016]) try to minimize this by avoiding slow paths, but they can't eliminate it.

Our FEC-based model is fundamentally different:

```
MPTCP:                          Raptorpath:

Path A: [P1] [P2] [P5] [P6]     Path A: [S1] [C] [S3] [C]
Path B: [P3] [P4]               Path B: [S2] [S4] [C]

Receiver must wait for P3       Decoder needs ANY k of n symbols.
before delivering P4,P5,P6.     Order doesn't matter.
HOL blocking on slow path.      No HOL blocking.
```

The decoder doesn't care WHICH symbols arrive — just HOW MANY. A repair symbol on Path A can recover a lost source on Path B. Source symbols can arrive in any order. The reorder buffer handles sequencing after decode.

13.2 Per-Path Model

Each path i runs independently with its own:

```
Copa_i:     rate_i = 1 / (d x dq_i)         total sending rate
GE_i:       (e_i, p_i, q_i)                 loss model
Taper_i:    tau_i(t) = A_i x (1-q_i)^t      correction density
r_i:        correction rate = A_i / q_i     source/correction ratio
```

All paths share: - **One source stream**: source symbols are distributed across paths - **One retransmit buffer**: any path can retransmit any un-ACKed symbol - **One FEC encoder window**: repair symbols cover the same source window

13.3 Unified Symbol Stream and Interleaving

Each path carries a **unified stream** of source + correction symbols, interleaved at that path's own taper ratio. The interleaving is essential for burst protection — corrections scattered among source symbols survive bursts that wipe out consecutive symbols:

```
Path A (e=0.05, r=0.05):  [S][S][S][S][C][S][S][S][S][C]...   5% corrections
Path B (e=0.10, r=0.11):  [S][S][C][S][S][C][S][S][C]...     11% corrections

During burst on Path A:   [S][X][X][X][C][S]...
                               lost      ^ this correction survives!
                                           -> decoder can recover
```

Source and corrections must NOT be separated between paths. If source goes on one path and corrections on another, the source path has no interleaved corrections, and burst protection fails:

```
BAD: source/correction separation
Path A: [S][S][S][S][S][S]...  <- burst wipes everything, no protection
Path B: [C][C][C][C][C][C]...  <- corrections exist but on wrong path
```

The scheduler distributes the combined stream across paths. Each path independently interleaves at its own ratio.

13.4 Correction Deficit

The **correction deficit** is the total expected corrections still needed across all paths:

```
deficit = SUM_{s in un-ACKed} e_s

where e_s = channel loss rate of path(s) at the time symbol s was sent
```

Each source or correction symbol sent on path i adds e_i to the deficit (it might be lost). Each ACKed symbol removes its send-time e_s (confirmed survived).

Why e/(1-e) is a geometric series: When a correction symbol is itself lost, it needs replacement. This creates a chain:

```
Source lost:              0.30   (30% loss)
Corrections also lost:    0.30 x 0.30 = 0.09
Replacements also lost:   0.30^3 = 0.027
...
Total = 0.30 + 0.09 + 0.027 + ... = 0.30 / (1 - 0.30) = 0.4286
```

This is why r_IT = e/(1-e), not just e. The (1-e) denominator accounts for the infinite chain of correction-of-correction loss. The deficit counter captures this naturally: lost corrections add to the deficit, generating more corrections, until enough survive.

Cross-path correction deficit: When source is on path A (e_A) and corrections go on path B (e_B), the formula becomes:

```
r = e_A / (1 - e_B)
```

This emerges from the deficit dynamics: source adds e_A, each surviving correction on path B removes (1-e_B). Equilibrium: e_A = r x (1-e_B).

13.5 Effective Delivery Time and Bandwidth

For each path i:

```
E_i     = RTT_i/2 + e_i x t_recovery_i       effective delivery time  [sec]
B_eff_i = C_i / (1 + r_i)                    source-carrying capacity [sym/s]
e_combined = SUM(C_i x e_i) / SUM(C_i)       throughput-weighted loss [prob]
```

where t_recovery_i = P_fec_i x t_fec_i + (1 - P_fec_i) x L_arq_i is the expected recovery time on path i if the symbol is lost. t_fec_i is the FEC recovery time (Section 3.4) and L_arq_i is the ARQ recovery latency on that path. The delay spectrum concept [Facenda2022] provides related per-link rate allocation theory for multi-path streaming.

13.6 Scheduler Ratio Adjustment

The scheduler can adjust the per-path source/correction ratio to favor certain paths for source symbols. This is only beneficial for **latency-sensitive** traffic (source symbols are immediately processable, corrections are not — so putting more source on a fast path saves latency):

```
Default (natural taper):
  Path A (fast, e=0.05):  [S][S][S][S][C][S][S][S][S][C]...   r=0.05

Latency-optimized (scheduler shifts source to fast path):
  Path A (fast):          [S][S][S][S][S][S][S][S][S][C]...   r'=0.02
  Path B (slow):          [S][C][C][S][C][C][S][C][C]...      r'=0.30
                                                              (absorbs deficit)
```

For **bandwidth-optimized** traffic: no adjustment needed. Source and corrections cost the same bandwidth. The natural taper ratio is optimal.

Constraint: Each path must maintain minimum interleaving for burst protection. The scheduler cannot reduce the correction ratio below a floor that leaves the path unprotected during bursts.

13.7 Burst Protection During Ratio Adjustment

When the scheduler reduces a path's correction ratio (more source, fewer corrections), that path has fewer same-path corrections to survive a burst. But burst protection is already handled by two existing mechanisms:

Global correction deficit (Section 13.4): When one path's corrections are reduced, the deficit grows — and other paths absorb it by generating more corrections. The total correction budget across all paths remains matched to the total loss. No per-path floor is needed because the correction budget is global, not per-path.

Cross-path diversity (Section 13.10): Corrections on the slow path survive bursts on the fast path (different channel, different burst pattern). A burst on path A is covered by corrections from path B. P(both paths burst simultaneously) = ε_A × ε_B — negligible for independent paths.

No correction floor (r_min) is needed. The global deficit and cross-path diversity provide burst protection without any per-path minimum. The taper self-corrects via BOCD if loss increases, and P_lost(t) naturally produces Mehrotra's optimal policy (repair during uncertainty, retransmit after confirmation).

13.8 Interpolated Objective Function

One parameterized objective with weights from the protocol hint:

```
minimize: w_lat x SUM(x_i x E_i) + w_bw x SUM(x_i x r_i)
           ^                          ^
      latency cost               bandwidth overhead cost

subject to: SUM(x_i) = 1                  all source distributed
            x_i x source_rate <= B_eff_i  per-path capacity

Realtime:   w_lat = 1.0,  w_bw = 0.0   minimize latency at any bandwidth cost
Balanced:   w_lat = 0.5,  w_bw = 0.5   balance latency and bandwidth
Bulk:       w_lat = 0.0,  w_bw = 1.0   minimize bandwidth waste (overhead)
```

In-order delivery coupling (L2 refinement — measured, asymmetric paths). The objective above assumes per-symbol delivery costs are independent: latency composes linearly in the allocation x_i. Two delivery mechanisms break that assumption:

1. **Block decode coupling.** A block decodes only when enough of its symbols have arrived across ALL the paths it was striped over. Its completion time is not a weighted sum but a max:

   ```
   T_blk(b) = max_{i : b_i > 0} ( b_i/C_i + E_i )

   b_i = the block's symbols on path i (serialization + delivery)
   ```

   Any nonzero share on the slowest path makes the WHOLE block pay that path's delivery time — and losses in that share recover at that path's RTT (its FEC/ARQ round), not the fast path's.

2. **Cross-block in-order delivery.** Decoded blocks are released in block-id order (the delivery contract that shields an inner TCP from tunnel-induced reordering; hold H = 4 x SRTT_max, clamped). A slow block at the head of the sequence delays every faster block behind it, and a block slower than H is force-delivered as a HOLE in the inner byte

stream — the latency skew converts into inner retransmissions and cwnd collapse, i.e. into THROUGHPUT loss. This is how Bulk (w_bw = 1), which nominally ignores latency, still pays for latency skew: under in-order delivery there is no allocation whose latency cost is free.

Measured at L1 C8 (path A 100 Mbit / 10 ms RTT / eps 2.5%, path B 20 Mbit / 40 ms RTT / eps 4.8%, bulk 50 MB, per-symbol striping): 27% of source landed on B, 15% of blocks striped across both paths; striped blocks completed at mean 189 ms vs 17.5 ms for A-only blocks (p50 131 vs 13 ms); 92% of in-order head-of-line waits were caused by blocks touching B; 151 holds per 100 MB expired at the 300 ms cap and were force-delivered as holes. Aggregate 8.8 Mbit/s — BELOW the fast path alone (14.0). The linear objective cannot see any of this.

Refinement: allocation granularity must match the delivery unit. Realize x_i per BLOCK, not per symbol: block k rides one path, and y_i is the long-run fraction of blocks assigned to path i. Per block of K symbols the delivery time on path i is D_i = K/C_i + E_i, and:

```
striping:        EVERY block pays   max_i (b_i/C_i + E_i)
block-granular:  y_i of blocks pay  D_i        (their own path only)
```

The block delivery time needs the per-BLOCK recovery term, not the per-symbol one: a block of K symbols on a path with loss eps needs a recovery round at THAT path's RTT with probability

```
P_blk = 1 - (1-eps)^K        (~1 for realistic K: K=56, eps=4.8% -> 0.94)

D_i = K/C_i + RTT_i/2 + P_blk x 2 x RTT_i
```

The per-symbol E_i (Section 13.5) undercounts this by roughly an order of magnitude (measured C8: E_B = 22 ms vs B-blocks actually completing at p50 94 ms). The interpolated objective keeps its form, evaluated per delivery unit:

```
minimize: w_lat x SUM(y_i x D_i) + w_bw x SUM(y_i x r_i)

subject to: SUM(y_i) = 1
            y_i x block_rate <= B_eff_i / K            per-path capacity
            D_i - min_j D_j <= H/4  for all y_i > 0    in-order hold horizon
```

The third constraint is the in-order coupling term: a path whose block delivery time exceeds the fastest path's by more than the reorder hold budget cannot carry source at all — its blocks would be force-delivered as holes — but it remains fully useful for corrections and cross-path retransmit (Section 13.10), which have no ordering deadline (this also keeps its estimators warm for re-admission when it improves). The H/4 factor maps the MEDIAN estimate D_i onto the TAIL event the constraint guards against: an expiry fires when a single block exceeds H, ARQ rounds stack the delivery tail to ~3-4x the median (measured C8: median 134 ms with expiries at 301+ ms), so a median skew above H/4 already pushes the tail past the horizon. For Bulk the solution is y_i proportional to B_eff_i over the feasible paths (fill capacities at block granularity); for Realtime it degenerates to the lowest-D_i path with capacity spill. Note this does NOT violate the Section 13.3 rule (source and corrections must not be separated per path): correction placement is unchanged, and burst protection on a source-carrying path is provided by cross-path diversity (Sections 13.7, 13.10).

The production scheduler realizes y_i with a smooth weighted round-robin on B_eff_i (Copa pacing rate deflated by 1 + r_i): the long-run shares converge to the capacity split while

consecutive blocks alternate paths as evenly as the weights allow, minimizing the skew the reorder buffer must absorb.

Measured after the refinement (same C8, one arm at a time): block-granular affinity alone lifted 8.8 -> 11.4 Mbit/s (striped blocks 15% -> 0, but B-blocks still delivered at p50 94-134 ms — the ARQ round at B's own RTT); adding the D_i eligibility constraint starved B of source (6% residual, estimator warm-up only) and reached 12.6 Mbit/s on 50 MB bulk and cut 1.8 MB object median completion from 3.07 s to 1.15 s (2.7x). Hold expiries fell 151 -> 96 per 100 MB. Symmetric C7 is unaffected (23.3 vs 23.9). At these parameters the model's optimum is fast-path source + slow-path corrections; the ~10% residual gap to the fast path alone (14.0) is warm-up admission plus tail expiries. Kernel MPTCP aggregates beyond its own single path here (12.6 vs 10.6) because its receiver absorbs cross-subflow reordering inside one sequence space — an option the tunnel's inner-TCP in-order delivery contract forecloses.

13.9 QoS Priority Cascade

When multiple protocol classes share the same paths, they pick in priority order from tightest to loosest latency requirement:

```
1. Realtime picks first:  lowest E_i paths, up to its source volume
2. Balanced picks next:   best remaining path capacity
3. Bulk gets the rest:    whatever capacity remains
```

13.10 Cross-Path Retransmit

The shared retransmit buffer enables recovery across paths. When source symbol S_k was sent on path A and lost, any path's correction slot can retransmit it:

```
P(retransmit on path j) = P_lost(t_k, e_A)
```

Cross-path diversity: P(both fail) = e_A x e_j. For e_A=0.10, e_j=0.02: P(both fail) = 0.002 — 50x improvement over single-path.

No explicit path routing needed. Cross-path retransmit emerges from the shared buffer. Each path independently generates correction symbols from its taper. When a correction slot produces a retransmit (via P_lost), it pulls from the shared buffer — which may contain symbols from any path. P_lost uses per-path ε, so symbols from lossier paths have higher P_lost and are retransmitted first.

Work-stealing analogy. Paths with spare Copa capacity (typically low-loss paths whose tapers need fewer corrections) have room to pull retransmits from the shared pool — like idle threads stealing work. High-loss paths are busy with their own corrections. This naturally routes retransmits through reliable paths without explicit selection.

Potential refinement: For latency-sensitive traffic, weighting retransmit pulls toward the lowest-RTT path would reduce recovery time. This follows the same interpolated objective as source scheduling (Section 13.8).

---

14. Future Directions and Considered Improvements

The following sections document insights discovered through building and testing the interactive visualizer simulation. They represent deeper considerations about how FEC latency,

ARQ latency, bandwidth, and the encoder window interact — areas where the current model can be refined in future work.

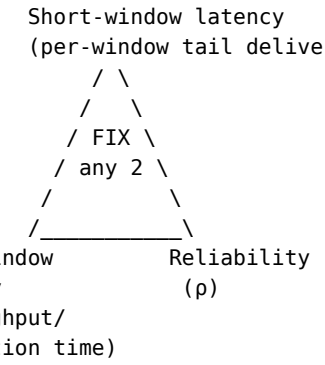
14.1 Reconsidering the Triangle: Bandwidth as Constraint, Not Variable

The current triangle (Section 1.4) treats r (bandwidth overhead) as a variable alongside δ (tail latency) and ρ (reliability). But Copa (Section 12) discovers a fixed link capacity C. We cannot create more bandwidth — we can only choose how to fill the pipe. Every FEC symbol displaces a source symbol:

```
source_rate = C / (1 + r)

More FEC (higher r) → faster per-window recovery → but slower completion
```

This means "bandwidth overhead" is not an independent dimension — it IS long-window latency at a fixed link rate. The real trade-off space may be:



r is the internal mechanism that trades short-window for long-window latency. The user sets latency targets; the system computes r. This reframing may lead to more intuitive protocol hint mappings.

14.2 FEC Latency Is Not Zero

Section 3.2 states "Latency cost: zero additional (repair arrives at roughly the same time as source)." This is an approximation. FEC repair symbols arrive AFTER the source symbols they cover. A lost symbol S3 is only recovered when enough repairs covering S3 have arrived AND the decoder resolves S3′s equation. This takes time t_fec > 0.

Per-symbol: FEC can never make a single symbol arrive faster than if it wasn't lost. The repair always arrives after the source it covers.

Per-window: FEC can recover all losses within a window once enough repairs accumulate. The decode latency depends on the encoder window size W and the correction rate r.

The distinction matters for real-time applications where per-symbol latency is the constraint. For file transfers, only per-window/total completion time matters, and the approximation is acceptable.

14.3 Unified FEC Latency Distribution

For m concurrent losses (a burst of length m), the time until all m are recovered follows a counting process. Every repair covers the entire encoder window (Section 3.2), so the useful-equation arrival rate after a loss is the AGGREGATE correction rate — corrections occupy a fraction r/(1+r) of wire slots, each surviving with probability (1-ε) — not the lost symbol's own

taper density. Using the Poisson approximation (valid when per-slot correction probabilities are small):

```
λ(T) = r × (1-ε) × T / (1+r)          T in wire slots after the loss

P(t_fec ≤ T | m) = P(Poisson(λ(T)) ≥ m)
                 = 1 - Σ_{k=0}^{m-1} e^{-λ(T)} × λ(T)^k / k!
```

λ(T) grows linearly until the window slides past the lost symbol — after W further source symbols, i.e. T ≈ W(1+r) wire slots — at which point λ = rW(1-ε), exactly the per-window Binomial mean of Section 8.2.

(An earlier formulation summed only the lost symbol's own taper, λ(T) = A(1-ε)(1-(1-q)^(T+1))/q, which saturates at r(1-ε) < 1 and wrongly predicts that even a single loss is usually unrecoverable — the same per-symbol undercount that invalidates Appendix E.)

This is the regularized incomplete gamma function Q(m, λ(T)).

Unified across codec types:

- Block codec (RaptorQ, RS): m = losses in block of size K. All m symbols decode simultaneously when the m-th repair arrives.
- Window codec (RLC, Streaming): m = losses in window of size W. Decoder can cascade — recovering one symbol may immediately resolve others via Gaussian elimination. The block CDF is a conservative lower bound (window codecs recover faster due to cascade).

14.4 Ambient FEC and the Pipeline Effect

With a sliding window codec, repair symbols are generated continuously as part of the steady-state r/(1+r) interleaving (Section 5.3). When a loss occurs, there are already repair symbols "in the pipeline" that cover the lost position — they were generated while the lost symbol was in the encoder window.

```
Window:  [S1][S2]...[S50][R1][S51][R2]...[S100][R3]...
                        ^          ^
                        S50 lost   R1 already covers [S1..S100]
                                   → decoder has one equation instantly
```

For a symbol that has been in the window for T_w ticks before loss:

```
λ_total(T) = λ_prior(T_w) + λ_new(T)

λ_prior(T_w) = accumulated surviving repairs from before the loss
             = r × (1-ε) × T_w / (1+r)
λ_new(T)     = new corrections generated after the loss (Section 14.3)
             = r × (1-ε) × T / (1+r)
```

Larger windows accumulate more ambient FEC → faster recovery. This means FEC latency is not just about new repairs generated after the loss — it includes the pipeline of repairs already in flight.

14.5 Optimal Encoder Window Size

The window should be large enough that ambient FEC covers typical bursts. For a burst of length B, we need B surviving repairs in the pipeline:

```
W × r × (1-ε) / (1+r) ≥ B

W_min(B) = B × (1+r) / (r × (1-ε))
```

With r = ε/(1-ε): r(1-ε) = ε and 1+r = 1/(1-ε), so W_min(B) = B/(ε(1-ε)). For the mean burst (B = 1/q), dropping the small (1-ε) correction:

```
W_min = 1 / (q × ε × (1-ε)) ≈ 1 / (q × ε)
```

Using the Section 2.4 scenario parameters (W_min ≈ B/ε):

```
Scenario          ε      q     W_min(mean)   B_99   W_min(p99)
-------------------------------------------------------------
WiFi           2.5%   0.50        80          7       ~280
LTE              5%   0.40        50         10       ~200
Satellite        9%   0.30        37         13       ~144
```

Note the two opposing effects: longer bursts (small q) raise W_min via B, but higher loss ALSO means more repairs per window at r = ε/(1-ε), which lowers it — so satellite (high ε) needs a smaller minimum window than WiFi (low ε) despite its longer bursts.

Setting W so that W × t_sym ≈ RTT gives FEC and ARQ roughly equal recovery latency. Below that threshold, FEC is strictly faster than ARQ for all burst lengths up to the pipeline capacity.

14.6 ARQ Latency and the Retransmit Sweet Spot

ARQ latency is well-defined: L_arq = T_retx + RTT/2 ≈ 1.5 × RTT. But T_retx depends on confidence that the symbol is actually lost. P_lost(t) (Section 3.4) models this confidence.

As T_retx shrinks (retransmit sooner):

- ↑ Faster recovery for truly lost symbols
- ↓ More false-positive retransmits (duplicates)
- ↓ Duplicates waste bandwidth ⟶ worse long-window latency

P_lost(t) is exactly this probability/waste trade-off curve. The sweet spot is where P_lost(T_retx) is high enough that few retransmits are wasted, but low enough for timely recovery.

For T_retx < RTT: **proactive retransmit** territory. We retransmit before knowing if the symbol was lost. This is bandwidth-expensive but latency-optimal for the individual symbol. Viable only at high ε where most symbols are lost anyway (P_lost(0) = ε is already high).

14.7 When FEC Beats ARQ (and Vice Versa)

FEC wins when: t_fec(W) < L_arq ≈ 1.5 × RTT

```
- High RTT (satellite): ARQ is slow, FEC's window decode is faster
- Short bursts: m small, few repairs needed, quick decode
- High bandwidth: t_sym small, window fills quickly
```

ARQ wins when: L_arq < t_fec(W)

```
- Low RTT (datacenter): ARQ round-trip is fast
- Very long bursts: m large, FEC needs many repairs
- Low bandwidth: large t_sym, window takes long to fill
```

The crossover point t_fec(W) = 1.5 × RTT determines the optimal FEC/ARQ balance for a given link. The window size optimization (Section 14.5) can be tuned to align this crossover with the link's RTT, making FEC optimal for all bursts shorter than the pipeline.

14.8 Per-Symbol Recovery Probability Function

Given GE parameters (p, q), the taper, and the encoder window size, we can compute a per-symbol recovery probability as a function of time:

```
P_recovery(T | m, T_w) = P(Poisson(λ_total(T)) ≥ m)

where:
  λ_total(T) = λ_prior(T_w) + λ_new(T)
  m          = burst length (geometric(q) from GE model)
  T_w        = time symbol has been in window before loss
```

This is computable analytically via the Poisson CDF (regularized incomplete gamma function). No Monte Carlo simulation needed, though a Markov chain on GE states would give the joint distribution of (burst_length, recovery_time) for more precise analysis.

Since we measure p and q directly from the GE estimator (Section 7.5), these parameters are available at runtime. This opens the possibility of computing the recovery CDF for each symbol to make optimal FEC/ARQ decisions — a refinement of the current P_lost heuristic.

14.9 Reconceived Delivery Time Distribution

The corrected per-symbol delivery time CDF:

```
P(delivered by T) =
    (1-ε)                                      not lost (arrives at RTT/2)
  + ε × P(t_fec ≤ T | m)                      lost, FEC recovers by T
  + ε × (1-P(t_fec ≤ T | m)) × I(T ≥ L_arq) lost, ARQ recovers by T
```

The tail: δ(T) = 1 - P(delivered by T) = P(delivery takes > T)

This allows the triangle to use a **time budget** T_budget instead of the current binary FEC/ARQ classification:

```
Fix T_budget + ρ → compute r   (minimum r for the latency target)
Fix r + ρ       → compute T    (what latency does this r achieve?)
Fix r + T_budget → compute ρ   (reliability at this latency budget)
```

The time-based formulation connects naturally to application requirements: "99% of packets within 33ms" maps directly to δ(33ms) ≤ 0.01, ρ ≥ 0.999.

14.10 Latency Tail vs Throughput: Not Always a Trade-off

With perfectly matched FEC (r = ε/(1-ε), zero waste):

- Short-window latency: FEC recovers bursts within the window ✓
- Long-window latency: minimal overhead (~ε/(1-ε) extra symbols) ✓
- Both improve simultaneously vs the no-FEC baseline

The trade-off only appears when:

1. **Over-provisioning FEC:** wastes bandwidth → worse completion time
2. **Over-provisioning ARQ:** duplicates waste bandwidth
3. **Under-provisioning either:** tail latency degrades

The taper function's role (Section 4) is to match FEC exactly to the loss distribution, avoiding both over- and under-provisioning. When well-matched, there is no latency-vs-throughput trade-off — only the inherent cost of channel loss (ε/(1-ε) overhead is the theoretical minimum regardless of mechanism).

14.11 Application Profiles Revisited

With the refined latency model, the application profiles from Section 1.4 gain additional precision:

VoIP/Gaming (T_budget ≈ 20ms, ρ = 98%):

```
FEC window should contain < 20ms of data.
At high ε: accept 2% loss rather than ARQ delay.
Proactive retransmit viable if ε > 30% (P_lost(0) = ε is already high).
W_optimal: small (minimize t_fec), accept higher per-window loss.
```

Large file transfer (T_budget = relaxed, ρ = 100%):

```
Only throughput (long-window latency) matters.
Minimize r → maximize source_rate = C/(1+r).
ARQ handles the tail — per-symbol latency irrelevant.
r = ε/(1-ε) is optimal (information-theoretic minimum).
W_optimal: large (maximize pipeline, minimize FEC overhead variance).
```

Live video (T_budget ≈ 33ms per frame, ρ = 99.9%):

```
Per-frame deadline. FEC window ≈ one frame of data.
Moderate FEC for fast burst recovery within frame.
ARQ backstop for rare multi-frame bursts.
W_optimal: ≈ frame_size / symbol_size (natural alignment).
```

Sensor/IoT (T_budget = relaxed, ρ = 95%):

```
Bandwidth-constrained (low-power link).
Minimal FEC, minimal ARQ. Accept 5% loss.
r kept minimal to maximize source throughput.
W_optimal: small (save memory on constrained device).
```

14.12 ARQ After FEC Decode: Not Redundant for the Decoder

When the decoder recovers a lost symbol S3 via FEC, the application can consume S3 immediately. But if an ARQ retransmit of S3 was already in flight, the retransmit still arrives at the receiver.

This ARQ is NOT redundant at the receiver. The decoder should still feed it as an equation — it is a source symbol (unit vector, guaranteed linearly independent) that can help decode OTHER lost symbols still in the decoder window. From the receiver's perspective, every arriving symbol is useful to the decoder until the window slides past it.

The ARQ IS redundant at the sender — once the sender knows (via ACK) that the receiver has decoded S3, further retransmits of S3 waste bandwidth. The sender should stop retransmitting ACKed symbols. This is already handled by the existing ACK mechanism: once the sender receives an ACK covering S3, it removes S3 from the retransmit buffer.

The interesting case: the sender sends an ARQ for S3 BEFORE learning that FEC decoded it. This is unavoidable due to ACK delay (~RTT). The "wasted" bandwidth is bounded by one

RTT worth of unnecessary ARQ. But the receiver still benefits from the equation — so the waste is only sender-side bandwidth, not receiver-side utility.

14.13 Proactive Retransmit vs FEC: FEC is Strictly Better

At the same overhead, FEC is strictly superior to proactive retransmit (sending every source symbol twice):

- One FEC repair covers ANY single loss in the window (W positions)
- One retransmit covers exactly ONE specific position

For a window of W=50, a single FEC repair is 50x more flexible than a single retransmit. Proactive retransmit only approaches FEC efficiency when $\varepsilon \to 50\%$ (half of all symbols are lost, so random "insurance" is as good as targeted repair). At typical loss rates (1-20%), FEC dominates by a factor of W/1.

Many real-world systems use packet duplication for simplicity. This model shows the quantitative cost of that simplicity.

14.14 Marginalizing Over Burst Length

The FEC latency CDF (Section 14.3) conditions on m (burst length). The unconditional CDF marginalizes over the GE burst length distribution:

```
P(t_fec ≤ T) = Σ_{m=1}^{∞} P(burst = m) × P(t_fec ≤ T | m)
             = Σ_{m=1}^{∞} (1-q)^{m-1} × q × Q(m, λ(T))
```

where Q(m, λ) is the regularized incomplete gamma function.

In practice, truncate at m_max = B_99 (99th percentile burst length). The tail P(burst > B_99) < 1% contributes negligibly to the CDF.

14.15 In-Burst FEC Survival

The Poisson model uses (1-ε) survival probability for all repairs. But during a burst (channel in Bad state), ALL symbols are lost — including FEC repairs. Only POST-burst repairs survive.

The corrected λ(T) should split into two phases:

```
For a burst starting at t=0 with length B:

  λ(T) = 0                                          for T < B (in-burst)
       = Σ_{t=B}^{T} τ(t) × (1-ε_good)             for T ≥ B (post-burst)

where ε_good ≈ p (probability of re-entering Bad at a post-burst slot;
the Good state itself is loss-free, h_G = 0) ≈ 0 for packet erasure
```

The practical effect: recovery begins only after the burst ends. The longer the burst, the later recovery starts. This makes burst length m doubly important — it determines both HOW MANY repairs are needed AND HOW LONG before repairs start arriving.

For the ambient pipeline (Section 14.4): repairs generated BEFORE the burst that are still in transit may arrive during or after the burst. These contribute if they survive the channel at the receiver — which they do, because they were transmitted during the Good state before the burst. The pipeline effect thus provides a head start on recovery.

14.16 The FEC/ARQ Race

FEC and ARQ work in parallel for a lost symbol. The actual delivery time is min(t_fec, L_arq) — whichever recovers the symbol first.

```
P(delivered by T) = 1 - P(t_fec > T) × P(L_arq > T)
```

P_lost(t) (Section 3.4) currently decides the MIX in correction slots — FEC or ARQ per slot. But both mechanisms are running simultaneously: FEC repairs accumulate regardless of whether ARQ fires, and ARQ retransmits arrive regardless of FEC state.

The min() combination means the actual tail latency is BETTER than either mechanism alone. The two mechanisms are complementary, not alternatives. The P_lost mix controls the bandwidth split, but both contribute to recovery probability.

14.17 Decode-Induced Jitter

FEC cascade decoding produces micro-bursts: when the m-th repair arrives and the decoder resolves a pivot row, cascade recovery may immediately decode several other symbols. Multiple symbols become available to the application simultaneously.

This creates "negative jitter" — symbols arriving too close together. While not harmful (data is available earlier than expected), it means the delivery time distribution has a step-function component at decode events, not a smooth CDF.

For jitter-sensitive applications (VoIP): a de-jitter buffer after the decoder smooths the micro-bursts into a steady stream. The buffer adds a constant baseline latency (typically one frame period). The net effect on jitter depends on whether the tail improvement from FEC exceeds the baseline latency added by the de-jitter buffer.

Tail latency improvement always reduces jitter in the upward direction (fewer late symbols). Decode micro-bursts create jitter in the downward direction (symbols arriving early). The de-jitter buffer absorbs the downward jitter at the cost of baseline latency. As long as:

```
de_jitter_buffer_delay < tail_latency_improvement
```

the net result is lower jitter. For most FEC-protected links, this condition holds because FEC reduces tail latency by an RTT (replacing ARQ with FEC recovery), while the de-jitter buffer adds only a few ms.

14.18 Estimator-Rate Feedback Stability

In "compute r" mode, the estimator's ε feeds into the triangle solver which produces r. But r affects correction volume, which affects how many symbols the ESTIMATOR observes as lost-and-recovered vs lost- permanently.

Potential oscillation:

```
high r → good recovery → estimator sees low ε → solver reduces r
→ poor recovery → estimator sees high ε → solver increases r → ...
```

This feedback loop is stabilized by two properties:

1. **BOCD's inertia**: the Bayesian changepoint detector maintains a run-length distribution with prior mass. It doesn't react instantly to a few ticks of different ε — it integrates over many observations before shifting the predictive quantile.

2. **The estimator observes CHANNEL loss, not APPLICATION loss**: the estimator tracks which symbols were lost on the channel (not received at the receiver), not which symbols the application eventually got (via FEC recovery). FEC recovery doesn't reduce the estimator's ε.

Property 2 is critical. If the estimator instead measured "application- level loss" (post-FEC), the feedback loop would be unstable. The estimator must measure RAW channel loss, independent of FEC recovery.

14.19 Consistency of P_fec Models

The paper has two FEC recovery models:

- **Section 8.2**: P_fec = $\Phi(\sqrt{W} \times (r(1-\varepsilon)-\varepsilon) / \sqrt{(\varepsilon(1-\varepsilon)(r+\sigma^2\_burst)))}$ Normal approximation to binomial. Answers: "given r and W, what's the probability that FEC recovers ALL losses in the window?"
- **Section 14.3**: P(t_fec ≤ T | m) = Q(m, λ(T)) Poisson CDF of the correction arrival process. Answers: "given m losses, what's the probability that m surviving repairs arrive by time T?"

These answer different questions:

- Section 8.2: probability of FEC success (regardless of time)
- Section 14.3: time distribution of FEC recovery

They should be consistent: by the time the window slides past the lost symbol, the Poisson CDF should approach the Section 8.2 P_fec.

Verify: the lost symbol stays in the window for W further source symbols = W(1+r) wire slots, so λ(window exit) = r(1-ε)/(1+r) × W(1+r) = rW(1-ε) — exactly the mean of the Binomial repair count in Section 8.2. The Poisson(λ) tail P(≥ m) matches the first moment with a slightly wider distribution than the Binomial. The Monte Carlo suite compares the two models numerically across scenarios (tolerance 0.15).

14.20 The δ Definition Question

The paper currently has two candidate definitions for δ:

- **Section 6.3**: δ = P(late delivery) / ρ — a probability (dimensionless)
- **Section 14.9**: δ(T) = P(delivery > T) — a function of time

These serve different purposes:

- The probability definition is useful for the triangle solver (binary search on r to achieve target δ)
- The time-based definition connects directly to application requirements ("99% of packets within 33ms")

Recommendation: keep both. The probability δ is the triangle variable. T_budget is the application requirement. They're connected by:

```
δ = δ(T_budget) = P(delivery > T_budget)
```

The user specifies T_budget (from application requirements). The system computes δ = δ(T_budget) using the delivery time CDF, then uses δ in the triangle solver. This preserves the triangle's mathematical structure while connecting to real-world requirements.

14.21 Sub-Capacity Operation (Emergent Behavior)

When the application data rate is below link capacity, the model should handle this naturally without a separate code path.

In the current model, Copa (Section 12) controls the total sending rate C. The taper determines the source/correction split: source_rate = C/(1+r). If the application produces data at rate S < C/(1+r), the sender has idle capacity.

In the current architecture, Copa already handles this: if the application has no data to send, Copa's window isn't fully utilized, and the sending rate naturally drops to match. FEC symbols are only generated when source symbols are sent (the taper is driven by source symbols entering the window).

An interesting possibility: when idle slots exist, the sender COULD generate additional FEC repairs proactively — filling spare capacity with extra protection. The taper's r determines the minimum FEC rate; spare capacity allows exceeding it at no throughput cost.

This should emerge from the triangle solver: when solving for r with a tight δ target, the solver may compute r > r_min. If the link has capacity for this higher r without reducing source throughput (because source_rate < C/(1+r)), the system naturally operates at the higher r.

The diminishing return: when the FEC pipeline $\lambda \gg B_{99}$ (more ambient repairs than any likely burst needs), additional FEC provides negligible improvement. The marginal benefit approaches zero exponentially (Poisson tail). The triangle solver would compute r at this saturation point when δ is set very tight — the solver naturally stops increasing r when further increase doesn't improve δ.

Open question: should the taper continue generating repairs during idle periods (no source data)? These repairs cover the existing window and strengthen the pipeline. The current model ties correction slots to source slots — decoupling them would allow "idle FEC" without changing the architecture, only the scheduling policy.

A quantitative saturation model. The L0 gate measured the diminishing return turning NEGATIVE: at C4-Satellite, Realtime (δ = 1e-7 ⟶ r ≈ 0.49) has a WORSE p99 than Auto (412 ms vs 297 ms). Past ~43% overhead the extra repairs displace source symbols and pressure the queue — more FEC hurts the tail. The controller increases r monotonically with hint tightness and needs a saturation point. The model below is continuous (no mode switch) and uses only estimator-known quantities: ε (loss), σ^2_burst (Section 8.3), W (window), SRTT, and t_sym = symbol_size / throughput (the wire slot time).

The p99 tail has components that pull in opposite directions in r:

```
tail_fec(r) = (1 - P_fec(r)) × L_arq                        [decreasing]
    FEC-miss cost: misses fall through to ARQ. P_fec from the
    Section 8.2 normal formula; L_arq ≈ 1.5 × SRTT (Section 14.9).

tail_rec(r) = B × t_sym × (1+r) / (r × (1-ε)),  B = (σ²+1)/2 [decreasing]
    Recovery wait: repairs occupy an r/(1+r) share of wire slots and
    survive with probability (1-ε), so the wait for the B surviving
    repairs a mean burst needs is B × t_sym × (1+r)/(r(1-ε)). This is
    why moderate r keeps helping even after P_fec ≈ 1: repairs arrive
    SOONER. B is the mean burst length recovered from the variance
    factor (σ² = 2B - 1 when p ≪ q, Section 8.3).

tail_svc(r) = c × (1+r) × W × t_sym,  c = 1/2                [increasing]
    Dilution cost: corrections consume wire share r/(1+r), stretching
    the effective per-source service time — the recovery window
    passes at the diluted source rate, taking (1+r) × W × t_sym.

p99_model(r) = tail_fec(r) + tail_rec(r) + tail_svc(r)
```

The sum has an interior minimum r_sat, and the controller should emit `min(r_hint, r_sat)`: the hint still picks the operating point on the rising side of reliability, but cannot push past the point where more FEC hurts. The cap is hint-independent — saturation is a channel property, not a preference.

Worked example (C4-Satellite). ε ≈ 9%, σ² ≈ 5 (B = 3), W = 64, SRTT ≈ 0.21 s (L_arq = 315 ms), throughput 2.5 MB/s, symbol 1225 B ⟶ t_sym = 4.9e-4 s, W × t_sym = 31.4 ms:

```
r      tail_fec   tail_rec   tail_svc   total
0.20    40.9 ms     9.7 ms    18.8 ms    69.4 ms
0.30     4.1 ms     7.0 ms    20.4 ms    31.5 ms
0.35     0.9 ms     6.2 ms    21.2 ms    28.3 ms
0.40     0.2 ms     5.7 ms    22.0 ms    27.8 ms   <- r_sat
0.49     0.0 ms     4.9 ms    23.4 ms    28.3 ms
```

r_sat = 0.40, below Realtime's uncapped 0.49 — consistent with the measured reversal. At C2-WiFi-like numbers (ε = 2.5%, σ² = 3, W = 64, SRTT = 13 ms, t_sym = 1e-4 s) the minimum sits at r_sat ≈ 0.255, ABOVE Realtime's ~0.20 request there: the cap is non-binding exactly where the measurements show more FEC still helping (C2 Realtime p99 28.1 ms beats Bulk's 31.5 ms).

Honesty about roughness. The dilution constant c = 1/2 is a fitted scale, not derived; queueing is ignored beyond linear dilution, so the model's minimum is much shallower than the measured one (the model says +0.5 ms from r_sat to 0.49 at C4; the gate measured +115 ms — the real cost past saturation is a queueing knee, not a line). The model is therefore trusted for the LOCATION of the minimum, not the depth. B from σ² assumes p ≪ q. Estimation error in ε moves r_sat by a few percent (higher ε ⟶ later saturation), which is the safe direction: when in doubt the cap loosens. When the estimator has no throughput sample, t_sym is unknown and no cap applies (degenerate inputs ⟶ no cap): the monotone hint behavior of the base model is preserved.

14.21.1 Soft Saturation (Continuous Approach to r_sat)

Section 14.21 emits `min(r_hint, r_sat)`. That hard minimum has a KINK at r_hint = r_sat: the emitted rate tracks the hint on the low side and then flat-lines the instant the request crosses r_sat. Physically nothing is discontinuous there — the p99 curve has a SMOOTH

interior minimum, so the cost of one more repair grows CONTINUOUSLY as r passes r_sat (queue delay rises smoothly toward the knee; there is no wall). The hard min was an implementation shortcut. This section replaces it with a kink-free cap and, as a by-product, turns the binary "cap binding / not binding" signal into a continuous **saturation pressure**.

What the p99 model implies. Near its interior minimum the tail model is locally quadratic, p99(r) ≈ p99(r_sat) + ½·p99″(r_sat)·(r − r_sat)², so the marginal p99 cost of exceeding r_sat is p99'(r) ≈ p99″(r_sat)·(r − r_sat) — zero at r_sat and rising linearly past it. A controller that "does not want to pay p99" should therefore not hit a wall at r_sat; it should ease off as the marginal penalty accrues. The natural kink-free family is a one-sided **softplus** cap:

```
r_eff = r_sat − s · softplus( (r_sat − r_hint) / s ),   softplus(x)=ln(1+eˣ)

  s = SAT_SOFTNESS · r_sat        (smoothing scale, a rate)
```

with the exact properties (all provable from softplus(x) ≥ max(x,0)):

```
r_hint ≪ r_sat : r_eff → r_hint           (unsaturated: request honored)
r_hint = r_sat : r_eff = r_sat − s·ln2     (smoothly just below)
r_hint ≫ r_sat : r_eff → r_sat             (asymptote — never crossed)
r_eff ≤ min(r_hint, r_sat)   ALWAYS        (the cap never ADDS FEC)
dr_eff/dr_hint = 1 − σ((r_hint−r_sat)/s) ∈ (0,1)   (C^∞, monotone)
```

The last line is the key: the derivative eases from 1 (below saturation, every requested increment is admitted) through ½ (at r_sat) to 0 (deep in saturation, the cap absorbs the whole increment). Its complement is a natural [0,1] indicator,

```
saturation_pressure(r_hint) = σ( (r_hint − r_sat) / s )
                            = 1 − dr_eff/dr_hint
```

— the fraction of a marginal repair-rate increment the cap is currently absorbing: 0 far below r_sat (more FEC still helps the tail), ½ exactly at r_sat (marginal p99 benefit = marginal harm — the Section 14.21 balance point), ⟶ 1 past it. This is the continuous quantity the visualizer now shows in place of the binary CAP BINDING badge.

Choosing the smoothing scale s (honest constant). The width over which the cap bends could be derived from the model's own curvature. Matching the softplus pressure slope dσ/dr = 1/(4s) at r_sat to the balance-point slope of the marginal terms gives s = C'_svc(r_sat) / p99″(r_sat) — the ratio of the (linear) dilution slope to the p99 curvature. But Section 14.21 is explicit that the model's DEPTH — hence its curvature — is NOT trusted: the true cost past r_sat is a queueing knee far steeper than the model's shallow quadratic (+115 ms measured vs +0.5 ms modeled at C4). A curvature-derived s would therefore track the model's OWN shallow curvature and pick too WIDE a band, softening the cap so much that FEC drifts into the measured-disaster regime. The safe, honest choice is a deliberately NARROW fixed fraction, SAT_SOFTNESS = 0.1, so the soft cap stays within 10 % of the gate-validated hard min while removing the discontinuity. As SAT_SOFTNESS ⟶ 0 the hard min `min(r_hint, r_sat)` is recovered exactly; the constant trades a hair of faithfulness to the hard cap for a continuous rate and a continuous pressure signal. (This mirrors the c = ½ dilution constant of 14.21: a fitted scale, declared as such.)

Effect. At the C4 worked example (r_sat = 0.40, Realtime's uncapped request 0.49, s = 0.04) the soft cap emits 0.396 with pressure σ(2.25) = 0.90 — indistinguishable from the hard cap's 0.40 in rate, but now the rate is a smooth function of every input and the operator sees a 0.90

pressure reading ("the cap is holding, hard") rather than a lit boolean. Where the request sits well below r_sat the soft cap is inert to O(e^−1/SAT_SOFTNESS); where it sits well above, r_eff pins to r_sat to machine precision. The change is purely in HOW the same ceiling is approached, so the 14.21 gate results are preserved.

14.22 Sequence-Aware P_lost

The current P_lost(t) (Section 3.4) uses only time since send. But SACK feedback (Section 6.2) provides stronger evidence: if subsequent symbols have been ACKed, an un-ACKed symbol is almost certainly lost.

```
P_lost_seq(k) = P(S_n lost | S_{n+1}..S_{n+k} all received)
```

On a FIFO channel (no reordering), if the next symbol arrived, the previous one was lost — P_lost_seq(1) ≈ 1.0. Real links are not perfectly FIFO: network jitter, multipath, and switch buffering can reorder packets.

Reorder probability estimation: the system should track the observed reorder rate (fraction of symbols that arrive out of order) as a running estimate. Starting assumption: FIFO (reorder_rate = 0). The SACK ranges provide the observation data — each out-of-order arrival updates the estimate.

```
P_lost_seq(k, reorder_rate) = 1 - reorder_rate^k

For FIFO (reorder_rate = 0):     P_lost_seq(1) = 1.0
For mild reorder (rate = 0.05):  P_lost_seq(1) = 0.95
                                 P_lost_seq(3) = 0.9999
```

The combined P_lost uses both time AND sequence evidence:

```
P_lost_combined = max(P_lost_time(t), P_lost_seq(k, reorder_rate))
```

This makes the FEC/ARQ decision faster: instead of waiting for P_lost_time to rise (which takes ~SRTT), a single subsequent ACK gives near-certain evidence on a FIFO link. Correction slots can switch to ARQ sooner for confirmed losses, freeing FEC capacity for uncertain losses.

14.23 Post-Burst FEC Boost (Reactive Deficit Recovery)

After an unexpectedly long burst (longer than the taper was provisioned for), the system has a correction deficit — more losses occurred than the ambient FEC pipeline can cover. The BOCD estimator will adapt ε within 5-15 batches, but during that lag the system is under-protected.

The GE model's state transitions provide a faster signal: the Bad⟶Good transition indicates burst end. At that moment, the deficit is known:

```
On Bad→Good transition:
  burst_length = observed consecutive losses
  repairs_in_pipeline = λ_prior (ambient FEC accumulated)
  deficit = max(0, burst_length - repairs_in_pipeline)
```

If deficit > 0, the system should temporarily boost FEC:

```
boost_duration = deficit / (r × (1-ε))    ticks
boost_r = r + deficit / boost_duration    (spread the extra FEC)
```

This is faster than waiting for BOCD to increase ε ⟶ solver to increase r ⟶ taper to generate more corrections. The GE state transition is observable within one symbol interval; BOCD takes 5-15 batches.

Interaction with ARQ: at burst end, the oldest lost symbols are ~burst_length ticks old. If burst_length > SRTT, P_lost_time is already high for those symbols — ARQ fires naturally. The boost is needed for the symbols where FEC should have covered them but didn't (pipeline was insufficient).

Open question: should the boost be additive (extra FEC on top of normal taper) or multiplicative (temporarily higher r)? Additive is simpler and doesn't affect the taper shape for new symbols. The extra repairs specifically target the deficit, not general protection.

14.24 Jitter-Horizon Encoder Lag

Discovered while building the L0 gate suite (ADR-0051): with per-packet jitter J, a repair symbol can overtake up to J × send_rate of the source symbols it covers. At arrival, those covered-but-still-in-flight symbols are unknowns to the decoder, so the repair parks as a deep pivot equation instead of decoding the actual loss — measured hole-fill p50 at C2-WiFi was ~21 ms instead of the expected ~2 ms.

A repair covering symbols that cannot yet have arrived carries no usable information at arrival time. The encoder should therefore TRAIL the send stream by the jitter horizon:

```
L = ceil(J × send_rate)          [symbols; 2..48 in practice]

repairs cover [sent − L − W, sent − L]   instead of  [sent − W, sent]
```

With the lag, a repair's unknowns at arrival are true losses, decodable immediately. The correction budget is unchanged (Section 4.2 shape invariance) — only the window placement moves. L adapts with measured jitter and send rate; on a jitter-free channel L ⟶ 0 and the windows coincide. This composes with Section 14.5′s window sizing: the effective protection span for fresh symbols shrinks by L, which matters only if L approaches W.

14.25 Completion-Tail FEC

For a finite transfer, completion time decomposes as:

```
T_completion = T_send  +  T_tail_recovery

T_send          = N x t_sym / (1 + r)⁻¹-adjusted source rate
                  (the time to push all N source symbols)
T_tail_recovery = the time to recover losses among the LAST
                  window's symbols after the send stream ends
```

The two terms have completely different loss economics. A mid-transfer loss is recovered in parallel with ongoing sends: ARQ (or a later repair) rides alongside new source symbols, so the recovery consumes wire budget but adds ZERO completion time — the link never goes idle waiting for it. A tail loss is different: once the last source symbol has been sent there is nothing left to overlap with, so every ARQ round on a final-window hole is serial — ~1.5 RTT each (detection at ~9/8 SRTT plus the retransmit flight), and a retransmit that is itself lost pays the full round again.

This yields an end-of-stream policy that is nearly free:

```
At end-of-stream (last source symbol sent), send a burst of
n_tail = ceil(r_tail x W) repair symbols covering the final window.

Cost:    r_tail x W symbols  ≈ negligible vs N for a large transfer
         (e.g. 0.2 x 64 = 13 symbols on a 1500-symbol transfer < 1%)
Saving:  P(≥1 tail loss) x ~1.5 RTT of completion time per avoided
         serial ARQ round, where
         P(≥1 tail loss) = 1 - (1-ε)^W    (≈ 80% at ε=2.5%, W=64)
```

r_tail comes from the exact transfer-matrix computation (Section 8.7): the smallest r such that P_fail(r, W) ≤ δ_tail for a modest tail-failure budget (e.g. δ_tail = 0.05 — one residual serial ARQ round in 20 transfers). The exact DP matters here: the tail burst is a one-shot, small-W event where the normal approximation's 30-50% tail under-provisioning (Section 8.7) directly converts into completion regressions.

Composition with Bulk's r → 0 steady state (Sections 5.3, 12.5). Bulk maps δ to "late is fine" (δ_bulk = $\hat{\varepsilon}$ mid-stream, Section 14.26), so the continuous r* formula (Section 8.4) is 0 mid-transfer: pure ARQ, volume parity with retransmission transports. Completion-tail FEC is the complement, not a contradiction: FEC vanishes in the steady state (where recovery overlaps sending and buys nothing) and reappears exactly at the one place it buys completion time — the stream tail. The r-δ-ρ triangle is respected at both operating points; only the effective δ differs, because the COST MODEL differs between mid-transfer (parallel recovery, late is genuinely fine) and the tail (serial recovery, late is 1.5 RTT each). Section 14.26 makes this composition CONTINUOUS: the one-shot burst described above is the limiting case of a δ glide driven by a completion-exposure kernel χ, and the glide supersedes the burst wherever T_rem is known.

Multipath corollary — tail reinjection. On asymmetric paths the same end-of-stream logic applies to slow-path IN-FLIGHTS, not just losses: once nothing overlaps recovery, an undelivered symbol whose path's residual wait (queue + propagation) exceeds a fast-path flight is worth duplicating onto the fast path (cross-path retransmit, Section 13.10). The duplicate rides spare end-of-stream tokens, so the cost is a few symbols of fast-path capacity against a saving of the slow path's queue drain (~10-25 ms measured on WiFi+LTE). Same principle, ARQ flavor: completion is bought exactly at the stream tail, nowhere else.

14.26 Completion-Exposure δ (the Bulk glide)

Section 14.25 established the two-regime cost model for Bulk: a mid-transfer loss recovers in parallel with ongoing sends (zero completion cost), a tail loss recovers serially (~1.5 SRTT per ARQ round). It implemented the tail side as a one-shot repair burst at end-of-stream. This section replaces the sharp mid/tail boundary with a continuous kernel and, in doing so, fixes two measured flaws in the original Bulk hint mapping δ_bulk = min(0.1, $\hat{\varepsilon}$).

The two flaws of min(0.1, $\hat{\varepsilon}$) (measured in the wasm simulator, which shares `controller_rate` with production):

```
M1 — cold start. The loss estimator's Beta(1,1) prior puts the 95%
predictive upper quantile at ε̂₉₅ ≈ 0.975 for the first ~1.5-3 RTT.
The clamp maps this to δ_eff = 0.1 << ε̂₉₅, so z_{δ/ε} is large, the
IT term ε/(1-ε) ≈ 39 dominates, and r pins at max_overhead = 0.5 —
wasting ~1/3 of the wire for 2-3 RTTs on a channel about which
nothing bad is actually known. Measured: a FIXED r = 0.01 floor beat
Bulk on completion in 20/24 grid cells (median +5%, worst +17%,
gap growing with RTT) and on overhead in 24/24 (excess overhead
2-14% vs ~0-1%).

M2 — moderate loss. At ε ≥ ~0.1 the upper quantile sits above the
0.1 clamp FOREVER, so δ_eff = 0.1 < ε̂₉₅ permanently and r* > 0 in
the steady state: Bulk pays FEC ≈ the IT floor AND ARQ for the same
losses — double payment, directly against the 14.25 cost model in
which mid-stream lateness is free.
```

Both flaws share one root: any δ_eff strictly below $\hat{\varepsilon}$ re-activates the margin machinery, whose entire purpose Bulk rejects mid-stream.

The completion-exposure kernel. What actually distinguishes a loss that costs completion time from one that does not is whether its ARQ round (~1.5 SRTT, Section 3.4) can still hide behind remaining sends. Let T_rem be the remaining send time (in seconds, like Section 5.4′s timing quantities: T_rem = remaining source symbols / send rate). Reusing the Section 3.4/5.4 normal-RTT-tail machinery (Φ = normal survival function):

```
chi(T_rem) = Phi_bar( (T_rem - 1.5 x SRTT) / sigma_arq )

sigma_arq  = max(4 x RTTVAR, SRTT / 4)
```

χ is the probability that a loss suffered NOW is EXPOSED — that its recovery outlives the send stream and becomes serial completion time. The 1.5·SRTT center is the Section 14.25 serial-round cost; σ_arq aggregates detection and flight-time variance (4×RTTVAR, floored at SRTT/4 against degenerate RTTVAR estimates). Mid-stream (T_rem ≫ SRTT) χ = 0; over the final ~1.5 SRTT it rises smoothly to 1. No cutoff anywhere: χ inherits its continuity from Φ.

The δ glide. Bulk's effective tail target is the exposure-weighted blend of "the channel itself" and the 14.25 tail budget:

```
delta_bulk = eps_hat + (delta_tail - eps_hat) x chi,   delta_tail = 0.05
```

- **Mid-stream (χ = 0):** δ_bulk = $\hat{\varepsilon}$ exactly, so $z_{\{\delta/\varepsilon\}} = \Phi^{-1}(0) = -\infty$ and $r^* = 0$ IDENTICALLY — independent of the estimator's uncertainty, because the target tracks the estimate ITSELF rather than a constant the estimate is compared against. This kills M1 ($\hat{\varepsilon}_{95} \approx 0.975 \longrightarrow \delta\_eff = 0.975 \longrightarrow r^* = 0$) and M2 ($\hat{\varepsilon}_{95} = 0.12 \longrightarrow \delta\_eff = 0.12 \longrightarrow r^* = 0$) in one stroke. It is exactly the "late is fine" semantics of the 14.25 cost model: mid-stream recovery is parallel and therefore free, so NO channel state justifies steady-state FEC under Bulk.
- **Stream tail (χ → 1):** δ_bulk → δ_tail = 0.05 regardless of $\hat{\varepsilon}$, and r ramps continuously to the exact rate that meets the completion budget — reaching it about 1.5 SRTT before the last source symbol, precisely when losses start being exposed.
- **T_rem unknown:** a production tunnel is an endless stream — there is no "last symbol" the sender can see, so χ = 0 permanently and the steady state is pure ARQ (existing production tail behavior is unchanged). Feeding χ from an application-known transfer size, or from an idle-onset heuristic (send queue drained = provisional end of stream, retracted if new data arrives), is future work; simulation drivers that know the transfer length feed χ directly.

The tail burst as the glide's limiting case. The 14.25 one-shot burst is what the χ ramp degenerates to as $\sigma\_arq \to 0$: a step from r = 0 to the tail rate at T_rem = 1.5 SRTT. The glide is therefore not a second mechanism but the burst made continuous — and it supersedes the burst wherever T_rem is known (firing both would double-pay the tail budget). Beyond removing the discontinuity, the ramp fixes a coverage gap of the burst at high RTT: with in-flight span (symbols per RTT) $\gg$ W, the last-window burst cannot protect late-stream losses that fall OUTSIDE the final window yet inside the final serial-recovery horizon. The ramp's repairs are emitted across the whole final ~1.5 SRTT of the send stream, each covering the sliding window at its emission time, so the entire exposed span gets coverage. The same mechanism suppresses the pure-ARQ (r = 0 floor) serial-retransmit tail outliers: a tail hole whose retransmit is itself lost pays full serial rounds (one observed 865-tick outlier vs 559 ticks with a 1% floor); ramped repairs give those holes a parallel recovery path.

Scope of the mid-stream guarantee. $\chi = 0$ requires T_rem to exceed the exposure horizon ~$1.5 \cdot$SRTT + $8 \cdot \sigma\_arq$ ($\approx$ 3.5-5.5 SRTT). A transfer SHORTER than that never has a mid-stream phase: $\chi > 0$ from the first symbol, and during the estimator cold start the glide inherits M1′s inflated $\hat{\varepsilon}_{95}$ (measured at 150 ms RTT with a 0.5 s transfer: neither mapping wins — the early rate is governed by the cold-start prior in both). The glide's guarantee is for streams long enough to HAVE a middle; the cold-start prior itself is estimator work, not δ-mapping work. A second CANDIDATE scope limit is the PAYLOAD's dynamics: "parallel recovery is free" prices the outer stream's volume, not delivery latency, so a payload whose own delivery latency feeds back into its throughput (TCP inside the tunnel) could in principle pay for every unrepaired loss. Section 14.28 derives the mid-stream repair floor for those inner-feedback flows — and reports the L1 measurement that, once 14.27′s reactive leg and in-order delivery exist, the inner flow absorbs the residual stalls and the floor buys nothing (C2) or actively hurts (C3). The glide as stated survives that test.

Verification (wasm simulator, shared formula, same seeds per cell; ε = 0.05/0.10, q = 0.5, RTT = 50 ms, W = 64): the glide vs the old mapping cuts completion 599 $\longrightarrow$ 562 ticks and excess overhead 5.99% $\longrightarrow$ 0.04% at ε = 0.05, and 674 $\longrightarrow$ 620 ticks / 8.21% $\longrightarrow$ 0.91% at ε = 0.10; Bulk now beats the fixed r = 0.01 floor on BOTH completion (562 vs 598 ticks) and overhead (5.31% vs 6.11%) at the reference cell — the mapping the 20/24 finding said it must at least match.

14.27 Block-Mode ARQ via Batch Acknowledgements

Section 5 defines corrections as repairs $\cup$ retransmits, both driven by P_lost; Section 14.26 then makes mid-stream $r^* = 0$ for Bulk on the grounds that a mid-stream loss recovers "for free" through the reactive path. The production BLOCK pipeline implemented neither reactive leg: on a decode failure the sender only updated stats and congestion control, and the receiver evicted the incomplete decoder after a timeout. Nothing was ever retransmitted. Under the glide this is not a degraded mode but a contradiction — $r^* = 0$ is only correct BECAUSE the retransmit path exists — and the L1 harness measured the consequence directly: at C2 (100 Mbit, 10 ms RTT, GE 1.3%/50% $\approx$ 2.6% loss) the production tunnel completed a 1.8 MB transfer in ~8 s against quinn's 0.175 s. The inner TCP saw the raw 2.6% loss and collapsed; every block with a hole waited out the 30 s decoder eviction. The window pipeline has its own retransmit buffer (Section 6.1); this section specifies the block-mode equivalent.

The Ack IS the P_lost evidence. The block receiver already acknowledges every Symbol-Batch it receives — each QUIC datagram is one batch, so batch loss is symbol loss. The v4

Ack echoes the batch's `batch_seq`, which keys a sender-side ledger of (batch_seq ⟶ path, [(block, symbol)], send time). This is the SACK limiting case of the Section 3.4 P_lost model: an Ack for batch j on a path, with none for an earlier batch i on the same path, drives P(lost_i) ⟶ 1 without waiting for a timer. Concretely a batch is declared lost when 3 later same-path batches have been acked (the dup-ACK analogue; tolerates datagram reordering) or when max(1.5·SRTT, 50 ms) elapses without an Ack (the timeout leg — needed for transfer tails, where no later traffic exists to accuse the loss; a 25 ms sweep task covers that case). The ledger is keyed by `batch_seq` and NOT by anything already in the v3 Ack: the echoed send timestamp is shared by every chunk of one drain call, and a batch may interleave symbols of several blocks, so the v3 fields identify neither the batch nor the victims. Adding the 8-byte field (protocol v4) was the robust option.

Fresh repairs beat resends. The sender retains source data for the last 64 blocks (LRU, ≤ 4 MB). On a loss event it re-derives the block encoder and, for rateless codes (RaptorQ, RLC), mints repair symbols at ESIs beyond anything previously sent: any repair fills any hole, so a fresh repair strictly dominates resending the specific lost symbol (which a burst may kill again — fresh repairs also compose across multiple holes in one block). Fixed-rate codes (RS, METTLE) cannot mint post-hoc, so they fall back to resending the exact missing symbols from the retained data, which every decoder accepts. Correction volume per event is missing + ⌊accumulated $\hat{\varepsilon}$ margin⌋ — the margin is fractional and carried continuously across events (at MTU-sized batches an event is typically ONE symbol; a per-event ceil would double the correction volume, i.e. cost 2ε instead of $\varepsilon(1+\varepsilon)$). Repair batches re-enter the ledger, so a lost repair triggers the next round with doubled margin, capped at 3 rounds per block; the receiver's eviction timeout remains the final backstop.

Recovery bound. Detection costs one RTT (the Ack of the next surviving batch), repair transmission another half: a mid-stream hole closes in ~1.5 RTT wall time WITHOUT stalling the send stream — which is exactly the "parallel recovery" the 14.25 cost model prices at zero completion cost, now actually implemented. Ack loss (~$\hat{\varepsilon}$ of batches) is indistinguishable from batch loss and costs one spurious repair event, bounded overhead ~$\hat{\varepsilon}^2$; the receiver drops symbols for already-decoded blocks, so spurious repairs are harmless. Corrections are charged against the same in_flight budget and pacing tokens as scheduled symbols (Section 12.5), and the estimator is NOT fed from the ledger — the receiver-side batch-gap accounting already reports these losses, and double-feeding would bias $\hat{\varepsilon}$ upward.

Honest constants: 64 retained blocks / 4 MB, dup-ACK threshold 3, loss timeout max(1.5·SRTT, 50 ms) clamped to 2 s, sweep cadence 25 ms, margin $\hat{\varepsilon}\cdot 2^{\text{round}}$ continuous, 3 rounds max, 4096-entry ledger cap. L1 verification of the C2 completion gap (8 s ⟶ competitive with the 0.175 s quinn bound) is pending; the mechanism is unit-verified at L0 (Ack-diff, dup-ACK and timeout legs, mixed-block batches, lost-Ack non-amplification, lost-repair second round, LRU caps, margin math, fresh-vs-resend per backend, decode-after-repair for RaptorQ and RS).

14.28 Inner-Feedback Flows and the Repair Floor

Section 14.26′s mid-stream guarantee — $\delta_{bulk} = \hat{\varepsilon}$, $r^* = 0$, pure ARQ — rests on the 14.25 cost model: a mid-stream loss recovers in parallel with ongoing sends, so lateness costs no completion time. This section derives the model revision that argument SEEMED to demand for tunneled control-loop payloads, and then reports the L1 measurement that refuted the revision's premise — kept in full because both the derivation and the refutation constrain

the model. The suspicion: at C2 (100 Mbit, 10 ms RTT, GE 1.3%/50%) the production tunnel carrying a kernel-TCP transfer completed 1.8 MB in 1.11 s median against quinn's 0.20 s, WITH block-mode ARQ (14.27) closing every hole in ~1.5 RTT; the P9b analysis attributed the residual gap to each GE loss event stalling the inner flow's in-order delivery ~1 ARQ round, ~20 events per transfer. A similar effect had appeared at L0 in the P6 goal-gate ablation: a small FIXED r = 0.01 floor beat pure ARQ on completion through straggler coverage.

Why "parallel recovery is free" could fail here. The 14.25/14.26 argument prices the VOLUME of the outer stream: while a hole waits for its repair, other symbols keep flowing, so outer throughput is unharmed and outer completion is untouched. But the tunnel's payload is not inert bytes — it is a control loop. Inner TCP consumes the tunnel's output IN ORDER; an unrepaired outer loss halts that delivery for one outer ARQ round, the inner ACK clock stalls with it, and — IF the stall exceeds the inner loss-detection tolerance — the inner congestion controller converts the stall into a rate cut. Late is fine for a file; late is potentially a throughput signal for a flow that watches its own latency. (The L1 verification below measures that "if": post-14.27 the stalls stay inside the inner tolerance.)

The stall cost model. Per unrepaired loss event the inner flow stalls for one outer ARQ round, capped by the inner transport's own patience (its RTO — after which it retransmits through the tunnel and eats the loss the expensive way):

```
L_stall = min(1.5 x SRTT_outer, RTO_inner)

RTO_inner >= max(RTO_MIN, SRTT_inner),  RTO_MIN = 200 ms (Linux),
SRTT_inner >= SRTT_outer (the inner path IS the outer path plus
tunnel processing), so the implementation uses the observable bound
L_stall = min(1.5 x SRTT, max(0.2, SRTT)).
```

Loss events are GE burst onsets: probability ≈ $\hat{\varepsilon} \cdot \hat{q}$ per wire slot ($\hat{q} = 1/\hat{B}$ from the estimator; $\hat{q} = 1$ degenerates to iid). A proactive repair stream at rate r races each event against the ARQ horizon T_arq = L_stall / t_sym wire slots exactly as in Section 14.16, with the Section 14.14 burst-marginalized recovery probability:

```
C(r) = P(event repaired within T_arq)
     = sum_m (1-q̂)^{m-1} q̂ x P(Poisson(lambda) >= m),
       lambda = T_arq x r(1-ε̂)/(1+r)
```

The expected fraction of wall time the inner delivery spends stalled is then

```
S(r) = ε̂x q̂ x T_arq x (1 - C(r))        [stalled slots per slot]
```

The floor. A stall is invisible to the inner flow when it is indistinguishable from the delivery jitter the flow already absorbs. The repair floor is the smallest rate whose residual stall sits at or below that jitter scale — a continuous trade, not a cutoff:

```
r_min = min { r >= 0 : S(r) <= theta },   theta = sigma_j / L_stall,
sigma_j = SRTT/4
```

σ_j is the 14.26 σ_{arq} evaluated at its SRTT/4 floor: the sender cannot observe the inner flow's actual tolerance, and the production RTTVAR estimate is a fixed-fraction heuristic, so the deterministic branch is the honest choice (it errs toward slightly more repair). S is continuous and nonincreasing in r, so r_min is continuous in every input; when $S(0) \le \theta$ already — clean channel, short horizon — the floor is exactly 0 with no branch. At C2 timescales the floor engages only above $\hat{\varepsilon} \approx 0.0016$; at the operating point it solves to

```
ε^= 0.026 → r_min = 0.029      (T_arq ≈ 200 slots, theta = 1/6)
ε^= 0.035 → r_min = 0.033
ε^= 0.045 → r_min = 0.036
```

— the 0.01-0.04 band the P6 fixed-floor ablation pointed at, now derived rather than tuned. An unknown t_sym (no throughput estimate) disables the floor, the same sentinel convention as the burst B/T term. The floor composes with the rest of the controller by max(): the χ glide still owns the stream tail, and the 14.21 saturation cap still overrides (where more FEC hurts the tail, the floor must not insist).

Who gets the floor: the weight w ∈ [0, 1]. The protocol hint alone cannot decide this. Bulk's "late is fine" is a statement about the PAYLOAD BYTES; whether lateness feeds back is a statement about the PAYLOAD'S DYNAMICS — a second, orthogonal input:

```
rate = max(rate_glide, w x r_min)

w = 1: inner-feedback payload (TCP-in-tunnel: the payload is a
       control loop). Opt-in via the tunnel's inner_feedback_weight
       config; the ORIGINAL plan made this the Bulk-tunnel default,
       retracted after the L1 verification below.
w = 0: file-transfer semantics — the L0 gate driver, the wasm
       simulator, bench_suite, and any payload that is itself the
       object being measured. There mid-stream ARQ recovery is
       GENUINELY free and 14.26 applies unweakened; the gate's Bulk
       volume-parity claims are stated at w = 0. Also the production
       tunnel default (see the verification).
```

Intermediate w scales the floor linearly (a mixed payload tolerates proportionally more stall); the rate is continuous in w, and w = 0 reproduces the old controller identically. When the future idle-onset/T_rem heuristics of 14.26 land, χ and w remain independent: χ says WHERE in the stream you are, w says WHAT the stream carries.

What the floor is not. It is not a retreat from 14.26: M1 and M2 (cold-start pin, permanent double payment) stay dead because the floor is finite, derived from the stall budget, and vanishes on clean channels — unlike the old min(0.1, ε̂) target, which re-activated the full margin machinery against an arbitrary constant. Nor does it replace 14.27′s reactive leg: ARQ still recovers every hole; the floor only buys back the ~1.5-RTT DETECTION latency that no reactive scheme can remove, and only where that latency is a cost. The honest price is volume parity: with w = 1 the tunnel pays ~r_min ≈ ε̂ extra overhead mid-stream, conceding 14.26′s parity claim for inner-feedback payloads — measured against the stall cost it removes (L1 ablation, w = 0 vs w = 1 at C2, below).

L1 verification — the premise is refuted post-14.27 (negative result). Measured on the L1 harness (rp-bulk tunnel, 1.8 MB objects, seed 42, fresh topology per arm, cross-session interference audited via the sudo journal):

```
Instrumentation first: the floor was initially INERT at L1 — the
production estimator had no local throughput measurement (the only
record_throughput feed was the peer's PathReport value, which is the
peer's estimator.throughput(): circular, both sides 0.0 forever), so
t_sym = 0 sentinel-disabled the floor — and, silently, the 14.21
saturation cap and the 8.4 burst B/T term on every real link. Fixed:
the report task now feeds the achieved send rate (symbols sent per
report interval), which is also the correct T_arq slot semantics.
The peer-feed is removed (it would mix the REVERSE direction's send
rate — an ACK trickle — into the data direction's t_sym).

C2 (100 Mbit, 10 ms RTT, GE 1.3%/50%, eps ~ 2.6%), 10 runs/arm:
  w = 0:  median 1.179 s, mean 1.107   (second arm, 5 runs: 0.784/0.955)
  w = 1:  median 1.192 s, mean 1.121   (second arm, 5 runs: 1.158/1.171)
  client FEC volume: 2.46% (w=0, reactive 14.27 repairs only)
                     4.66% (w=1, floor verified ACTIVE on top)
  inner TCP RetransSegs per 5 transfers: 11 (w=0) vs 21 (w=1)
C3 (20 Mbit, 40 ms RTT, GE 2%/40%, eps ~ 4.8%), 5 runs/arm:
  w = 0:  median 5.34 s, mean 5.90
  w = 1:  median 6.82 s, mean 7.00     (+28% median — the floor HURTS)
```

The floor demonstrably fires and pays its budget, and buys nothing: completion is flat at C2, the inner flow's loss-recovery signature does not shrink, and C3 regresses 28%. Two lessons, honestly taken:

1. **The stall premise over-priced lateness.** The P9b residual analysis (~20 events × ~1.5 SRTT) dated from before 14.27′s reactive leg and the P9b in-order delivery hold worked TOGETHER: a mid-stream hole now closes in ~1.5 RTT while the receiver's SRTT-adaptive hold delivers around it, so the inner TCP sees delivery jitter well inside its own tolerance (RTO_inner ≥ 200 ms ≫ the 20-60 ms stalls; 11 retransmits per 9 MB). The model priced EVERY unrepaired loss at L_stall; the true cost gates on P(stall exceeds the inner loss-detection tolerance), which post-14.27 is ~0 at C2/C3 — i.e. the honest r_min is 0, which is what production now defaults to (inner_feedback_weight = 0, knob kept for genuinely brittle payloads — measure before enabling).
2. **Floor repairs are not free in the closed loop.** Repairs are charged against the same cwnd/pacing budget as source symbols (12.5), so at inner-limited rates they displace the very traffic whose latency they protect — the 14.21 dilution cost made system-visible. Worse, T_arq is measured in SEND-PROCESS slots: at low operating rates the horizon holds few slots, r_min grows (5-10% at C3′s 2-3 Mbit/s effective rate), and the tax compounds exactly where the link is tightest. That is the C3 regression.

The 14.26 mid-stream guarantee therefore survives its hardest test to date: even for the TCP-in-tunnel payload it was suspected to fail, pure mid-stream ARQ (with 14.27 implemented) beats ARQ + repair floor. What remains of the C2 gap (P9b: ~1.1 s vs quinn 0.20 s) is NOT unrepaired- loss stalls; the suspects are the residual Copa backoff ceiling and the inner flow's own slow-start (goal-gate P9b items b and c).

14.29 The End-of-Stream Taper Completion Term (All Hints)

Section 4.2′s truncation note established the problem: at a finite transfer's end the taper's forward integral is cut, so the last W source symbols are progressively under-covered and their losses fall to serial ARQ — an end-of-stream reliability cliff. Section 14.25 patched it for Bulk with a one-shot repair burst at end-of-stream, and Section 14.26 made THAT continuous

via the completion-exposure δ glide — but only for Bulk. The tight hints (Auto, Realtime) were left with the ad-hoc one-shot burst. This section derives the truncation loss and gives one continuous completion term that serves every hint, of which the 14.25 burst is the discrete limiting case.

The truncation loss. Let a symbol sit at distance j = N − i from the last source position N. Mid-stream a symbol accumulates coverage r·W over its W window-lifetimes (r = the hint's steady correction rate); at distance j < W only j of those lifetimes occur before the stream ends, so its coverage is r·j and its DEFICIT is

```
Δcov(j) = r · (W − j),     0 ≤ j ≤ W       (0 for j ≥ W)
```

Repairs are window-fungible (Section 3.2), so what matters for the final window is the TOTAL repair mass emitted into [N−W, N): mid-stream that mass is r·W, at the tail it is only what the truncated positions provide. The expected uncovered tail loss is the probability that the final window carries more losses than its (deficient) repairs cover — for the whole window,

```
P(uncovered tail) = P_fail(r_eff, W),   r_eff = coverage mass / W
```

with P_fail the exact transfer-matrix tail (Section 8.7; the normal approximation under-provisions small-W one-shot events by 30–50 %, Section 14.25). Untreated, r_eff → 0 as the window empties and P_fail → 1 − (1−ε)^W ≈ 80 % at ε = 2.5 %, W = 64 — every finite transfer ends with a near-certain serial-ARQ round. The position dependence is entirely through j: the loss is concentrated in the last W symbols and is RTT-independent (the final window has nothing to overlap recovery with, at any RTT).

The completion term. Restore the final window to its full repair mass by injecting the missing budget. The 14.25 burst does this in one shot: B_tail = r_tail·W repairs, r_tail = the exact-DP rate meeting the hint's tail target δ_hint on a window of W (for Bulk, δ_tail = 0.05). The continuous generalization meters that SAME budget as a Stieltjes measure over a completion kernel χ_trunc that rises 0 → 1 across the truncated region:

```
completion debt per source symbol = B_tail · dχ_trunc

χ_trunc(remaining) = Φ( (remaining − W/2) / (W/4) )      remaining = N − i
```

Because χ_trunc is monotone 0 → 1 as the window empties, the total metered is exactly B_tail — one window's worth, released continuously instead of dumped at an instant. Two properties make this the RIGHT kernel:

- **It is over SOURCE POSITION, not wall time.** The truncation is a source-position phenomenon (only the last W symbols), unlike Section 14.26′s completion-exposure χ(T_rem), which is a wall-time ECONOMICS kernel ("is this loss's recovery serial?"). Driving the metering by T_rem would spread the fixed budget over the final ~1.5 SRTT of WALL time — a span of many W at high RTT — diluting the final window and REGRESSING its tail (measured: Realtime last-window p99 27 → 49 ms). The source-position kernel concentrates the budget on exactly the deficient symbols.
- **The one-shot burst is its σ → 0 limit.** As the kernel width W/4 → 0, χ_trunc becomes a step at remaining = W/2 and the whole budget releases at once — the 14.25 burst. The continuous form additionally covers late-stream losses that the burst's single final window misses but that still recover serially at high RTT (the Section 14.26 high-RTT coverage gap), because its repairs are emitted ACROSS the tail rather than only at [N−W, N).

Relation to the Bulk glide (14.26). Bulk and the tight hints now both get a continuous, χ-driven completion term, but through different channels that must not double-fire. Bulk maps completion exposure into δ_eff (the glide raises r from 0 mid-stream to the tail-budget rate), which is simultaneously its "late is fine ⟶ r = 0 mid-stream" economics AND its truncation refill; that is correct BECAUSE Bulk's mid-stream rate is 0, so its final-window refill is the whole of its tail FEC. The tight hints already run r > 0 mid-stream (their δ is tight), so their truncation refill is ADDITIVE on top of the steady rate — the B_tail·dχ_trunc term — and their δ is untouched. In both cases mid-stream coverage is unchanged and exactly one window's worth of extra repairs lands at the tail.

Scope. Like 14.26 this needs a KNOWN end of stream: the driver (or an application-declared transfer size) supplies N. An endless production stream has no last window, so remaining = ∞, χ_trunc = 0, and the term vanishes — production window mode instead closes tail holes with its NACK/tail-sweep path (Sections 6.1, 14.27). Verification (wasm simulator, ε = 5 %/8 %, RTT 50/150 ms, W = 64, shared seeds): replacing the one-shot burst with the metered ramp holds last-window p99 at parity mid-stream (Auto 25 ⟶ 26 ms, Realtime 27 ⟶ 27 ms at RTT 50) and IMPROVES it where the burst's single window under-covers the exposed span (both hints 80 ⟶ 76 ms at RTT 150), for ≤ 2 % extra overhead. The end-of-stream cliff test (last-window p99 ≈ mid-stream p99) passes for both tight hints.

---

15. The Unified Sliding-Window Model (Blocks and Streams as Two Knobs)

The document so far has treated FEC and ARQ as one correction stream (Section 5) but has left a second split standing: the transport still carries *two* FEC data paths, chosen once per tunnel by protocol hint. This section removes that split. It shows that the BLOCK path and the WINDOW path are not two codes but one sliding-window RLC evaluated at two settings of two continuous knobs — window advance and repair timing — and that block mode is the limiting case of streaming under exactly the σ ⟶ 0 collapse the paper already uses for the end-of-stream burst (Section 14.29) and the χ glide (Section 14.26). Making them one code makes *per-stream* triangles possible: one tunnel carrying a tight-δ realtime flow and a loose-δ bulk flow over the same paths at the same time, which the global-mode design structurally cannot do. (A measured amendment, Section 15.7: the two knobs capture the shared coding *algebra* but not the delivery *semantics* — retention policy turned out to be primary — and Section 15.7 further resolves it INTO the triangle: it is ρ, not a new dimension.)

This is a design section (no measured implementation yet); it reuses the existing `RlcEncoder`/`RlcDecoder` (raptorpath-math/src/rlc.rs), `TaperFunction` (Section 4), `derive_window` (Section 8.8), and the Section 13.8 scheduler, and states honestly what it costs and buys.

15.1 The Defect: One Triangle per Tunnel

Two independent FEC data paths exist today, selected per tunnel by the protocol hint:

- **BLOCK mode.** Accumulate K source symbols into a block, FEC-encode the block (RaptorQ / Reed-Solomon / block-RLC backends), send source + repairs, and recover holes reactively with the batch-ACK ARQ of Section 14.27. A dedicated block decoder runs per block.

- **WINDOW / streaming mode.** A sliding-window RLC (the `RlcEncoder` / `RlcDecoder` of raptorpath-math/src/rlc.rs) emits repairs continuously per the taper τ(t) and recovers holes reactively via the NACK / SACK-gap path (Section 14.27′s window analogue, P10b).

Because the choice is **global to the tunnel**, the transport commits to a single point of the bandwidth/latency/reliability triangle (δ, ρ, r) of Section 1.4. A tunnel in block mode is optimising one (δ, ρ, r); a tunnel in window mode is optimising another. Neither can carry a realtime stream (tight δ, small W, high r) *and* a bulk stream (loose δ, large W, r ⟶ 0) at the same time over the same paths. That is the core defect: the triangle is a per-tunnel constant when it should be a per-stream one.

15.2 The Unified Sliding-Window RLC Model

Fix a single mechanism: the sliding-window RLC of Section 3.2, exactly as `RlcEncoder` implements it. The encoder holds a **live window** of the source symbols whose sequence numbers lie in [w_start, w_start + W). A **repair** is a random linear combination over GF(256) of the live window,

```
  p = Σ_{j ∈ window} c_j · s_j ,   c_j = gf256::generate_window_coefficients(w_start,
W, k)
```

(`RlcEncoder::generate_repair`), carrying its (w_start, W, repair_index) so the decoder can reconstruct the coefficients. The window advances by dropping symbols below a new oldest sequence (`RlcEncoder::advance`), and source data is retained for exact ARQ resends (`RlcEncoder::get_source`). Two continuous knobs parameterise the whole design:

```
(a) advance / overlap.  Per emitted window state, move w_start forward by a
    source positions, a ∈ [1, W]. Define overlap o = (W − a)/W ∈ [0, 1−1/W].
      a = 1  (o → 1):  slide-by-1  → consecutive windows overlap in W−1
                        symbols; a symbol is covered by ~W successive windows.
      a = W  (o = 0):  RESET       → consecutive windows are DISJOINT; a
                        repair never mixes symbols across the boundary.
                        Disjoint windows ARE blocks (K = W).

(b) repair schedule.  A per-window-lifetime measure dμ(t) of repair mass,
    total r per source symbol (Section 4.3):
      continuous:  dμ(t) = τ(t) dt = A(1−q)^t dt,  A = r·q   → streaming
      spike:       dμ(t) = (r·W) · δ_Dirac(t − W)           → block batch
```

A third axis this section originally missed: retention policy. As first written, this section claimed the two knobs above were sufficient — that block and window mode "are one sliding-window RLC at two settings of two continuous knobs." A subsequent experiment (the windowed-RLC-all-profiles run, Section 15.7) refuted the sufficiency: the two production modes also differ in *reliability policy* — whether un-acked source symbols may be **evicted** (window mode force-advances past MAX_WINDOW_SIZE and force-delivers past unrecoverable holes) or must be **retained until acked/decoded** (block mode's Section 14.27 ledger). That policy is a delivery-semantics contract, not a schedule, and switching it changes outcomes categorically (10/10 completions ⟶ 0/10 DNF, Section 15.7). The knob algebra of (a) and (b) stands; it is just not the whole state. Section 15.7 amends this section accordingly.

One decoder subsumes both — algebraically. `RlcDecoder` runs incremental Gaussian elimination over equations keyed by sequence number: a source symbol is an identity equation, a repair is the window combination above. It recovers a hole the instant the pivot table

spans it — the number of linearly independent equations touching the unknown reaches the number of unknowns. This single procedure *is* both decoders of Section 15.1:

- **Block decode** is the special case where the fed equations partition by disjoint window (o = 0): the K × K submatrix over one block reaches full rank exactly at the classic "K-of-N present" condition, and the block's pivots complete together.
- **Streaming decode** is the overlapping case (o → 1): pivots complete incrementally as repairs arrive, each covering the whole live window (Section 3.2 window-fungibility).

There is no second code and no second decoder — *as decoders*. But "one decoder subsumes both" is a statement about the shared linear algebra, not about the delivery semantics wrapped around it: the same decoder embedded in an evicting pipeline and in a retaining pipeline produces categorically different transports (Section 15.7). Block and stream differ in (a), (b), **and** in retention policy.

15.3 Block Mode as a Limiting Case

The reduction is exact. Take the unified encoder of Section 15.2 and set

```
BLOCK     = ( advance a = W  [reset, o = 0],
              repair schedule = a spike of mass r·W at the window boundary )

STREAMING = ( advance a = 1  [slide, o = 1−1/W],
              repair schedule = τ(t) = A(1−q)^t at rate r,  A = r·q )
```

Both feed the *same* `RlcDecoder`. What differs is (a) which symbols a repair may combine — a reset window's repair is a random linear combination over exactly the K = W symbols of one block, i.e. a block fountain/RLC repair; a slid window's repair is a combination over the W most-recent symbols — and (b) *when* the r·W repairs per window are emitted.

Block is the σ → 0 limit of the continuous schedule. The block batch "emit nothing until the window is full, then dump r·W repairs at the boundary" is the repair schedule collapsed to a Dirac spike of mass r·W. This is precisely the limiting-case pattern the paper already uses twice:

- Section 14.29 meters the end-of-stream completion budget B_tail = r_tail·W as a Stieltjes measure B_tail · dχ_trunc over a source-position kernel of width W/4, and calls the one-shot burst its **σ → 0 (width → 0) limit**.
- Section 14.26′s χ glide raises Bulk's r from 0 to the tail-budget rate over the final ~1.5 SRTT, with the discrete burst as the width → 0 endpoint.

The block repair batch is the *same* width → 0 spike of the *same* mass r·W — here applied at every window boundary (period W) rather than only once at end-of-stream. Block mode is therefore not a different mechanism; it is streaming with the repair kernel's width taken to zero and its period set to W. The continuous knob is the kernel width σ (and the advance a); block and stream are its two endpoints, with every intermediate (a partial-overlap window emitting a narrow but non-zero repair burst) a valid operating point.

The latency difference falls out of (a) and (b). Under the boundary spike, no repair for source symbol s_i exists until its window fills, so s_i has *no proactive protection* until W − 1 further source symbols arrive:

```
block fill latency   L_fill = W · t_sym = W / send_rate      (before ANY repair)
streaming fill latency ≈ 0   (τ(0) = A > 0: protection from offset 0)
```

L_fill is the same quantity as the Section 8.8 W_lat term and the Section 14.5 W·t_sym recovery span — but paid **up front as encode latency**, not as recovery span. Streaming pays ≈ 0 fill latency and instead *spreads* the same r·W repairs across the window lifetime (each early repair covers fewer already-present symbols; the Section 4.4 taper-never-zero and Section 14.24 encoder-lag considerations apply). The trade is continuous in a: at a = 1 the fill wait is ~t_sym, at a = W it is W·t_sym, and every batch size in between interpolates.

Where each is optimal — as a knob setting, not a mode. Batching amortises: one decode per W symbols instead of incremental GE per symbol (Sections 9, 14.17), and one batch-ACK per block (Section 14.27) instead of per-symbol SACK accounting. So

```
block-like (large a, spike):  optimal when W·t_sym ≪ latency budget AND
                              amortisation matters (high send_rate, non-
                              negligible per-symbol decode/ACK cost).
stream-like (a = 1, τ(t)):    optimal when W·t_sym is a meaningful fraction
                              of the budget (tight δ, low rate, or high RTT).
```

This is exactly the Section 8.8 W* binding logic read through the advance knob: loose-δ Bulk rides a large W with the overhead/latency slack to batch (⟶ block-like), tight-δ Realtime rides a small W at slide-1 (⟶ stream-like). The current binary mode is just these two endpoints with the interior of the (a, σ) square deleted.

15.4 Per-Stream Triangle Multiplexing

Once block and stream are one code, a tunnel need not pick one triangle. Give each application stream m ∈ {1..N} its own sliding-window context with its own triangle (δ_m, ρ_m, r_m, W_m): W_m from `derive_window` at δ_m (Section 8.8), r_m from the controller at δ_m and the stream's own operating point (Section 8.4), advance/repair schedule from where (δ_m, W_m) lands on the Section 15.3 knobs. Each context is an independent `RlcEncoder` / `RlcDecoder` pair.

Independent coding = budget isolation. A repair for stream m combines only stream m's live window, so a bulk stream's burst of losses draws down only the bulk stream's repair budget — it cannot consume the realtime stream's repairs. This is the property the global mode cannot provide and the reason mixed-δ streams need separate contexts (below).

The scheduler multiplexes; it does not block-align. There are no blocks to align across streams: a small message emits its symbols on arrival, into its own window. The Section 13.8 objective already ranks a mixed source+repair symbol stream across paths; it gains a per-stream weight. For a symbol i belonging to stream m = m(i), the per-symbol scheduling cost becomes

```
cost_i = w_lat^{m} · u_m(i) · E_i  +  w_bw^{m} · r_m

  w_lat^{m}, w_bw^{m} : stream m's hint weights (Section 13.8; Realtime
                        1/0, Balanced ½/½, Bulk 0/1)
  u_m(i)              : δ-urgency of symbol i — a monotone function of how
                        close i is to stream m's deadline relative to δ_m.
```

A principled u_m reuses existing machinery: the Section 14.26 exposure kernel χ evaluated at stream m's δ_m and remaining slack, so a symbol whose recovery is about to become serial for a *tight-δ* stream outranks a bulk symbol with ample slack. The scheduler then interleaves all streams' source and repair symbols across paths in urgency order, subject to the Section 13.8

per-path capacity and (for any block-like stream) the in-order delivery-unit coupling already derived there.

N = 1 recovers today's behaviour. One stream, u ≡ 1, weights = the tunnel hint: cost_i reduces to the Section 13.8 objective verbatim, and the single context reduces to the current single global-δ mode. The unification is a strict generalisation — the present design is its degenerate point.

Coding gain vs isolation — stated honestly. Per-stream contexts *lose* cross-stream coding gain: a single shared window over stream A ∪ B would let one repair cover a hole in either, and the combined loss process has lower relative variance (statistical multiplexing), so a shared window needs slightly less *total* overhead for the same aggregate reliability — the Section 8.4 margin scales as $z\cdot\sqrt{(\varepsilon\ \sigma^2\ /\ (W(1-\varepsilon)))}$, and W_A + W_B > either W alone, so the shared-window margin is $O(1/\sqrt{W})$ smaller. Against that, the isolation failure of a shared window is a *δ violation on the tight stream*: a loose-δ flow's loss burst consuming the shared repair budget pushes the realtime flow past its δ_m — a latency-class breach the triangle treats as categorical, not marginal. Trading a bounded $O(1/\sqrt{W})$ overhead saving for a categorical latency-class guarantee favours **isolation whenever the streams' δ differ materially**. When the δ's are equal (homogeneous streams), they *should* share a window — which is simply N = 1 at a larger W, no contradiction.

UEP as a named advanced variant. The alternative that keeps shared coding gain *and* δ-differentiation is unequal error protection: one shared window, but repair coefficients weighted to preferentially protect the tight-δ symbols. It recovers the $O(1/\sqrt{W})$ gain, at the cost of a coupled encoder (repairs are no longer independent per stream, a shared-window decode failure can strand both classes, and the weighting is an extra continuous knob to tune and estimate). Per-stream contexts are the **recommended default** — they are the principled construction (clean isolation, and they reuse the existing single-window encoder N times with no new coding machinery); UEP is future work for capacity-critical links where the coding-gain term is worth the coupling.

15.5 Cost and Benefit (Honest)

What it costs.

- **Per-stream state.** N encoder/decoder contexts, N triangles, N `derive_window` / controller evaluations, and a stream demultiplexer on the receive side. Memory and decode work scale with the number of *active* streams, not tunnels.
- **Scheduler changes.** Section 13.8 must carry per-stream weights and the u_m urgency term, and the interleaver must tag every symbol with its stream id (already needed for demultiplexing).
- **The block backends.** RaptorQ and Reed-Solomon are *not* sliding-window RLC. Two honest options: (i) retire them, accepting RLC's higher decode overhead (Section 9) as the price of one code path; or (ii) keep them as the o = 0 (reset), spike-schedule special case of Section 15.3 — a "large-W batched-repair" backend selected when a stream lands on the block-like corner — behind the same context interface. Option (ii) preserves RaptorQ's low codec overhead for bulk while still presenting one decoder abstraction to the scheduler; it is the recommended migration target.

What it buys.

- **Mixed-δ streams on one tunnel** — the defect of Section 15.1 removed: a realtime and a bulk flow share the paths with their own triangles, isolated budgets, and urgency-ranked multiplexing.
- **One decoder, block and stream as continuous limits** — fewer latent bugs. The two worst FEC bugs the L1 harness found were both artefacts of the *two separate paths*: (1) a **dead reactive-repair path** in window mode — `WindowNack` deprecated with no producer, the SACK-gap wiring cut, the sender draining NACKs only after a TUN read, so *only* proactive FEC ever repaired a window-mode loss (P10b); and (2) a **mis-wired BlockStart** (ADR-0008) — the sender never emitted `BlockStart` and the receiver had no match arm, so block decoders were created with `source_symbols = 0` and silently produced empty data. Neither failure mode exists in a single code path where source and repair feed one incremental decoder.
- **The split is not even delivering its intended benefit** — the measured motivation. At C2 (100 Mbit, 10 ms RTT, GE 1.3%/50%), 1200 B messages at 50/s (goal-gate L2 workstream 2), block mode (Bulk) held a **91 ms p99** tail while window mode (Realtime) — the mode whose entire purpose is the low tail — sat at **513 ms p99**, at equal p50. The mode meant to be the low-latency path had the *worse* tail, for two path-specific reasons the unification erases: 508-byte window-mode MTU symbols fragmenting inner segments (P9a) and window mode's late-maturing NACK path (P10b, item above).

15.6 Migration Sketch

Paper-level, not code. Three phases, each shippable and reversible, ordered so that the risky decoder change lands first behind the existing behaviour.

```
Phase 1 — unify DECODE on the sliding-window RLC decoder.
    Route both paths' received symbols through one RlcDecoder. Block mode
    becomes "feed a disjoint window, decode when the block's pivots
    complete" (Section 15.2). Behaviour-preserving: the reset/spike setting
    reproduces block decode exactly. Retires the separate block decoder.

Phase 2 — express block mode as batched-repair, large-W streaming.
    Move the block ENCODER onto the unified (advance, repair-schedule)
    knobs: block = (reset advance, spike of mass r·W at the boundary),
    Section 15.3. RaptorQ/RS survive as the o = 0 spike-schedule backend
    (option (ii) above) behind the context interface. The binary hint
    switch becomes a point in the continuous (a, σ) square.

Phase 3 — per-stream contexts + scheduler urgency weighting.
    Give each stream its own context and triangle; extend the Section 13.8
    objective with the per-stream weights and u_m urgency term (Section
    15.4). N = 1 stays bit-identical to today; N > 1 unlocks mixed-δ tunnels.
    UEP (shared window, weighted repairs) is a later, optional variant.
```

Already exists (reused, not built): `RlcEncoder` / `RlcDecoder` with GF(256) window combinations and incremental Gaussian elimination (raptorpath-math/src/rlc.rs); `TaperFunction` for τ(t) (Section 4); the block-mode batch-ACK ARQ (Section 14.27) and window SACK-gap recovery (P10b); `derive_window` for W_m (Section 8.8); the Section 13.8 multipath scheduler and its delivery-unit coupling.

New (to build): the (advance, repair-schedule) knob generalisation of the encoder emit loop; the per-stream context table and receive-side stream demultiplexer; the per-stream weight

and u_m urgency term in the scheduler; and the RaptorQ/RS wrapper that presents the o = 0 spike-schedule backend through the context interface (or their retirement, per Section 15.5).

15.7 Amendment: Retention Is the Triangle's ρ, Not a New Axis (measured)

This subsection amends Sections 15.2–15.3 in light of a negative experiment (branch `exp/windowed-rlc-all`, goal-gate 2026-07-05) that tested the unification's implicit premise directly.

An earlier draft of this amendment called retention "a third, primary axis" with two values {evict | retain-until-acked}. That was a new binary where the model already has a continuum: **retention is the triangle's ρ, realized by T_cut** (Section 6.1 age-based give-up; Section 6.2 receiver pruning). The sent-data store's removal rule is: remove on ACK, or when the entry's age exceeds T_cut(ρ) — with ρ = 1 giving T_cut = ∞ (ack-only, the bulk contract) and ρ < 1 giving bounded retention continuously. "Reliable" and "lossy" are not policies to switch between; they are the ρ ⟶ 1 limit and the finite-T_cut interior of one dial. A corollary that the current window pipeline violates: give-up must be AGE-based (T_cut, from ρ), never SPACE-based — buffer fullness is a flow-control signal (backpressure), not a licence to destroy data. The measured failure below is exactly a space-based eviction masquerading as a reliability policy.

The experiment. Production Bulk/Auto were switched onto the window pipeline — `is_window_mode` extended from Realtime-only to all hints, RLC selected — exactly the "one sliding-window code for everything" this section argues for. Result at C2 (100 Mbit, 10 ms RTT, GE 1.3%/50%, seed 42, rp-native `perf`, 1.8 MB × 10, same binary A/B):

```
block-mode Bulk (RaptorQ + §14.27 batch-ACK ARQ):  10/10 complete,
                                                   mean 0.895 s (16.1 Mbit/s)
window-pipeline Bulk (RLC, the realtime pipeline): 0/10 — ALL DNF
                                                   (600 s wall)
```

Root cause — a policy, not the code. The window pipeline is *loss-tolerant by design*: the sender never blocks on window fullness — past `MAX_WINDOW_SIZE = 200` it force-advances, **evicting un-acked source symbols** that can then never be regenerated or retransmitted — and the receiver **force-delivers past unrecoverable holes** on reorder-buffer expiry. Both behaviours are *correct* for Realtime's triangle (a drop-tolerant δ: a stale packet is worthless, so eviction is the right spend of the budget) and *fatal* for bulk's every-byte contract: at C2′s loss rate a 1.8 MB object is ~1520 symbols with ~20 loss events, and any loss not repaired within the ~200 symbols the window spans (≈ one RTT at line rate) is permanently gone ⟶ DNF.

The amendment. Sections 15.2–15.3 present block and window as *two settings of two knobs* (advance/overlap, repair schedule) over one decoder. That is true of the coding algebra and remains the basis for the migration sketch — but the experiment shows the two production modes ALSO differ in a third property the knobs do not capture:

```
retention policy ∈ { EVICT              (advance unconditionally; losses
                                         past the horizon become holes;
                                         bounded memory, bounded delay)
                   | RETAIN-UNTIL-ACKED (advance only on ack/decode;
                                         window fullness becomes back-
                                         pressure on the source, never
                                         data loss) }
```

This axis is **primary, not a tuning knob**: moving a bulk flow across it flipped completions from 10/10 to 0/10. It is a *delivery-semantics contract* — the ρ corner of the Section 1.4 triangle made structural — where (a) and (b) merely reshape latency and overhead within a fixed contract.

Policy is not codec. The failure was NOT "RLC is unreliable": RLC with a retention policy — sent source bytes retained in an ARQ-layer store until acked, retransmitted until delivered (exactly what the wasm visualizer sim implements at ρ = 1, and what the Section 6.1 retransmit machinery provides once it carries data, not just metadata) — is a fully reliable code. The coding window itself need not be gated: reliability is the ARQ store's contract, the window is only the FEC horizon (see 16.3). Conversely RaptorQ blocks are reliable only because the Section 14.27 retention machinery (64-block LRU ledger, batch-ACK diff, fresh-repair minting) around them makes them so — strip it (the pre-P8 production state) and block mode abandoned failed blocks too. The reliability axis lives in the *pipeline policy*; the codec (RLC / RaptorQ / RS / METTLE) is orthogonal symbol algebra. Any unified design must therefore carry retention as an explicit per-stream policy parameter alongside (a) and (b) — the per-stream contexts of Section 15.4 are its natural home (Realtime streams run EVICT at their δ; bulk streams run RETAIN-UNTIL-ACKED), and the Section 15.6 migration phases inherit it as a third context field. Section 16.3 builds on exactly this axis; Section 16.4 explains why the resolution is one policy-parameterised pipeline rather than mode/backend switching.

---

16. Reliable Windowed Multipath: an Order-Statistic Formulation

This section replaces an earlier draft ("Fountain Multipath Aggregation — Out-of-Order is the Unlock") whose central claim — that *in-order delivery* is the structural obstacle to heterogeneous multipath aggregation, and that out-of-order fountain delivery is the unlock — did not survive scrutiny. Two things overturned it. First, a negative experiment: moving bulk onto the window pipeline (the paper's own unification, Section 15) regressed 10/10 completions at 0.90 s to 0/10 DNF, because the pipeline's *eviction policy*, not its ordering, destroyed the transfer (Section 15.7). Second, a sharper reading of the measured C8 numbers: what caps aggregation is not the in-order contract itself but **per-path-affine atomic delivery units** — with cross-path coding over a sliding window, an in-order frontier can aggregate at the full Σ g_i (Section 16.2). Out-of-order delivery survives only in two subordinate roles: as the mechanism *within* the coding window, and as a convenience for native whole-object delivery.

Epistemic convention for this section. Every quantitative claim is tagged:

- **MEASURED** — a number from the goal-gate record (L1/L2 harness, real links, seeds stated there).
- **DERIVED** — follows from the stated model (bounds, functionals); no new measurement needed.
- **PREDICTION** — awaits the Section 16.6 experiment; falsifiable as stated.

The measured baseline this section builds on (all at L2 workstream 1, topology C8 = WiFi 100 Mbit / 10 ms RTT / GE ε ≈ 2.5% + LTE 20 Mbit / 40 ms RTT / GE ε ≈ 4.8%; 50 MB bulk, seed 42):

```
within-block per-symbol striping     8.8 Mbit/s   (below fast path alone)
whole-block path affinity (§13.8)   12.6 Mbit/s   (= kernel-MPTCP parity)
fast path alone                     14.0 Mbit/s
kernel MPTCP dual                   12.6 Mbit/s
C7 symmetric dual (control)         23.9 Mbit/s   (MPTCP: 15.4)
```

Notation as in Sections 1.1 and 13.2: g_i = C_i·(1−ε_i) the per-path goodput [sym/s], d_i the one-way delay, K the object size in source symbols, s the symbol size, W the window, φ the rateless/codec overhead (Section 9), δ/ρ/r the Section 1.4 triangle.

16.1 Three Regimes, Three Decode Predicates

A transport delivers data in **units** — a packet, a 64 KB block, a coding window, a whole object. Multipath completion is governed by *which symbols the decoder must wait for* before a unit (or the stream frontier) can advance. The three schedules measured at C8 correspond to three decode predicates, and each predicate has a known queueing-theoretic shape.

(1) Within-unit striping across paths = fork-join. The unit's K_u symbols are split x_i·K_u to path i; the decoder needs **all of these specific symbols, scattered over paths**. Per-unit completion is a maximum:

```
T_unit = max_i ( x_i·K_u / g_i + d_i + a_i·RTT_i )            (16.1)
```

— path i's share serialised at its goodput, plus its delay, plus a_i ≥ 0 recovery rounds at *its own* RTT when its share contains a loss (per-unit loss probability 1−(1−ε_i)^{x_i·K_u}: at C8, 1−(1−ε_B)^{K_u} = 0.94 for a whole K_u = 56 unit at ε_B = 4.8% — the Section 13.8 measured mechanism). Completion pays the **expectation of a maximum, once per unit**. This is the classic fork-join queue: for homogeneous branches the mean response grows as ~H_N (the harmonic-number law of [Nelson1988]); for heterogeneous branches the slowest leg dominates and E[max] ≫ max_i E[·] because the max concentrates on the straggler's *tail*, not its mean — the redundant-request literature quantifies exactly this penalty ([Joshi2017]). Heterogeneous delays and losses therefore make striping strictly worse than not using the slow path at all. **MEASURED: 8.8 Mbit/s at C8 — below the 14.0 fast path alone.**

(2) Whole-unit path affinity + in-order release = resequencing queue. Each unit rides one path (block-granular affinity), so units complete independently at their path's own rate — but they must be *released* in unit order. The delivery frontier is then a **running maximum** over unit completion times: frontier(n) = max_{m ≤ n} T_m. This is the resequencing buffer of the multipath literature ([Xia2003]); kernel MPTCP is the same queue with unit = packet. The receiver holds an out-of-order unit at most H (production: 4·SRTT clamped [60, 300] ms) before force-delivering a hole, so a path may carry ordered units only while its per-unit delivery time stays within H of the fastest path's. Define the **eligibility set**

```
E = { i : D_i − min_j D_j ≤ H },   D_i = K_u/C_i + d_i + P_blk,i·2·RTT_i
```

Then for any affinity schedule the sustained in-order rate collapses to the eligible paths' goodput (DERIVED — the ordered frontier can make progress at most as fast as ordered units arrive, and an ineligible path's units arrive as holes, contributing zero or negative useful throughput):

```
T_inorder ≥ K / Σ_{i ∈ E} g_i                                  (16.2)
```

On sufficiently heterogeneous paths E collapses to {fast} (at C8: D_B − D_A ≈ 130 ms > H/4 — the slow path is hold-infeasible, Section 13.8) and the bound degenerates to K/g_A — **the fast**

path alone. MEASURED: 12.6 Mbit/s at C8, kernel-MPTCP parity (MPTCP: 12.6), below fast-path-alone 14.0 — both transports sitting on the same resequencing bound. On *homogeneous* paths E = {all} and affinity aggregates fine — MEASURED: C7 symmetric dual 23.9 Mbit/s vs MPTCP 15.4.

(3) Rateless coding over a horizon = K-of-N. Code over a horizon of K_h source symbols (a window, or a whole object) and pour encoded symbols across all paths. The decoder needs **any K_h·(1+φ) useful symbols from the pooled arrival process** — no symbol is bound to a path or a position within the horizon. Completion is the **K_h·(1+φ)-th order statistic of the superposed arrival process**, whose rate is Σ_i g_i:

```
T_horizon ≈ K_h·(1+φ) / Σ_i g_i  +  skew term paid ONCE per horizon (16.3)
```

The skew term (startup d_i spread, one decode pass, geometric short-fall top-up) is O(RTT_max) and does not scale with K_h. The structural difference from (1) and (2): there is **no per-unit max and no resequencing coupling**. The coding gain for multipath is exactly the move from *the expectation of a per-unit maximum* (paid K/K_u times) to *one interior order statistic of the pooled process* (paid once per horizon) — a **mean AND variance win, and one that grows with path heterogeneity**, because E[max] − (interior order statistic) widens as the per-path delay/loss distributions spread apart, while on symmetric paths the two coincide and the gain vanishes. That monotonicity is this section's sharp, testable signature (PREDICTION for our stack; the functional forms themselves are DERIVED, and are the transport analogue of the coded-download latency results of [Joshi2014, Joshi2017]).

Summary:

schedule	predicate	completion functional	C8 measured
within-unit striping	ALL of these, scattered	Σ_units E[max_i(...)] (max per unit)	8.8 Mbit/s
whole-unit affinity + in-order release	units independent, released in order	running max ⇒ K/Σ_{E} g_i	12.6 Mbit/s (= MPTCP)
rateless over horizon	ANY K_h(1+φ) of the pooled arrivals	K_h(1+φ)/Σ_all g_i + skew once	− (§16.6)

16.2 The Sliding-Window Realization (In-Order Is Not the Bottleneck)

Regime (3) as stated assumes a horizon. The earlier draft took the horizon to be the *whole object* — encode the object as one fountain, deliver out-of-order, decode on total. That is **not implementable as the general mechanism**: a tunnel does not know object boundaries or sizes ahead of time, streams are unbounded, and a whole-object horizon means unbounded encoder/decoder memory and a decode that cannot begin until the end. It survives only as a special case for the native object API, where the object is known and bounded (Section 16.6).

The realistic carrier of the order-statistic gain is a sliding window spanning multiple former blocks. Repairs are combinations over the current window (the Section 15.2 algebra, unchanged); source and repair symbols are distributed across paths by work-conserving pull (Section 16.6, prerequisite 2 — each path drains the shared window at its own CC-gated rate, so allocation converges to per-path goodput without an explicit splitter); and the receiver maintains an **in-order delivery frontier** that advances whenever the window *prefix* decodes from ANY sufficient subset of received symbols. A gap at the frontier does not wait for a specific path's retransmit: it is filled by whichever path's repair lands first — the fill rate is

the pooled Σ g_i, not the losing path's RTT. In-order delivery-delay analyses of exactly this construction (streaming code + in-order release) exist in the literature and show the frontier delay collapsing once repairs are cross-scheduled [Cloud2014].

This yields the correction that renames this section (DERIVED):

> **In-order delivery is not the bottleneck.** With cross-path coding over a sliding window and a retention policy (Section 15.7), the in-order frontier advances at ≈ Σ_i g_i — the full aggregate. What caps aggregation in the measured system is (i) **per-path-affine atomic units**: a 64 KB block whose source symbols ride exactly one path makes the *unit*, not the ordering, the thing that serialises at one path's rate and recovers at one path's RTT — the eligibility set E and the resequencing bound (16.2) are consequences of block atomicity, and MPTCP hits the same bound because its unit (the packet with a fixed sequence number) is equally path-atomic once sent; and (ii) in the lossy window pipeline, **eviction** — which converts a late repair into a permanent hole (Section 15.7).

Caveat measured after the fact (see Section 16.7). The claim above — frontier at Σ g_i — holds only when the window is **rateless-fungible** (the frontier advances on ANY sufficient K_h(1+φ) symbols). At bulk's operating point r ≈ ε (~2%), the sliding window is *systematic* (source striped in sequence order, tiny redundancy), so a source symbol on the slow path IS a specific in-order position the fast path cannot decode around, and the frontier reverts to fork-join. Section 16.7 states this r-regime qualifier precisely and gives the measured pivot (RWM Phase B: symmetric 1.41×, heterogeneous 12.5). Two dials — the reorder horizon H and the repair rate r — restore aggregation; the triangle's δ picks which.

The earlier draft's theorem ("FEC-multipath beats ARQ-multipath **iff** delivery is out-of-order") overclaimed. What the eligibility argument actually proves is bound (16.2) *for per-path-affine atomic units*; it says nothing about in-order delivery of a cross-path-coded window, which evades the bound not by dropping order but by making symbols fungible across paths *within the horizon*. Out-of-order delivery retains two legitimate roles:

- **Within the window horizon** symbols arrive in any order, from any path, and are fungible inputs to the prefix decode — this is where the order-statistic gain (16.3) lives, W symbols at a time.
- **Whole-object atomic release** remains a convenience for the native object API (`raptorpath perf` reassembles by (obj_id, chunk_idx) and tolerates arbitrary reordering) — it lets the frontier machinery be skipped entirely, but it is an API simplification, not the source of the aggregation gain.

16.3 The Missing Quadrant: Reliable Windowed Multipath

Two independent design axes emerged from the measured record — and neither is the codec. The **reliability policy** axis (Section 15.7): { EVICT | RETAIN-UNTIL-ACKED } — a pipeline policy; RLC under retention is fully reliable, RaptorQ under the pre-P8 pipeline (no ledger) silently abandoned failed blocks. The **unit structure** axis (Section 16.1): { atomic blocks, path-affine | sliding window, cross-path striped }. The production system and the experiments populate three of the four quadrants:

	atomic blocks, path-affine	sliding window, cross-path striped
EVICT (lossy by policy)	(uninteresting: lossy blocks would drop 64 KB at a time)	production REALTIME. Correct for its δ — MEASURED working; single path by construction (F3: no striping sender exists)
RETAIN-UNTIL-ACKED (reliable)	production BULK/AUTO (RaptorQ 64 KB blocks + §14.27 batch-ACK ARQ). Correct single-path — MEASURED 10/10 @ 0.90 s; multipath capped by (16.2) — MEASURED 12.6	— EMPTY — Reliable Windowed Multipath (RWM): the code this section proposes. Exists nowhere in production today.

and the measured anti-diagonal: EVICT × window for BULK — the exp/windowed-rlc-all experiment — 10/10 @ 0.90 s → 0/10 DNF (Section 15.7). Reliability policy is a PRIMARY axis, not a tuning knob.

RWM defined. Reliable Windowed Multipath = RETAIN × sliding window × cross-path striped:

1. **Retention — at the ARQ layer, not in the coding window.** The window slides freely: it is only the FEC horizon (fungible repair coverage for recent, not-yet-localized losses). Reliability is the contract of a **sent-data store**: every sent source symbol's bytes are retained until ACKed or until age exceeds T_cut(ρ) (Section 6.1) — ρ = 1 gives T_cut = ∞, i.e. ack-only removal; never removal by space pressure; a SACK-confirmed hole that has aged out of the window is recovered by a targeted retransmit of exactly that symbol from the store, on the best available path — once a loss is localized, fungibility has no value and one exact symbol is the cheapest correction (Section 5′s corrections = repairs ∪ retransmits, with the window/ARQ split falling out of loss-localization). Store fullness (~ a few × BDP of plain bytes, no coding cost) becomes **backpressure** on the source (flow control), never data loss — the exact place the evicting pipeline instead destroys data. The Section 14.27 batch-ACK ledger (retained source, ACK-diff, targeted resend) is the donor: it already implements this for blocks. A consequence: the W_mp bound of 16.5 SOFTENS from a requirement to a continuous trade — W sets the share of recovery that is fungible-repair (in-window) versus targeted-ARQ (aged), so an undersized window costs recovery latency on aged holes, not correctness.
2. **Striping — one continuous placement law, no load regimes.** No proportional (and certainly no equal-weight) splitter, and no backlog-vs-partial-load case split either: every symbol is placed by a single marginal-cost rule — the Section 13.8 objective completed with live congestion and fate terms, `cost_i(s) = w_lat·E_i(load) + w_bw·r_i + w_div·ρ_fate(s,i)`, sampled as P(i) ∝ exp(−cost_i/T). Because E_i includes the path's CURRENT queueing delay (the dq the delay-based CC already measures), the apparent regimes are equilibria of this one rule, not modes: under light load the empty best path has lowest cost and traffic concentrates there; as its queue builds, its marginal cost rises continuously until the next path's cost is crossed and symbols spill gradually; under backlog all marginal costs equalize at the capacity waterlines — water-filling is the FIXED POINT of marginal-cost equalization, and per-path CC-gated pull is its distributed imple-

mentation (each path's token availability IS its marginal-cost signal). The temperature T is the one dial from strict ordering (T⟶0) to burst-decorrelating dithering (deterministic order maps consecutive window positions to one path, so a GE burst punches a contiguous window hole; dithering scatters the damage — PREDICTION, measure before claiming). Repair placement uses the SAME law: the old hard avoid-rule becomes the continuous ρ_fate penalty (fate-correlation with the symbols the repair covers). Block affinity dissolves per-symbol (no block unit remains to bind). Convention note: hard sets remain legitimate inside derived BOUNDS (e.g. the 16.2 eligibility set is an analysis device); the no-cutoffs rule binds MECHANISMS — no control law may case-split. No symbol is bound to a path.

3. **Frontier decode.** The receiver delivers the in-order prefix the moment any sufficient symbol subset decodes it; holes at the frontier are raced by all paths' repairs (Section 16.2).

RWM is the transport that realises predicate (3) for bulk: completion governed by the pooled order statistic (16.3) window-by-window, target rate Σ_i g_i.

What exists today, honestly. Production window mode has **no multipath**: every source symbol goes to `best_source_path()` — the single lowest-cost path with cwnd headroom, spilling to the next-best only on saturation — and repairs to `best_repair_path()`; nothing stripes (MEASURED code fact, Phase-0 reconnaissance of the windowed-RLC experiment). Bulk never touches the window pipeline (Section 15.1). And the closest existing relative of RWM is not in production at all: it is the **wasm visualizer simulation**, which runs a sliding-window RLC for all hints with ρ = 1 — retransmit-until-delivered, no forced eviction — i.e. a *reliable sliding window*, but single-path. The sim therefore models RWM's reliability semantics and window mechanics while modelling neither its striping nor, for bulk, anything production actually runs — a correspondence gap the paper's earlier sections (which lean on the sim) inherit and that is now stated explicitly.

The fungible construction, made concrete (the coded-object mode this task builds). §16.7 measured that the reliable striped window, at bulk's systematic operating point, caps at fork-join parity (≈ ×0.92 in the oracle, 0.76–0.81 at L1) — and localized *why*: a **systematic** window sends raw source symbols in sequence order plus sparse repair, so a source symbol striped onto the slow path IS a specific in-order position the fast path holds no degrees of freedom to decode around. It arrives at the slow path's rate; the frontier waits for it; the aggregate collapses to the order- eligible set E = {fast}. That specific-symbol **long pole** is the whole of the cap. The empty quadrant is filled by removing it at the source:

> **Emit CODED symbols only.** In coded-object mode every transmitted symbol is a random linear combination over the *current sliding window* of K_W source symbols (the §15.2 RLC algebra the encoder already generates as "repairs"; the change is that this mode sends **no raw systematic source at all** — the systematic pass-through is switched off). Each coded symbol carries its window coordinates + coefficient seed on the existing repair wire, so it is self-describing to the decoder.

Three consequences make this the fungible regime (3) of §16.1, not the fork-join regime (1):

1. **No symbol is specific → no long pole (DERIVED).** Any K_W linearly independent coded symbols — *from any path, in any order* — reconstruct the K_W window sources by one bounded Gaussian elimination. Nothing waits for a *particular* symbol, so a slow-path

symbol is never a position the fast path must stall on; it is one interchangeable degree of freedom among many. Reception overhead is effectively MDS over GF(256): expected excess ≈ 1/255 of a symbol for independence (tighter than RaptorQ's +1–2 per block), so completion is the K_W(1+φ)-th order statistic of the *pooled* cross-path arrival process — rate Σ_i g_i (16.3), not the straggler's rate. This is exactly why the systematic window caps at ≈0.92 (a source symbol is a fixed position ⟶ §16.2 fork-join bound) while the coded window reaches Σ g_i (no fixed position exists). The two differ only in whether a transmitted symbol is raw-and-positional or coded-and-fungible; everything else — retention, placement, frontier — is shared.

2. **Window sized to the cross-path lag: W ≈ W_mp (DERIVED + oracle-checked).** The §16.5 lower bound W_mp ≳ Σ_i g_i·(RTT_max + t_slack) ≈ 600 symbols at C8 sets how far the window must span so a slow-path symbol's lag still falls inside the fungible horizon (covered by a later coded symbol, not stranded as aged ARQ). The independent-GE oracle (`multipath_oracle.rs::oracle_c8_fungible_wmp_window`), run at the exact C8 netem params, confirms the *finite* window suffices: a coded fungible window aggregates to **×1.19** (the goodput ceiling ×1.195) for every W ≥ 384 at a modest repair rate r ≥ 0.05, and to ×1.15–1.18 even at r = 0 for W = 600–1024 (MEASURED, oracle). The earlier §16.7 sweep's ×0.99 at H = 256 was simply W < W_mp; at and above W_mp the fungible ceiling is reached. So the production target is proven reachable by *this specific finite-window design*, not only by the unbounded whole-object horizon.

3. **Object completion out-of-order; ARQ backstops the tail (DERIVED).** The receiver decodes each window on any sufficient K_W-subset and delivers the recovered symbols out-of-order, reassembled by offset (the §16.7 Phase C delivery policy, H ⟶ ∞ for a file). The retention/ARQ layer (§16.3 point 1, Phase A) remains the backstop for the last partial window and any window that never accumulates K_W in-flight.

Decode-cost bound (MEASURED). Coded-only decode is one incremental RLC Gaussian elimination over W_mp, cost ~O(W²) in the window. The direct throughput measurement (§16.5: 1200 B symbols, single core, encode+decode) gives **708 Mbit/s at W = 512** and 1.28 Gbit/s at W = 256 — 7–35× headroom over the 20–100 Mbit/s lossy cells at W ≈ W_mp, so compute does not bind below roughly gigabit line rates at these windows.

Scope caveat — bulk / loose-δ only (DERIVED). Coded-only pays a K_W-symbol **decode latency before *any* byte is delivered**: nothing decodes until the window accumulates K_W(1+φ) independent symbols. That is correct for a bulk object (no consumer reads offset k before the file is whole — §16.7′s decode-on-total equivalence) but **wrong for realtime or in-order low-latency byte streams**, which need the systematic window's immediate per-symbol pass-through. So coded-object is a *bulk-object, loose-δ* mode, composed behind a flag with the reliable window (Phase A) and out-of-order delivery (Phase C); realtime and in-order-stream profiles stay on their existing systematic modes untouched (§16.4′s one-pipeline, per-stream-parameterised thesis — coded-vs-systematic is one more parameter point, not a new pipeline).

Status — built, oracle-confirmed reachable, MEASURED at L1, and REFUTED there (goal-gate "Fungible Frontier", 2026-07-07). The coded-object mode was implemented exactly as above (coded-only wire symbols, W widened to W_mp = 640, retention/ARQ backstop, out-of-order object completion; behind a `window_coded_only` flag composing with

`window_reliable` + out-of-order) and measured at C8 = c2+c3 netem (independent qdiscs, 50 MB native `perf`). The prediction that it would strictly beat fast-path-alone (15.68 Mbit/s) is **REFUTED**:

```
coded-only C8 het  (c2+c3) dual    3.9 Mbit/s mean (median 4.5, stdev high, x6)
coded-only C7 sym  (c2+c2) dual    5.5 Mbit/s mean                        (x3)
coded-only         SINGLE c2       12.9 Mbit/s                            (x2)
systematic fast-path-alone c2      15.68 Mbit/s   (the bar)
```

Three facts localize the failure and are sharper than "it did not aggregate":

1. **The coded-object mechanism is CORRECT.** Loopback and netem both complete with all bytes, decode-on-K, zero systematic passthrough. It is a working fungible transport, not a broken build.
2. **The bottleneck is NOT heterogeneity/straggler — it is cross-path coding itself.** Coded-only *single-path* runs at 12.9 (near systematic, the ~18 % gap is the O(W)-per-symbol codec cost of making 100 % of symbols W_mp-wide combinations instead of ~2 %). But *dual* is WORSE than single on BOTH symmetric (5.5) and heterogeneous (3.9) paths. Adding any second path drags the coded pipeline down — the opposite of the oracle's monotone aggregation. So the independent-GE oracle's ×1.19 is **not realized on the real sliding-window + per-path-timing + CC + ARQ stack**.
3. **The mechanism of the drag (DERIVED from the above).** A coded symbol is a combination over the sender's window *at send time*; a symbol striped to a path arrives one path-delay later, by which point the sender's window (and the receiver's decoded frontier) has advanced — on the fast path by ~$\Sigma g \cdot RTT \gg W\_mp$ — so a second path's symbols land covering windows already decoded past (redundant) or misaligned, contributing little useful pooled rank while adding decode load, cross-path reordering, and NACK/recovery churn (each transient undecoded seq is congestion-throttled ARQ, §16.7). The oracle abstracts all of this away (instantaneous pooled decode over a fixed horizon); the gap between ×1.19 (oracle) and ×0.25 (L1) is precisely that abstraction.

Honest §16 position (updated). The §16.3 empty quadrant now has a *correct implementation* and an *independent-GE proof of achievability* (×1.19), but the L1 transport does not realize it: heterogeneous aggregation-above-fast-path remains **OPEN and unrealized in production**, and coded-only over the current per-path-timed sliding window aggregates *negatively*. What the oracle's ×1.19 and the L1′s ×0.25 together establish is that the missing piece is not fungibility-in-the-abstract (built, proven) but **cross-path window alignment** — coding horizons whose per-path arrivals pool over the *same* live window — which the send-time-windowed RLC does not provide. That is a named, scoped next mechanism, not a knob; §16.5′s W_mp sizing is necessary (it lifted the number from 2 to 4.5) but not sufficient.

16.4 One Pipeline, Not Mode Switching

The two-pipeline structure (Section 15.1) invites a tempting patch: keep both pipelines and *switch* between them — or between codecs — as conditions change. The record says the switching itself is the defect, for four reasons, while *codec choice per stream* is legitimate and cheap:

(a) No cross-code algebra. A repair symbol of one code cannot help decode another code's in-flight data. Any mid-stream switch therefore either strands every in-flight symbol of the old code or forces a drain barrier (stop, flush, restart) — a latency cliff by construction.

MEASURED: the P9a bring-up found window-mode backend switching had to be **pinned off** in production — a switch rebuilt the encoder with sequence numbers restarting at 0 while the receiver's delivery/ACK state (highest_delivered_seq, SACK ranges) and the sender's retransmit buffer kept the old numbering, leaving the ACK/NACK repair machinery blind for ~a full window of traffic; at lossy cells that repair blackout wedged the inner TCP for minutes (the hazard note in net/mod.rs stands).

(b) State does not transfer. The estimator, controller, and ARQ ledgers carry code-specific semantics — a block ledger keyed by (block, batch) has no meaning to a window's per-seq SACK state, a window's taper phase has no block analogue. A switch discards exactly the state the next seconds of recovery need.

(c) The existing auto-switch is a threshold machine. Block mode today selects its backend by hard loss thresholds: $\hat{\varepsilon} < 0.01 \longrightarrow$ RaptorQ, $0.01 \le \hat{\varepsilon} < 0.12 \longrightarrow$ RLC, $\hat{\varepsilon} \ge 0.12 \longrightarrow$ METTLE, re-evaluated with a ≥ 5 s minimum interval plus debounce (`config.rs`, `backend_selector.rs`). This violates the paper's own no-hard-cutoffs convention (the same discipline that replaced the r* cutoff branch in Section 8.4 and the hard saturation cap in Section 14.21.1) and carries the standard oscillation hazard: an $\hat{\varepsilon}$ sitting near a threshold buys periodic switches, each paying cost (a). An honest note: this existing mechanism deserves reconsideration on the same grounds as this section — it predates the switching analysis above.

(d) Two pipelines means everything is built twice — and debugged never. MEASURED: window mode's reactive repair path was *dead code* (no NACK producer, SACK gaps ignored, drain gated on TUN reads — P10b) while the block pipeline received P7 pacing and P8 ARQ; the two worst L1 FEC bugs were each single-mode artefacts (Section 15.5). Every subsystem — CC coupling, ARQ, reorder delivery, MTU handling — exists twice and diverges.

The principled resolution (strengthening Section 15.4): **one pipeline, parameterised by the policy axes** — retention (Section 15.7), advance/ overlap a, window W, the (δ, ρ, r) triangle, and striping — with codecs as per-STREAM plug-ins behind the context interface, chosen once at stream setup by workload economics: RaptorQ where its near-optimal overhead at large K pays (bulk objects), RLC where the natural sliding window fits (streams), and **never switched mid-stream**. A new stream gets a new context, so no cross-code boundary ever exists *inside* a stream — cost (a) becomes structurally impossible rather than carefully avoided. Profiles (Realtime/Bulk/Auto) become parameter points of one machine, not modes of two; mixed-profile streams coexist on one tunnel via the Section 15.4 per-stream contexts.

16.5 Choosing W for Multipath: a Fourth Bound

Section 8.8 derives W* from three bounds — the overhead knee W_over, the latency ceiling W_lat, the burst floor W_bur. RWM adds a fourth, a **lower** bound: the window must span the cross-path recovery horizon, or the frontier starves on skew.

The argument (DERIVED): a hole at the frontier is raced by repairs that are combinations over the *current window*. The race takes up to one feedback round on the slowest useful path, ≈ RTT_max, plus scheduling slack t_slack (one repair-generation + pacing interval, of order the fast path's RTT). During that race the aggregate keeps arriving at $\Sigma_i\, g_i$; if those arrivals overrun the window span, the encoder faces exactly the choice Section 15.7 forbids resolving by eviction — and under retention the window instead stalls the source (backpressure), collapsing the pour to stop-and-wait on skew events. So sustained $\Sigma\, g_i$ aggregation requires

```
W_mp ≥ Σ_i g_i · (RTT_max + t_slack)        [g_i in sym/s]     (16.4)

W*   = clamp( max(W_over, W_mp),  min(W_bur, W_lat),  W_lat ) — multipath
       reading of §8.8; single path leaves W_mp = g·RTT ≈ the §14.5 term
```

Worked C8 example. g_A = 100 Mbit/s·(1−0.025) ≈ 10 160 sym/s at s = 1200 B; g_B = 20·(1−0.048) ≈ 1 980 sym/s; Σ ≈ 12 100 sym/s. RTT_max = 40 ms, t_slack ≈ RTT_A = 10 ms:

```
W_mp ≈ 12 100 × 0.050 ≈ 600 symbols
```

— i.e. **3× the window pipeline's MAX_WINDOW_SIZE = 200 and above Section 8.8′s [16, 512] clamp ceiling**. The current constants would starve RWM at C8 by construction; this is a paper-level extension of the Section 8.8 derivation (the production `derive_window` is not changed here). The upper bounds still apply and now genuinely bind: W remains capped by the latency budget (W_lat), by memory (W symbols retained per stream under the retention policy), and by decode cost — RLC's incremental pivot GE grows ~O(W²) in the window, though the measured costs at production windows leave headroom (MEASURED, P9b non-finding: RLC decode p50 37 µs, p99 < 1 ms; it was explicitly ruled out as a bottleneck at C2 rates). A direct decode-throughput measurement at bulk-realistic parameters (MEASURED, 2026-07-06: 1200 B symbols, 2.6 % GE loss, r = 5 %, single core, encode + decode combined) gives 2.84 Gbit/s at W = 64, 1.28 Gbit/s at W = 256, and 708 Mbit/s at W = 512 — 7–140× headroom over the 20–100 Mbit/s lossy cells, and reception overhead is effectively MDS (expected excess ≈ 1/255 of one symbol over GF(256), tighter than RaptorQ's ~+1–2 symbols per block at K = 56). Compute, not overhead, is therefore RLC's only scaling cost, and it does not bind below roughly gigabit line rates at these windows. A W ≈ 600 window at C8′s symbol rate is ~50 ms of traffic and ~0.7 MB retained — well within all three ceilings; the binding constraint at satellite-class RTT × high rate would be memory and decode, and (16.4) says such links need either a larger W or an honest admission that Σ g_i is not reachable there.

16.6 Predictions, Prerequisites, and the Experiment

Claim ledger.

DERIVED (no new measurement needed): - The resequencing bound (16.2): no per-path-affine in-order transport can exceed Σ_{i∈E} g_i; at C8′s heterogeneity E = {fast}, so 14.0 Mbit/s (fast-path-alone) is a ceiling for that whole design class — raptorpath's affinity scheduler and kernel MPTCP both sit under it (12.6 ≈ 12.6). - The fork-join penalty (16.1): striping atomic units across heterogeneous paths pays E[max] per unit; consistent with the measured 8.8 < 14.0. - The frontier rate: cross-path window coding + retention advances the in-order frontier at ≈ Σ g_i (16.3), window-by-window (16.4 sizing).

MEASURED (goal-gate, cited above): 8.8 / 12.6 / 14.0 / 23.9 Mbit/s; MPTCP 15.4 (C7) and 12.6 (C8); the eviction DNF (10/10 ⟶ 0/10, Section 15.7); the P9a switch hazard; no striping sender in window mode; sim-vs-production correspondence (Section 16.3).

MEASURED AFTER THE FACT (RWM Phase C, see Section 16.7 — this P1 prediction was tested and REFUTED): - **P1 — aggregation. REFUTED.** RWM at C8, 50 MB native bulk out-of-order (the H ⟶ ∞ corner) measured **11.9 Mbit/s** — 0.76× fast-path-alone (15.7), i.e. it does NOT cross the (16.2) ceiling; it lands at kernel-MPTCP parity (12.6). Out-of-order is 1.42× the in-order mean and far steadier (stdev 3.2 vs 6.9) — a real robustness gain — but not the aggregation- above-fast-path the prediction claimed. The raise-r companion (r ≈ 0.18) did not cross it either (7.9). Full numbers and mechanism: Section 16.7.

Superseded PREDICTION (kept for the record, now MEASURED-refuted above): - ~~**P1 — aggregation.** RWM at C8, 50 MB native bulk: completion goodput **strictly > 14.0 Mbit/s** — beyond the (16.2) ceiling of every per-path-affine in-order transport measured on this topology — trending toward Σ ≈ g_A + g_B.~~ - **P2 — no regression.** C7 symmetric control stays ≈ 23 Mbit/s: where E = {all}, affinity already aggregates, and RWM must not lose to it (the order-statistic gain ⟶ 0 on symmetric paths, so parity is the prediction, not improvement). - **P3 — the signature scaling law.** RWM's advantage over whole-unit affinity **grows with path heterogeneity** (sweep RTT_B and ε_B upward from C7-symmetric toward C8 and beyond): the E[max]-to-order-statistic gap widens with skew while both schemes coincide at zero skew. This monotonicity is the falsifiable fingerprint that distinguishes the order-statistic mechanism from generic "more tuning" — a flat or non-monotone curve refutes the formulation even if P1 happens to pass.

Implementation prerequisites, plainly. None of this exists today; the experiment requires building, in order:

1. **ARQ-layer retention** — a sent-data store in the window pipeline: sent source bytes retained until acked or T_cut(ρ)-aged (ρ = 1 ⟹ ack-only; Section 6.1 — one dial, no reliable/lossy mode), targeted retransmit from the store for aged SACK-confirmed holes on the best path, store-full ⟹ source backpressure; receiver never force-delivers past a hole on a reliable stream. The coding window keeps sliding freely (today's retransmit buffer holds metadata only — the data dies with window eviction; carrying the bytes is the change). The Section 14.27 P8 ledger (retained source, ACK-diff, targeted resend, sweep) is the natural donor machinery.
2. **A striping window sender** — window mode has none (16.3): stripe source + repairs ∝ g_i, reusing the proportional-goodput logic `Scheduler::schedule` already applies to block repairs.
3. **Frontier decode** — deliver the in-order prefix on any sufficient subset; the RlcDecoder's incremental GE already exposes exactly this (Section 15.2); what is new is wiring delivery to pivot-completion of the prefix rather than to per-block completion.

Workload: C8 per ADR-0051 (netem harness), `raptorpath perf` native objects, 50 MB (aggregate goodput) + 1.8 MB × N (completion distribution), seed 42, one arm at a time against the affinity baseline; C7 as the P2 control; then the P3 heterogeneity sweep.

What carries over unchanged from the earlier draft. Single-path bulk stays ARQ: with N = 1 there is nothing to pool — Σ g_i = g_1 — and the fountain/window overhead φ is wasted wire against ARQ's retransmit-exactly- the-erasures, whose mid-stream recovery is completion-free (Section 14.26). The Bulk strategy therefore remains **path-count dependent**: N = 1 ⟶ r* = 0 pure ARQ (the χ glide); N ≥ 2 heterogeneous ⟶ cross-path window coding (RWM) — the same result as before, with "rateless fountain, out-of-order" replaced by its implementable form.

16.7 The Reorder Horizon H as the Aggregation Dial; Ordering as a Delivery Policy

Section 16.2 asserted, unqualified, that "in-order delivery is not the bottleneck" — cross-path window coding advances the frontier at Σ g_i. RWM Phase B measured the qualifier the claim was missing.

MEASURED (goal-gate, RWM Phase B — the striping placement law §16.3, built and run at C8, seed 42):

```
symmetric  C7 (c2+c2):  21.7 Mbit/s = 1.41× fast-path-alone (15.4)  — aggregates
heterog.   C8 (c2+c3):  12.5 Mbit/s = 0.81× fast-path-alone (15.4)  — FAILS
```

The symmetric win proves the striping MECHANISM is sound; the heterogeneous failure localizes the missing assumption. §16.2′s frontier-at-Σg holds only when the window is **rateless-fungible** — the frontier advances on ANY sufficient K_h(1+φ) symbols, so no symbol is a specific position anyone must wait for. At bulk's operating point r ≈ ε (~2%, loss-matched), the sliding window is **systematic** (source striped in sequence order, tiny redundancy): a source symbol placed on the slow path IS a specific in-order position, and the fast path lacks the coded degrees of freedom to decode around it. The frontier is then fork-join (§16.1 regime 1, the §16.2 bound), and the aggregate collapses to the order-eligible set E = {fast} = 14.0. This **reconciles the section's two prior framings**, each of which was one regime of a single dial (DERIVED):

- *"out-of-order is the unlock"* (the pre-§16 draft) — TRUE at low r / for objects;
- *"in-order is fine; per-path affinity is the bottleneck"* (§16.2) — TRUE only in the rateless (high-r) regime, where the window is fungible and affinity is the last thing binding.

H, the reorder horizon, is the continuous dial. §16.2′s eligibility set E = {i : D_i − D_min ≤ H} already contains it: H is the delivery-latency budget the CONSUMER tolerates before a lagging path's unit counts as a hole. It is not binary (DERIVED):

- Small H (tight latency) ⟹ the slow path is excluded, E = {fast}, aggregate = fork-join/MPTCP parity — **MEASURED 12.5 at C8**.
- Large H ⟹ the slow path is admitted, completion ⟶ K / Σ_{all} g_i.
- **Out-of-order delivery is simply H ⟶ ∞** — the limit of the same dial, not a separate mechanism.

The equivalence that makes this precise (DERIVED). For a bounded OBJECT, these two are IDENTICAL in completion time:

1. deliver each decoded symbol immediately, out of order, and reassemble by offset;
2. deliver strictly in order through a reorder buffer deep enough to hold to completion.

Both finish at **decode-on-total** — the instant the last still-missing symbol decodes anywhere, on any path. The in-order frontier costs throughput ONLY for a consumer that must eat bytes incrementally, in order, at low latency (an inner TCP byte stream, live media). A file has no such consumer: nothing reads offset k before the file is whole. Phase B's 12.5 therefore measured the frontier under the WRONG completion metric for an object — it imposed a small-H incremental-consumer contract on a workload whose correct metric is decode-on-total (H = ∞). **The L0 visualizer, whose completion metric IS decode-on-total, already shows ×1.18 at C8** (goal-gate L0) — same topology, same code, the H ⟶ ∞ metric.

Two knobs buy heterogeneous aggregation; δ picks between them. Fungibility can be bought with latency OR with bandwidth (DERIVED):

knob	what it buys	cost	free in	right for
H	admit the slow path by tolerating its lag (H→∞ = out-of-order = decode-on-total)	LATENCY	bandwidth	files / loose-δ (bulk objects)
r	make the window rateless-fungible so no symbol is a fixed position (raise r to K_h(1+φ), §16.5)	BANDWIDTH (≈ slow path's share ≈16% @C8)	latency	tight-δ streams (live media, TCP-in-tunnel)

At C8 the slow path carries ≈ g_B/Σg ≈ 16% of the symbols, so making the window rateless-fungible costs ≈ 16% repair overhead (the §16.5 K_h(1+φ) provisioning) — paid in bandwidth, buying aggregation with NO added latency, so even a tight-δ in-order stream can aggregate. Conversely H ⟶ ∞ buys the same aggregation FREE in bandwidth, paid in delivery latency a file does not feel. This is one **(H, r) surface**, and the triangle's **δ selects the operating point**: loose δ ⟹ raise H (out-of-order the natural limit); tight δ ⟹ raise r (§16.5). The multipath dial collapses into the same (δ, ρ, r) triangle as the rest of §15/§16, with H a fourth axis dual to r through the fungibility each buys.

Ordering is a per-stream delivery POLICY, orthogonal to the coding triangle. It composes with the reliability policy ρ (§15.7) into four real, all-useful modes (DERIVED):

	reliable (ρ=1, retain)	lossy (ρ<1, evict)
ordered (H = ∞)	file / TCP-in-tunnel (byte stream — must wait)	ordered media (live video, skip stale)
unordered (H = 0 hold)	reliable messaging / objects (reassemble-by-offset, all delivered)	datagram / telemetry (fire-and-forget, newest wins)

The two readings of H are dual: the reorder HOLD you impose at the receiver (H = 0 unordered ↔ H = ∞ strict in-order) is the same quantity as the delivery LAG you tolerate from a path (the eligibility H of §16.2), read from the receiver vs the scheduler side. **Ordering and multipath eligibility are one horizon seen from two workloads.** And crucially **unordered is the SIMPLER implementation**: it REMOVES the reorder buffer (deliver each decoded unit the instant it decodes) rather than adding machinery — reinforcing the profiles-as-parameters thesis (§16.4). RWM Phase C implements exactly this general unordered-delivery capability on the reliable window as a delivery policy flag (off = today's in-order); the native object API is its first consumer (reassemble-by-offset), but message / datagram / RPC / telemetry streams are equally served.

Phase C result (MEASURED, goal-gate RWM Phase C — native `perf`, C8 = c2+c3, 50 MB × 8, seed 42, floor-free binary; single-path fast-alone 15.68 Mbit/s this binary). The prediction that the H ⟶ ∞ corner would **strictly beat fast-path-alone** is **REFUTED**:

C8 in-order (H bounded)	8.4 Mbit/s mean / 8.1 median	stdev 6.9	(8 runs)
C8 out-of-order (H→∞)	11.9 Mbit/s mean / 12.0 median	stdev 3.2	(8 runs)
fast-path-alone (c2)	15.7 Mbit/s		
C7 symmetric o-o-o (ctl)	21.6 Mbit/s	stdev 0.5	(3 runs)

Three things the numbers say, honestly:

1. **H ⟶ ∞ does NOT reach Σ g_i, and does not beat fast-alone.** Out-of-order completion goodput is 11.9 Mbit/s — 0.76× the single fast path, ≈ kernel- MPTCP / whole-block-affinity parity (12.6). The §16.1(3)/§16.2 picture that a bounded object at H ⟶ ∞ completes at K/

Σ g_i is **not realized at L1**: the slow path's *source* symbols still gate decode-on-total (at bulk's systematic r the window is not fungible, so the fast path cannot reconstruct them), and the straggler drag holds the aggregate below one fast path. The earlier expectation (and the L0 sim's ×1.18) overstated what H alone buys on a real heterogeneous link.

2. **The equivalence holds in direction, not in constant.** Out-of-order and in-order-with-retention are *supposed* to be identical (both decode-on- total). Measured, out-of-order is **1.42× the in-order mean and ~2× lower variance** (stdev 3.2 vs 6.9). The gap is implementation overhead, not theory: the in-order reorder buffer accumulates the entire out-of-order suffix behind each hole and drains it in bursts, and its recovery interacts with the CC/ARQ timing more erratically; removing the buffer (H = 0 hold, deliver-on-decode) removes that overhead and its tail. So out-of-order is the **more robust** realization of decode-on-total, but it moves the median from 8 to 12, not past 15 — it buys stability, not aggregation.
3. **The r knob did not unlock it either (at the level tried).** The companion raise-r arm — in-order frontier, per-symbol repair floor r ≈ 0.18 (the slow path's symbol share) — measured **7.9 Mbit/s**, no better than r ≈ 0 (8.4): forcing 18% repair added straggler load without making the window fungible enough to recover the slow-path source positions from fast-path repairs. Raising r on a genuinely congested lossy path spends bandwidth the straggler cannot afford (the same reason a blanket reactive-repair floor was measured to *regress* C8 14⟶9, goal-gate Phase C). Whether a much larger r (with repairs pinned to the slow path so they are fungible degrees of freedom, not straggler load) crosses fast-alone is **open** — neither knob crossed it here.

Regression control (MEASURED). C7 symmetric out-of-order = 21.6 Mbit/s (≈ Phase B's 21.7), stdev 0.5 — where the paths match there is no straggler, the order-statistic gain is zero, and out-of-order neither helps nor hurts. The mechanism is sound; C8 heterogeneity is where both knobs fall short.

Standing interpretation. Out-of-order delivery is a correct, useful, lower-variance *general* capability (the four-mode table above — objects, messaging, datagram), and it is the right delivery policy for bulk objects. But at C8 it does **not** deliver the heterogeneous aggregation the earlier draft and the L0 sim advertised: the H ⟶ ∞ corner lands at MPTCP parity, not above the fast path. The honest §16 position is therefore weaker than "out-of-order is the unlock" and weaker than "either knob aggregates" — it is: *for a file, deliver out-of-order (it is simpler and more stable); the heterogeneous-aggregation-above-fast-path result is unproven on this stack by either H or a modest r, and is a measured open problem, not a demonstrated win.*

Oracle adjudication of the L0/L1 contradiction (formula- AND sim-independent Monte-Carlo; `raptorpath-math/tests/multipath_oracle.rs`). The L0 sim said ×1.18, L1 measured ×0.76 — a hard contradiction. An independent oracle (models per-path capacity + one-way delay + GE loss, striped placement, fungible repairs over a sliding horizon *with eviction*, cross-path ARQ, in-order frontier decode; calls none of the model formulas or the sim) was run at the exact C8 netem params. It reproduces BOTH numbers as different transport configs, and thereby localizes the cause:

```
goodput ceiling  Σg_i/g_fast                              ×1.195   physical max
FUNGIBLE cross-path RWM, whole-object horizon (H→∞)       ×1.19    == L0 sim
ATOMIC path-affine (regime 1/2) + cross-path ARQ          ×0.92    sub-unity
ATOMIC + SAME-path recovery                               ×0.48–0.57
```

The physically-correct object case (fungible, H → ∞) AGGREGATES to the goodput ceiling — so §16.1(3)/§16.2′s K/Σg_i is REALIZABLE and heterogeneous aggregation is **NOT fundamentally fork-join-bounded**. L1′s ×0.76 sits inside the *broken-transport* band (between atomic-clean ×0.92 and atomic+same-path ×0.48–0.57), reproducible only by removing fungibility AND cross-path recovery. So the measured refutation is a **production limitation, not a theorem**: block/path-affine atomic units (§16.2(i)) + same-path/suppressed recovery + eviction (§16.2(ii)) — precisely the two caps §16.2 already names.

Lever ordering (oracle, independent-GE), best→worst — a correction to the intuition: (1) **fungible cross-path frontier decode (RWM) is DOMINANT** — atomic ×0.92 → fungible ×1.19; without it even perfect pull + cross-path recovery caps sub-unity. (2) **cross-path recovery** — same-path ×0.48 → cross-path ×0.92. (3) **placement (pull vs push) is NEGLIGIBLE** — ×1.190 vs ×1.190 in the fungible case, because fungible frontier fill masks the slow-path long pole. The r-sweep confirms the raise-r finding mechanistically: at H → ∞ the dual beats fast-alone at r = 0 already; raising r only helps a too-small coding window (crosses fast-alone at r ≈ 0.18 only when H ≈ 256). Thus Phase C's raise-r = 0.18 "no unlock" is expected — r cannot make a path-affine atomic unit fungible.

Updated §16 position (oracle-grounded). Symmetric aggregates (measured C7 ×1.71; oracle ×1.99). Heterogeneous OBJECT completion aggregates to ~×1.19 at C8 **iff** the transport realizes windowed fungible cross-path frontier decode — the §16.3 RWM (the EMPTY quadrant). Production BULK (RaptorQ 64 KB atomic blocks) is path-affine → oracle-capped at ~×0.92 even with perfect pull + cross-path recovery; the measured ×0.76 is that ceiling dragged down by same-path/suppressed recovery + eviction. The route to ×1.19 is the RWM subsystem (fungible sliding-window frontier decode + never-suppressed cross-path repair supply; the ADR-0046 idle-triggered recovery fix, now landed, is one prerequisite of the latter), NOT a placement change or a modest r. Heterogeneous aggregation-above-fast-path is **OPEN in production but proven ACHIEVABLE in principle (oracle ×1.19)** with a named, scoped mechanism — sharper than "unproven open problem." (Caveat: the oracle's per-path GE chains are independent; shared-bottleneck path CORRELATION — where pull placement may matter more — is not modeled and is flagged for real-trace validation.)

Appendix A: Summary of Key Formulas

```
   Channel (Section 2):
     e = p/(p+q)                          average loss rate          [probability]
     B = 1/q                              mean burst length          [symbols]
     P(burst ≥ t) = (1-q)^{t-1}           burst survival             [probability]

   Taper function (Section 4):
     τ*(t) = A* x (1-q)^t                 optimal taper function     [corrections/
symbol]
     A* = r* x q                          taper amplitude (ρ=100%)   [corrections/
symbol]

   Burst variance correction (Section 8.3):
     σ²_burst = 1 + 2(1-p-q)/(p+q)        burst variance inflation   [dimensionless]
     Var_GE(K) = W x e x (1-e) x σ²_burst loss count variance        [symbols^2]

   Optimal correction rate (Section 8.4):
     r* = max(0, e/(1-e) + z_{delta/e} x sqrt(e x s2_burst / (W x (1-e))))  [ratio]
```

```
     z_{delta/e} = normal_quantile(1 - delta/e)  (r* -> 0 smoothly as delta -> e)
     With codec: replace e with e_hat (see Section 9.2)
     P_fec = Phi(sqrt(W) x (r(1-e)-e) / sqrt(e(1-e)(r+s2_burst)))      [probability]
     Exact P_fec: transfer-matrix DP over the GE chain (Section 8.7)   [probability]

   Codec overhead (Section 9.2):
     e_codec_eff = e_codec x (1-(1-e)^W)   weighted codec overhead     [probability]
     e_hat = e + e_codec_eff               effective loss rate         [probability]

   Bulk completion-exposure glide (Section 14.26):
     chi(T_rem) = Phi_bar((T_rem - 1.5 x SRTT) / sigma_arq)            [probability]
     sigma_arq = max(4 x RTTVAR, SRTT/4)                               [seconds]
     delta_bulk = e_hat + (delta_tail - e_hat) x chi,  delta_tail=0.05 [probability]
     (chi = 0 mid-stream / unknown T_rem -> delta_bulk = e_hat -> r* = 0)

   Inner-feedback repair floor (Section 14.28):
     L_stall = min(1.5 x SRTT, max(0.2, SRTT))                         [seconds]
     T_arq = L_stall / t_sym                                           [slots]
     C(r) = sum_m (1-q)^{m-1} q x P(Poisson(T_arq x r(1-e)/(1+r)) >= m) [probability]
     S(r) = e x q x T_arq x (1 - C(r))                                 [stall
fraction]
     r_min = min{ r : S(r) <= theta },  theta = (SRTT/4) / L_stall     [ratio]
     rate = max(rate_glide, w x r_min),  w = inner-feedback weight     [ratio]
     (w = 0 everywhere by default: the L1 ablation refuted the premise
      post-14.27 — completion-neutral at C2, -28% at C3; opt-in knob)

   Three-variable optimization (Section 8.6):
     Taper cutoff: τ(t) = 0 for t > T_cut                (ρ<100%)
     Correction rate with cutoff: r = A x (1-(1-q)^{T_cut+1}) / q
     Given (δ, ρ) → r:  find T_cut from ρ, find A from δ, r = A(1-(1-q)^{T_cut+1})/q
     Given (r, ρ) → δ:  find T_cut from ρ, compute A from r, δ = e(1-P_fec)P_arq/ρ
     Given (r, δ) → ρ:  iterate T_cut until budget constraint met, ρ = recovery within
T_cut

   Per-symbol delivery (Section 6.3):
     P(on-time)   = (1-e) + e x P_fec                                  [probability]
     P(late)      = e x (1-P_fec) x P_arq                              [probability]
     P(lost)      = e x (1-P_fec) x (1-P_arq) = 1-ρ                    [probability]
     P_arq = 1 - (1-rho) / (e x (1-P_fec))                             [probability]

   Recovery latency (Section 3.4):
     t_sym = symbol_size / throughput      symbol transmission time    [seconds]
     t_fec = m x (1+r) / (r x (1-e)) x t_sym   FEC recovery time       [seconds]
     t_recovery_i = P_fec_i x t_fec_i + (1-P_fec_i) x L_arq_i          [seconds]
     P_lost(t) = e / [e + (1-e) x P(RTT>t)]  loss confidence           [probability]
     P(RTT > t) = 1 - Phi((t - SRTT) / RTTVAR)                         [probability]
     L_actual = min(t_fec, retransmit arrival)                         [seconds]

   Retransmit buffer (Section 6.1):
     B_max = ceil(ln(0.0001) / ln(1-q))                                [symbols]
     buffer_max = source_rate x (RTT + B_max/(r*(1-e))*t_sym)          [symbols,
rho=100%]
     buffer_max = source_rate x T_cut                                  [symbols,
rho<100%]

   Per-slot decision (Section 5.4):
     P_retx = P_lost(t)                                                (time-based
mixing)
     with probability P_retx:     send source retransmit               (immediate
```

```
decode)
     with probability 1-P_retx:   send repair symbol                    (FEC, any loss)
     Optional refinement: P_retx = P_lost(t) x (1 - e_burst)
       for long-burst scenarios (Appendix C.6)

   Congestion control (Section 12):
     Copa: rate = 1 / (d_copa x dq)                                     [symbols/sec]
     dq = RTT_current - RTT_min                                         [seconds]
     source_rate = total_rate / (1 + r*)                                [symbols/sec]
     correction_rate = total_rate x r* / (1 + r*)                       [symbols/sec]

   Multi-path scheduling (Section 13):
     E_i = RTT_i/2 + e_i x t_recovery_i                                 [seconds]
     B_eff_i = C_i / (1 + r_i)                                          [symbols/sec]
     e_combined = SUM(C_i x e_i) / SUM(C_i)                             [probability]
     deficit = SUM_{un-ACKed s}(e_s)                                    [expected
corrections]
     cross-path: r = e_source / (1 - e_correction)                      [ratio]
     minimize: w_lat x SUM(x_i x E_i) + w_bw x SUM(x_i x r_i)
     P(cross-path retx) = P_lost(t, e_src)
     P(both paths fail) = e_src x e_retx                                [probability]
```

Appendix B: Related Work

ACK-Based vs NACK-Based Recovery

Our model uses ACK-based (positive acknowledgment) feedback, the same approach that TCP has used successfully for over 40 years. In contrast, many real-time protocols (e.g., RTCP NACK [RFC4585]) use negative acknowledgment where the receiver explicitly reports losses.

Why ACK-based is more robust for unicast: - ACKs confirm what works; NACKs report what failed. If a NACK is lost, the sender never learns about the loss. If an ACK is lost, the sender simply retransmits after a timeout (self-healing). - SACK [RFC2018] provides the same information as NACKs (which symbols are missing) but derived from positive evidence (which symbols arrived). - The sender can infer losses from SACK gaps without requiring the receiver to detect and report them.

NACK-based approaches remain useful for multicast (one sender, many receivers) where ACK implosion is a concern [RFC4585]. For our unicast tunnel model, ACK+SACK is strictly superior.

Hybrid FEC-ARQ for Lossless Streaming

The closest prior work is Mehrotra, Li & Huang [Mehrotra2010], who solve the same core problem: minimize delivery delay for lossless, in-order streaming over a lossy channel using hybrid FEC+ARQ. Their key result: sometimes it is optimal to **preempt original data packets with FEC packets** — delaying new data to send proactive repair prevents an ARQ round-trip and reduces overall latency. This maps directly to our taper function concept.

Differences from our model: - They use a Markov Decision Process (MDP) formulation and compute the optimal policy numerically. We derive a closed-form taper function from the GE survival function, which is more directly implementable. - Their model doesn't account for burst correlation in the margin term (our σ^2_burst correction). - They don't address multi-protocol traffic classes or interleaving.

The precursor paper [Mehrotra2009] establishes the basic hybrid FEC-ARQ protocol framework.

Streaming Codes over Gilbert-Elliott Channels
Vajha et al. [Vajha2020] provide the first **analytical** (not simulation-only) bounds on block-erasure probability for streaming codes over GE channels. Previously, streaming codes were designed for a simplified proxy channel (the delay-constrained sliding window model from [Badr2017]) and then evaluated via simulation on GE. Their upper and lower bounds could be used to verify our P_fec predictions without simulation.

Very recent work [RLC_GE2025] analyzes Random Linear Codes (our RLC backend) specifically over GE channels, providing analytical performance characterization that directly applies to our system. A related result [Vajha2020b] formalizes the sliding window approximation of the GE channel used in streaming code design.

The foundational work on delay-constrained burst erasure correction [Martinian2004] established the theoretical framework that later streaming code constructions build upon. Krishnan & Ramkumar [Krishnan2020] provide the simplest rate-optimal streaming code construction (staggered diagonal embedding), demonstrating that optimal streaming codes need not be complex. Tambur [Rudow2023] is a production-quality implementation of streaming codes for videoconferencing, demonstrating 26% fewer decode failures and 35% less redundancy bandwidth — validating that streaming code theory translates to practical gains.

Tail Latency Optimization with Proactive FEC
CloudBurst [Zeng2021] uses proactive FEC over multipath to reduce p99 tail latency by 60-75% in commodity datacenters. Different setting (datacenter LAN vs our WAN/wireless focus) but the same core insight: proactive FEC-coded packets spread across paths, recovered from the first arrivals. They use rateless fountain codes; we use windowed RLC/streaming codes with a taper.

Fork-Join Queues, Coded Queueing, and Resequencing Delay
Section 16′s three-regime formulation grounds each multipath delivery discipline in a known queueing-theoretic object.

Fork-join queues. Nelson & Tantawi [Nelson1988] give the classic approximate analysis of fork/join synchronisation in parallel queues — the mean response of a job forked to K servers grows with the *maximum* of the branch times (harmonic growth for homogeneous exponential branches). This is Section 16.1′s regime (1): a delivery unit striped across paths completes at the max over the paths it touches, and heterogeneity inflates E[max] well past any single path's mean.

Coded queueing / redundancy for latency. Joshi, Liu & Soljanin [Joshi2014] analyse content download from (n, k) coded storage as a fork-join system where the job completes when ANY k of n branches finish — the k-th order statistic replaces the maximum, trading storage/traffic redundancy for latency. Joshi, Soljanin & Wornell [Joshi2017] generalise to full/partial forking and quantify when redundancy reduces both mean latency and cost. Section 16.1′s regime (3) is the transport-layer analogue: rateless symbols poured over N paths make completion the K(1+φ)-th order statistic of the superposed arrival process, and the E[max] →

interior-order-statistic gap is exactly the coded-queueing gain, growing with branch heterogeneity.

Resequencing delay. Xia & Tse [Xia2003] analyse the resequencing buffer of reliable protocols under out-of-order arrivals — packets delivered over parallel channels wait for the in-order point to catch up. Section 16.1′s regime (2) (whole-unit path affinity + in-order release) is this queue at block granularity: the delivery frontier is a running max over per-unit completion times, which is why raptorpath's affinity scheduler and kernel MPTCP (the same queue with unit = packet) measure identically at C8 (12.6 Mbit/s parity).

In-order delivery of transport-layer coding. Cloud, Leith & Médard [Cloud2014] analyse the in-order delivery delay of a streaming code that inserts coded packets into the stream — showing coding shrinks head-of-line waits without abandoning in-order semantics. This is the direct antecedent of Section 16.2′s claim that in-order delivery is NOT the aggregation bottleneck once repairs are fungible across the window: the frontier advances on any sufficient subset, at the pooled arrival rate.

Joint FEC/ARQ under Gilbert-Elliott

Razavi et al. [Razavi2008] develop adaptive heuristic algorithms that jointly select FEC redundancy and ARQ persistence under GE channel conditions, applied to video streaming. They search the joint parameter space numerically. Our approach is more principled: the GE parameters directly determine the taper shape, and the tail latency constraint determines the amplitude analytically.

Water-Filling and Optimal Resource Allocation

The water-filling principle [Gallager1968] — allocating resources proportional to channel quality — is foundational in information theory. Our taper function applies this principle in the time domain: allocate correction density proportional to the conditional loss probability (1-q)^t. While water-filling is well-known for power allocation in OFDM, its application to correction symbol density matching the GE burst survival function appears novel.

What This Model Contributes

Aspect	Prior work	This model
Optimization target	Throughput, avg delay, or weighted cost	Tail latency only (= tail loss under 100% reliability)
FEC/ARQ balance	MDP or heuristic search	Closed-form from GE params + tail constraint
Taper function	Not formalized	GE survival function $\tau(t) = A(1-q)^t$
Burst correction	Not addressed	$\sigma^2_{burst} = 1+2(1-p-q)/(p+q)$
Protocol hint	Separate FEC/ARQ tuning knobs	Single knob: tail latency target δ
Correction rate formula	Numerical	$r^* = \varepsilon/(1-\varepsilon) + z_\delta\sqrt{(\varepsilon\cdot\sigma^2_{burst}/(W(1-\varepsilon)))}$
Feedback mechanism	NACK-based or separate FEC/ARQ	Unified correction symbols with ACK+SACK

Appendix C: Model Extensions from Related Work

The following extensions are motivated by concrete results from related research and represent improvements over the base model in Sections 1-11.

C.1 Correction Symbol Preemption of Source Data [Mehrotra2010] (resolved)

Mehrotra & Li show via MDP that during detected bursts, the optimal policy shifts from sending new source data to sending correction symbols. Our model produces the same behavior through two mechanisms:

1. **P_lost(t) during bursts**: Since the ACK hasn't had time to return (RTT hasn't elapsed), P_lost stays at the base rate ε — almost all correction slots are FEC repair. This is exactly Mehrotra's recommendation: send repair (not retransmit) during bursts.
2. **P_lost(t) after bursts**: Once ACKs reveal gaps (after RTT), P_lost rises to ~1 for lost symbols — correction slots switch to retransmit. Immediate decode at receiver, targeted recovery.
3. **Source preemption via r***: When BOCD detects increased ε, the taper amplitude A* increases, and $\tau(t)$ naturally exceeds 1.0 at small t. This means more correction slots than source slots — source is effectively paused. No special $\tau > 1.0$ rule needed; it emerges from the formula.

The P_lost model and Mehrotra's MDP agree on the optimal policy. The difference is derivation: we arrive at the answer from a Bayesian posterior (P_lost) and rate adaptation (r*), while Mehrotra uses dynamic programming.

C.2 Information Debt for Exact P_fec [RLC_GE2025]

The base model uses a normal approximation for P_fec. The information debt framework tracks the running deficit between received and needed symbols:

```
I_d(t) = symbols_needed(t) - symbols_received(t)
```

Recovery occurs when I_d returns to 0. The slot error probability is:

```
p_e = E{error_slots} / E{debt_cycle_length}
```

where error_slots counts time steps where $I_d \geq \zeta$ (the maximum recoverable debt, determined by window size and code rate).

Key finding: For systematic codes, successfully receiving a source symbol can sometimes **reduce** decodability of earlier lost symbols through Gaussian elimination interactions. Our normal approximation misses this effect.

Extension: Replace the normal-approximation P_fec with debt-tracking:

```
Current model:  P_fec ≈ 1 - Φ(-z) where z depends on (e, W, σ²_burst)
Extended model: P_fec = 1 - p_e where p_e from debt Markov chain
```

The debt Markov chain has state space $\{0, 1, \ldots, \zeta\}$ with transition probabilities depending on GE state and repair density. This is computable (finite-state Markov chain) though not as simple as the closed-form formula.

Status: The normal formula is used in the paper for analytical insight. For implementation, the exact $O(W^2)$ transfer matrix computation (Section 8.7, implemented as `p_fec_exact`) provides the same precision as the debt model. Both are finite-state Markov chain computations over

the GE channel — the debt model tracks decoder state, the transfer matrix tracks the loss-minus-repair deficit. Either gives exact P_fec for implementation use.

C.3 Analytical P_fec Bounds [Vajha2020]

Vajha et al. derive upper and lower bounds on block-erasure probability for streaming codes over GE without simulation. Their bounds depend on: - GE parameters (p, q) - Code rate R = K/N - Decoding delay constraint T

Extension to verification (Section 11):

```
For our taper with rate r* and window W:
  Effective code rate R = 1/(1+r*)
  Delay constraint T = W

Vajha lower bound ≤ P(erasure) ≤ Vajha upper bound

If our predicted P_fec falls within these bounds: ✓ model validated
If not: normal approximation is inadequate → use debt model (C.2)
```

This gives us an analytical verification path that complements simulation. Every verification method strengthens confidence in the model — simulation validates end-to-end behavior, Vajha bounds validate the P_fec formula analytically, and the exact $O(W^2)$ computation validates the normal approximation numerically.

C.4 Multipath Diversity Gain [Zeng2021]

CloudBurst sends FEC-coded packets across multiple paths and recovers from whichever path delivers first. This is a **diversity gain**: the same repair on two independent paths has survival probability `1 - ε₁ε₂` instead of `1 - ε₁`.

Extension: For multipath, the effective repair arrival rate is higher than single-path. With M independent paths of loss rates $\varepsilon_1 \ldots \varepsilon_m$:

```
P(repair arrives on at least one path) = 1 - Π(e_i)

Single path:  P(arrive) = 1-e
Dual path:    P(arrive) = 1-e^2 ≈ 1 for small e
```

This means the multipath taper can use a **lower amplitude** A than single-path for the same P_fec target. The optimal multipath taper:

```
τ_multi(t) = A_multi x (1-q)^t

where A_multi = A_single x (1-e) / (1-Π(e_i))
```

For two paths with 5% loss each: A_multi = A_single × 0.95/0.9975 ≈ 0.95 × A_single. Modest gain for similar paths, but significant when paths have different characteristics (one lossy WiFi, one reliable Ethernet).

Not applicable to our model. CloudBurst duplicates the same repair symbol across paths. Our model uses shared-buffer cross-path retransmit instead (Section 13.10): each path generates its own corrections, and the shared retransmit buffer enables cross-path recovery. The diversity gain P(both fail) = ε_A × ε_B is already captured in Section 13.10 without repair duplication.

C.5 DCSW Worst-Case Taper Floor

The Delay-Constrained Sliding Window model [Badr2017], [Fong2019] provides a **hard guarantee**: within any window of W symbols, at most B consecutive erasures or N random erasures can occur, and recovery must happen within delay T.

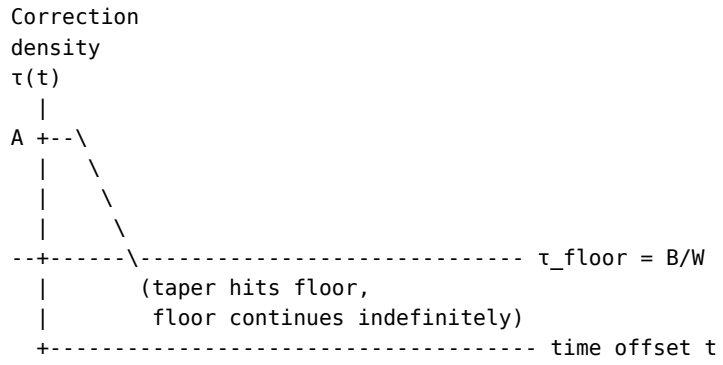
The streaming capacity for this model is C(T,B) = T/(T+B) [Badr2017].

Extension: Regardless of the probabilistic taper, enforce a minimum correction density that survives the worst-case DCSW pattern:

```
τ_floor = B / W                    (enough to survive one full burst per window)

τ*(t) = max(A x (1-q)^t, τ_floor)  (probabilistic taper with hard floor)
```

The floor ensures that even if the GE estimator underestimates burst length, we always have enough correction to survive at least B consecutive erasures.



Not applicable to our model. The DCSW floor is a hard guarantee from a worst-case adversarial channel model. Our model already handles burst protection through multiple mechanisms: (1) the taper's front-loaded density exceeds B/W for any reasonable r*, (2) the retransmit buffer holds burst-lost symbols for ARQ recovery after detection, (3) cross-path diversity on multipath, and (4) backpressure guarantees delivery for ρ=100%. Even if a burst overwhelms the taper entirely, ARQ kicks in — the retransmit buffer IS the hard guarantee. The DCSW floor is redundant.

Corrected total correction rate with floor (for reference):

```
r* = max(A/q, τ_floor x W) / W

In practice: A/q dominates when loss is high (large A).
Floor dominates when loss is low but bursts are long (large B, small e).
```

C.6 Burst-Aware Channel Discount (considered refinement)

The per-slot decision can optionally include a channel-state discount:

```
P_retx = P_lost(t) x (1 - e_burst)
```

where e_burst is a fast-reacting estimate of current channel loss (from GE state or fast EWMA). This suppresses retransmit during ongoing bursts, where repair is more flexible (covers any lost symbol).

Two separate loss estimates serve different purposes:

- **ε** (long-run, from BOCD): Bayesian prior for per-symbol loss confidence in P_lost

- **ε_burst** (fast, from GE state): current channel condition discount

This was considered for the primary model but deemed unnecessary because:

1. P_lost(t) alone handles the primary FEC/ARQ regulation via timing
2. Before burst detection (t < RTT): P_lost is already low ⟶ mostly repair
3. After burst detection (t > RTT): burst is usually over ⟶ ε_burst has dropped
4. The improvement is only significant for long bursts still ongoing at detection time (~1ms improvement)

The source/correction ratio was also considered for fast burst adjustment (two-speed taper: A_effective = A_baseline x (1 + burst_boost)), but the existing BOCD rate adaptation handles this with negligible delay.

Preserved here for future reference if extreme burst scenarios require it.

C.7 Summary of Extensions

Extension	From	Effect on correction rate	Effect on P_fec accuracy
Correction preemption	[Mehrotra2010]	Reduces delay during bursts	Improves burst recovery
Information debt	[RLC_GE2025]	More precise	Exact (Markov chain)
Analytical bounds	[Vajha2020]	Verification only	Bounds, not point estimate
Multipath diversity	[Zeng2021]	Reduces A_multi	Higher for same budget
DCSW taper floor	[Badr2017]	Hard minimum guarantee	Worst-case protection
Burst-aware discount	C.6	Modest improvement for long bursts	Suppresses retransmit during bursts

Appendix D: Open Questions

1. **Finite window truncation (resolved):** The taper is infinite-tailed but the encoder window has finite size W. After eviction, FEC repair is no longer possible — but the retransmit buffer (Section 5.3) still holds the exact source symbol for ARQ retransmission. The truncation error (1-q)^W only affects the FEC component; the unified model's ARQ fallback covers it.

2. **Multi-path (resolved):** Each path has its own taper, Copa, and GE estimator (Section 13). Unified stream per path preserves interleaving. Scheduler adjusts per-path source/correction ratio (latency mode only). Global correction deficit tracks outstanding corrections. Cross-path retransmit via shared buffer. Burst protection during scheduler ratio adjustment is largely resolved: P_lost timing + BOCD adapts within ~1ms (Section 13.7). Two-speed taper is an optional refinement for extreme scenarios (Appendix C.6).

3. **Interaction with congestion control (resolved):** Copa [Copa2018] controls total rate, taper controls source/correction split (Section 12). When source_rate = total_rate/(1+r*) drops below the app's minimum, the system signals back-pressure. Copa is preferred over BBR: no ProbeRTT, no FEC protection gaps, simpler formula, taper-compatible.

4. **Normal approximation validity (resolved).** The normal approximation uses the EXACT mean (We) and EXACT variance (We(1-e)s2_burst) from the GE model — only the distribution SHAPE is approximated as a bell curve. For W >> B (all practical scenarios: W=50,

B=2-3), the CLT makes this accurate. For edge cases (W close to B), the exact GE tail probability is computable via transfer matrix dynamic programming in O(W^2) — trivially cheap (2500 operations for W=50). The normal formula provides analytical insight (clean, closed-form); the exact computation is recommended for implementation and validation. IMPLEMENTED: Section 8.7 specifies the recursion; `p_fec_exact` / `compute_r_star_exact` in raptorpath-math implement it, verified against Monte Carlo and an independent-Binomial reference.

5. **Optimal retransmit timing (resolved):** Replaced by the P_lost(t) model (Section 3.4). No hard timeout — the repair/retransmit mix is determined probabilistically by P_lost(t) = ε / [ε + (1-ε) × P(RTT > t)]. This smoothly transitions from repair to retransmit as confidence in loss grows, naturally handles proactive retransmit at high loss rates, and minimizes expected waste at E[waste] = P_lost × (1 - P_lost).

Remaining Open Points (full audit)

The following were identified by systematic review of the paper. Rated by difficulty: LOW = can be resolved with a sentence or formula, MEDIUM = needs a paragraph or derivation, HIGH = needs new analysis or algorithm design.

Core model gaps (critical):

6. **RTT distribution for P_lost not specified (resolved).** (Section 3.4) [LOW] P_lost uses P(RTT > t) but never specifies the distribution. Assuming normal with SRTT and RTTVAR gives P(RTT > t) = 1 - Phi((t-SRTT)/RTTVAR). Needs one sentence.

7. **P_arq never defined (resolved).** (Section 6.3, 8.6, Appendix A) [MEDIUM] Derived from reliability target: P_arq = 1 - (1-rho)/(e x (1-P_fec)). See Section 6.3.

8. **P_fec: two contradictory models (resolved).** (Appendix E vs Section 8.2) [MEDIUM] Appendix E sections marked as preliminary (Poisson, superseded). Section 8.2 marked as canonical (Binomial/Normal). Progression preserved to show why per-symbol model fails.

9. **Poisson model error not characterized (resolved).** (Appendix E.4) [LOW] The transition from Poisson (Appendix E.3) to corrected (Section 8.2) says "too pessimistic" but doesn't explain why. The error: per-symbol accounting counts only the lost symbol's own taper (~r corrections) instead of all window-covering repairs (~rW) — a factor-of-W undercount. See E.4.

10. **r* formula: which version is canonical (resolved)?** (Section 8.4 vs 9.2) [LOW] Section 8.4 uses raw ε. Section 9.2 adds codec overhead ε_codec. The canonical formula should be r* with ε_hat = ε + ε_codec × P(decoder). Needs a clarifying note in Section 8.4.

11. **T_cut computation algorithm (resolved).** (Section 8.6) [MEDIUM] Binary search on T_cut. P(recovered) is monotone in T_cut. Algorithm added to Mode 1. Converges in ~20 iterations.

12. **Mode 3 convergence (resolved).** (Section 8.6) [MEDIUM] Binary search on T_cut with monotonicity in ρ and r. Algorithm added to Mode 3. Convergence guaranteed.

13. **t_recovery_i undefined in multipath context (resolved).** (Section 13.5) [LOW] E_i = RTT_i/2 + ε_i × t_recovery_i but t_recovery_i not defined for multipath. It should reference Section 3.4: t_recovery_i = P_fec_i × t_fec_i + (1-P_fec_i) × L_arq_i. One sentence.

Implementation details (medium priority):

14. **Retransmit buffer saturation (resolved).** (Section 6.1) [LOW] Resolved: dual-mechanism model — T_cut age eviction + buffer_max size backpressure. Buffer is NOT bounded by encoder window. See Section 6.1.

15. **SACK timing and format (resolved).** (Section 6.2) [MEDIUM] Per-packet ACK (0.6% overhead). 5 fields: cumulative_ack, sack_ranges (cumulative within T_cut), echo_timestamp, jitter_us (u32), cumulative_received. Gap pruning at T_cut. ACK loss self-healing via cumulative semantics.

16. **GE parameter initialization (resolved).** (Section 7.5) [LOW] Initial state: all counters = 0, start in Good state, use the Beta prior (weak uniform) until enough transitions observed. GE is_valid() already gates usage (existing code). One sentence.

17. **Copa parameter d tuning (resolved).** (Section 12.4) [MEDIUM] Renamed to d_copa to avoid confusion with tail latency delta. Default 0.5 sufficient. Adaptive d_copa (Copa+) is a negligible optimization (~1ms gain).

18. **Copa min_rtt refresh (resolved).** (Section 12.4) [LOW] Copa's natural oscillation refreshes min_rtt. Sliding window of 10s (same as BBR). If min_rtt seems stale (RTT consistently above by 2x), force a brief rate reduction. One paragraph.

19. **Back-pressure signaling (resolved).** (Section 12.7) [MEDIUM] Blocking write() like TCP — sender stops reading from source when buffer_max reached. Optional stats API for advanced applications. Implementation-specific, not part of core model.

20. **Scheduler burst protection floor (resolved).** (Section 13.7) [LOW] No floor needed. Global correction deficit + cross-path diversity handle burst protection when the scheduler reduces a path's correction ratio. The three options and floor discussion were removed.

21. **Cross-path retransmit path selection (resolved).** (Section 13.10) [LOW] No explicit path selection needed. Cross-path retransmit emerges from shared buffer + per-path P_lost. Paths with spare capacity naturally pull retransmits (work-stealing). Weighted path preference for latency-sensitive traffic noted as potential refinement.

Minor (notation, justification):

22. **z_d vs z_δ inconsistency (resolved).** (Section 8.4) [LOW] Line uses "z_d" and "d" where it should be "z_δ" and "δ".

23. **Beta decay 0.995 not justified (resolved).** (Section 7.3) [LOW] Standard value for slow-forgetting Bayesian update. Half-life ≈ 138 samples. Sensitivity is low — values 0.99-0.999 give similar results.

24. **BOCD "5-15 batches" — what's a batch (resolved)?** (Section 4.5) [LOW] A batch = one ACK feedback cycle. With per-batch ACKs at ~10-100ms intervals, 5-15 batches = 50ms-1.5s adaptation time.

25. **GE simplified model error bounds (resolved).** (Section 2.1) [MEDIUM] h_G=0, h_B=1 is exact for packet-level erasure (UDP checksums ensure no partial delivery). Not an approximation for our use case.

26. **P_arq missing from Appendix A (resolved).** [LOW] Added P_arq formula to Appendix A.

Appendix E: Preliminary Poisson Model (superseded)

These sections were originally Section 6.2-6.5. They present a Poisson model that gives unrealistic results. The corrected Binomial/Normal model is in Section 8.2. This appendix is preserved for pedagogical value — it shows why the simpler per-symbol approach fails.

E.1 FEC Recovery Probability (preliminary — superseded by Section 8.2)

Note: Sections E.1-E.4 develop a preliminary Poisson model that gives unrealistic results. The model is superseded by the corrected Binomial model in Section 8.2. The preliminary derivation is kept to show why the simpler per-symbol approach fails and motivate the per-window model.

Consider a symbol lost at position 0. Correction symbols generated at offsets t = 0, 1, 2, ... each have: - Probability $\tau(t)$ of being generated (fractional: may or may not generate one) - Probability $(1-\varepsilon)$ of surviving the channel

The expected number of correction symbols covering the lost position that arrive:

```
R(A, q) = Σ_{t=0}^{W-1} τ(t) x (1-e)
        = A x (1-e) x Σ_{t=0}^{W-1} (1-q)^t
        = A x (1-e) x (1 - (1-q)^W) / q
```

For large W (window much larger than burst length): (1-q)^W ≈ 0, so:

```
R(A, q) ≈ A x (1-e) / q = r x (1-e)
```

Recovery model: The number of useful correction symbols arriving is approximately Poisson(R). For FEC recovery, we need at least 1 repair symbol (plus codec overhead, see Section 9). Simplified to needing at least 1:

```
P_fec(A, q) = 1 - P(Poisson(R) = 0) = 1 - e^{-R}
```

E.2 Solving for A* (preliminary)

Substituting into the constraint:

```
e x (1 - P_fec) ≤ δ
e x e^{-R} ≤ δ
e^{-R} ≤ δ/e
-R ≤ ln(δ/e)
R ≥ ln(e/δ)                    (note: e > δ, so ln(e/δ) > 0)
```

Using $R \approx A \times (1-\varepsilon) / q$:

```
A x (1-e) / q ≥ ln(e/δ)

A* = q x ln(e/δ) / (1-e)

r* = A*/q = ln(e/δ) / (1-e)
```

E.3 The Optimal Correction Rate Formula (preliminary — see Section 8.4)

```
┌──────────────────────────────────────────┐
│                                          │
│   r* = ln(e/δ) / (1-e)                   │
│                                          │
│   where:                                 │
│     e = average loss rate (from BOCD)    │
│     δ = tail latency target              │
│     r* = optimal correction rate         │
│                                          │
└──────────────────────────────────────────┘
```

Properties: - r* depends only on ε and δ, not on q (burst length doesn't affect the TOTAL correction budget, only its distribution over time via the taper shape) - As δ ⟶ 0 (tighter tail): r* ⟶ ∞ (need infinite FEC for zero ARQ events) - As δ ⟶ ε (loose tail = every loss goes to ARQ): r* ⟶ 0 (no FEC needed) - As ε ⟶ 0 (perfect channel): r* ⟶ 0 (no correction needed)

The **taper amplitude** is:

```
A* = r* x q = q x ln(e/δ) / (1-e)
```

And the **complete taper function** is:

```
τ*(t) = A* x (1-q)^t = q x ln(e/δ) / (1-e) x (1-q)^t
```

E.4 Comparison with Information-Theoretic Minimum

The IT minimum (Shannon limit for the erasure channel [Shannon1948]) is:

```
r_IT = e / (1-e)
```

Why ε/(1-ε), not just ε? Correction symbols are also subject to channel loss. A correction sent to replace a lost source might itself be lost, needing another correction, which might also be lost:

```
e + e^2 + e^3 + ... = e / (1-e)
```

At 30% loss: need 43% overhead (not 30%), because 30% of corrections are lost too. The (1-ε) denominator IS this geometric series. See also Section 13.4.

Our optimal rate:

```
r* = ln(e/δ) / (1-e) = r_IT x ln(e/δ) / e = r_IT x ln(1/δ)/e + r_IT x ln(e)/e
```

The ratio r*/r_IT = ln(ε/δ)/ε. This is the **unavoidable overhead** of targeting a tail latency of δ — it's the price of proactive protection.

For ε = 0.025 (WiFi), δ = 1e-4 (Realtime):

```
r*/r_IT = ln(0.025/0.0001) / 0.025 = ln(250) / 0.025 = 5.52/0.025 = 221
```

This seems very high. Let's check: r_IT = 0.025/0.975 = 0.0256, so r* = 0.0256 × 221 = 5.66. That's 566% overhead — clearly too much.

The issue: The dominant error is not Poisson-vs-Binomial — it is per-symbol accounting. E.1 counts only the corrections attributed to the lost symbol by its own taper, Σ_t τ(t) = r per source symbol, so the expected help is R ≈ r(1-e) < 1 and the model concludes that even a single loss usually cannot be recovered. But every repair is a combination of the ENTIRE window (Section 3.2): while the lost symbol remains in the window, roughly rW repairs are

generated that all cover it. The corrected model in Section 8.2 counts repairs per window — Binomial(rW, 1-e) — and compares them against the per-window loss count K. (The secondary Poisson-vs-Binomial distinction matters far less than this factor-of-W undercount.)

References

Channel Models

- **[Gilbert1960]** E.N. Gilbert, "Capacity of a burst-noise channel," *Bell System Technical Journal*, vol. 39, pp. 1253-1265, 1960. The original two-state channel model.
- **[Elliott1963]** E.O. Elliott, "Estimates of error rates for codes on burst-noise channels," *Bell System Technical Journal*, vol. 42, pp. 1977-1997, 1963. Extension of Gilbert's model with per-state loss probabilities.

Estimation and Detection

- **[Adams2007]** R.P. Adams, D.J.C. MacKay, "Bayesian Online Changepoint Detection," arXiv:0710.3742, 2007. The BOCD algorithm used in Section 7.4 for regime-aware loss estimation. Maintains run-length distribution with O(r_max) per update.
- **[RFC3550]** H. Schulzrinne, S. Casner, R. Frederick, V. Jacobson, "RTP: A Transport Protocol for Real-Time Applications," IETF RFC 3550, 2003. Appendix A.8 defines the interarrival jitter calculation used in our estimator.
- **[RFC6298]** V. Paxson, M. Allman, J. Chu, M. Sargent, "Computing TCP's Retransmission Timer," IETF RFC 6298, 2011. Standard for TCP RTO computation using SRTT and RTTVAR.
- **[Abramowitz1964]** M. Abramowitz, I.A. Stegun, *Handbook of Mathematical Functions*, National Bureau of Standards, 1964. Rational approximation for the standard normal quantile (Section 26.2.23), used in our Beta quantile and z_δ computation.

FEC Codes

- **[RFC6330]** M. Luby, A. Shokrollahi, M. Watson, T. Stockhammer, L. Minder, "RaptorQ Forward Error Correction Scheme," IETF RFC 6330, 2012. Fountain code with ~1% decode overhead. Our RaptorQ backend.
- **[RFC8681]** V. Roca, B. Teibi, "Sliding Window Random Linear Code (RLC) Forward Erasure Correction (FEC) Schemes," IETF RFC 8681, 2020. Window-mode FEC over GF(2^8). Our RLC backend.
- **[Yu2025]** S. Yu, J. Yang, Q. Meng, L. Xu, "METTLE: Streaming Codes Based on SC-MET-LDGM," arXiv:2602.10020, 2025. Sparse random matrix streaming code. ~15% decode overhead at small windows.

Streaming Codes Theory

- **[Martinian2004]** E. Martinian, C.-E.W. Sundberg, "Burst erasure correction codes with low decoding delay," *IEEE Trans. Information Theory*, 2004. Foundational work on delay-constrained erasure correction.

- **[Badr2017]** A. Badr, P. Patil, A. Tan, A. Dey, "Layered Constructions for Low-Delay Streaming Codes," *IEEE Trans. Information Theory*, 2017, arXiv:1308.3827. Proves streaming capacity C(T,B) = T/(T+B). Layered burst+random construction.
- **[Fong2019]** S.L. Fong, A. Khisti, B. Li, A. Tan, "Optimal Streaming Codes for Channels with Burst and Arbitrary Erasures," *IEEE Trans. Information Theory*, vol. 65 no. 7, 2019, arXiv:1801.04241. Generalizes to adversarial model: C(T,B,N) = (T-N+1)/(T-N+1+B).
- **[Krishnan2020]** M.N. Krishnan, D. Ramkumar, "Simple Streaming Codes for Reliable, Low-Latency Communication," *IEEE Comm. Letters*, vol. 24 no. 2, 2020. Staggered diagonal embedding (SDE): simplest rate-optimal construction.
- **[Rudow2023]** M. Rudow et al., "Tambur: Efficient loss recovery for videoconferencing via streaming codes," NSDI, 2023. Production implementation with ML-based rate adaptation. github.com/Thesys-lab/tambur

Multipath and Scheduling

- **[Ferlin2016]** S. Ferlin et al., "BLEST: Blocking Estimation-based MPTCP Scheduler," IFIP Networking, 2016. Skip slow paths that would stall receiver.
- **[Xia2003]** Y. Xia, D.N.C. Tse, "Analysis on Packet Resequencing for Reliable Network Protocols," *IEEE INFOCOM*, San Francisco, 2003, pp. 990-1000. Resequencing-buffer delay of reliable protocols under out-of-order arrivals — the queue behind Section 16.1′s regime (2) bound.
- **[Cloud2014]** J. Cloud, D. Leith, M. Médard, "In-Order Delivery Delay of Transport Layer Coding," arXiv:1408.1440, 2014. Expected in-order delivery delay (and variance) when coded packets are inserted into a reliable stream — coding shrinks head-of-line waits without dropping in-order semantics (Section 16.2).

Fork-Join and Coded Queueing

- **[Nelson1988]** R. Nelson, A.N. Tantawi, "Approximate Analysis of Fork/Join Synchronization in Parallel Queues," *IEEE Trans. Computers*, vol. 37, no. 6, pp. 739-743, 1988. Classic fork-join response-time analysis: completion is the max over parallel branches (Section 16.1 regime (1)).
- **[Joshi2014]** G. Joshi, Y. Liu, E. Soljanin, "On the Delay-Storage Trade-off in Content Download from Coded Distributed Storage Systems," *IEEE JSAC*, vol. 32, no. 5, May 2014. (n, k) fork-join: download completes on ANY k of n coded branches — the k-th order statistic replaces the fork-join max (Section 16.1 regime (3)).
- **[Joshi2017]** G. Joshi, E. Soljanin, G.W. Wornell, "Efficient Redundancy Techniques for Latency Reduction in Cloud Systems," *ACM ToMPECS*, vol. 2, no. 2, 2017. arXiv:1508.03599. When and how much redundancy reduces latency and cost in (n, k) fork-join service; quantifies the E[max] vs order-statistic gap Section 16.1 uses.
- **[Facenda2022]** T. Facenda, M.N. Krishnan et al., "Error-correcting codes for low latency streaming over multiple link relay networks," arXiv:2201.06609, 2022. Introduces delay spectrum concept for per-link rate allocation.

Hybrid FEC-ARQ

- **[Mehrotra2010]** S. Mehrotra, J. Li, Y. Huang, "Optimizing FEC Transmission Strategy for Minimizing Delay in Lossless Sequential Streaming," *IEEE Trans. Multimedia*, 2010. MSR-TR-2010-134. Closest prior work: MDP-based optimization of when to send FEC vs data for lossless streaming. Key insight: preempting data with FEC can reduce latency.
- **[Mehrotra2009]** S. Mehrotra, J. Li, "A hybrid FEC-ARQ protocol for low-delay lossless sequential data streaming," *IEEE MMSP*, 2009. Precursor establishing the hybrid FEC-ARQ protocol framework.
- **[Razavi2008]** R. Razavi et al., "Performance Evaluation of Joint FEC and ARQ Optimization Heuristic Algorithms under Gilbert-Elliot Wireless Channel," *IEEE CCNC*, 2008. Adaptive joint FEC/ARQ parameter search under GE channels for video.

Streaming Code Analysis over GE

- **[Vajha2020]** M. Vajha, V. Ramkumar, M. Jhamtani, P.V. Kumar, "On the Performance Analysis of Streaming Codes over the Gilbert-Elliott Channel," arXiv:2005.06921, 2020. *ITW 2021*. First analytical bounds on block-erasure probability for streaming codes over GE channels, replacing simulation-only evaluation.
- **[RLC_GE2025]** "On the Analysis of Random Linear Streaming Codes in Stochastic Channels," arXiv:2509.01894, 2025. Analytical performance of RLC over GE channels — directly applicable to our RLC backend.

Tail Latency

- **[Zeng2021]** G. Zeng, L. Chen, B. Yi, K. Chen, "Optimizing Tail Latency in Commodity Datacenters using Forward Error Correction," arXiv:2110.15157, 2021. (CloudBurst) Proactive FEC over multipath reduces p99 latency by 60-75% in datacenters.

Congestion Control

- **[Copa2018]** V. Arun, H. Balakrishnan, "Copa: Practical Delay-Based Congestion Control for the Internet," NSDI 2018. Delay-based CC with natural queue draining (no ProbeRTT). Rate formula: rate = 1/(δ × dq). Recommended for raptorpath (Section 12).

Information Theory

- **[Shannon1948]** C.E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379-423, 623-656, 1948. The erasure channel capacity result C = 1-ε gives the IT minimum repair rate r_IT = ε/(1-ε) used throughout this paper.
- **[Gallager1968]** R.G. Gallager, *Information Theory and Reliable Communication*, John Wiley & Sons, 1968. Water-filling theorem for optimal resource allocation across channels.

TCP SACK

- **[RFC2018]** M. Mathis, J. Mahdavi, S. Floyd, A. Romanow, "TCP Selective Acknowledgment Options," IETF RFC 2018, 1996. Defines SACK for TCP: receiver reports non-contiguous blocks of received data, allowing sender to infer exactly which segments were lost. Our ACK+SACK feedback mechanism adapts this proven approach.

ECN and Multicast Feedback

- **[RFC3168]** K. Ramakrishnan, S. Floyd, D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP," IETF RFC 3168, 2001. ECN mechanism for router-signaled congestion without packet drops.
- **[RFC4585]** J. Ott, S. Wenger, N. Sato, C. Burmeister, J. Rey, "Extended RTP Profile for RTCP-Based Feedback," IETF RFC 4585, 2006. RTCP NACK-based feedback for RTP multicast — the multicast feedback mechanism where ACK implosion is relevant.

Sliding Window Channel Models

- **[Vajha2020b]** M. Vajha, V. Ramkumar, P.V. Kumar, "On Sliding Window Approximation of Gilbert-Elliott Channel for Delay Constrained Setting," arXiv:2005.06914, 2020. Formalizes the DCSW-to-GE approximation used in streaming code design.